

Sri Lanka Institute of Information Technology



SE3090 – Software Engineering Frameworks
Year 3, Semester 1- 2026

INTEGRATED FULL-STACK AND AGENTIC AI APPLICATION DEVELOPMENT

ChargeSync **Intelligent EV Charging Reservation and Recommendation** **Platform**

SE_034

Student ID	Student Name
IT24102295	Sandaru P.H.B.
IT24102953	Yasara R.P.M
IT24103921	Samarawickrama N. A. N. D
IT24103826	Ranaweera K.R.

Submission Date : 06.10.2026

Repository URL : <https://github.com/Naviya2/ChargeSync-Platform>

React URL: **chargesync-platform.vercel.app**

ASP.NET Core API or health URL: <https://chargesync-platform-backend.onrender.com> & <https://chargesync-platform-backend.onrender.com/health>

Swagger URL: <https://chargesync-platform-backend.onrender.com/swagger>

Agentic ai URL: <https://chargesync-platform-agentic-ai.onrender.com>

Android APK: [ChargeSync-v1.0.0 .apk](#)

Installation Instructions: [ChargeSync Installation Instruc.md](#)

Agentic AI setup or access information: [ChargeSync Agentic AI Setup and.md](#)

Startup Instructions: [ChargeSync Startup Instructions.md](#)

Demonstration video link:

https://drive.google.com/drive/folders/1QJri3icB0qgE4XrFcr52vE5_GA8Y3Mi?usp=sharing

Group Declaration

We declare that this report and the submitted ChargeSync application represent the work completed by the members of the SE3090 Assignment 1 group.

All group members contributed technically to the development, testing, documentation, integration and evaluation of the system. Individual responsibilities and technical contributions are documented in the Individual Report sections of this consolidated report.

Artificial Intelligence tools were used during permitted development activities in accordance with the SE3090 Assignment 1 AI Use Level 4 requirements. AI-assisted work was reviewed, tested, modified where necessary and verified by the relevant student. Individual AI usage logs and reflections are included in the respective Individual Report sections.

We confirm that no external AI assistant, chatbot, IDE copilot or agentic coding tool was used during the final demonstration and viva to answer questions, generate explanations or modify the submitted work.

We further confirm that confidential information, passwords, API keys and other secrets have not been intentionally included in the submitted repository or report.

Table of Contents

Group Declaration	3
1. Executive Summary.....	6
2. Introduction	7
2.1 Background.....	7
2.2 Project Purpose.....	7
2.3 Project Scope	7
2.4 Problem Statement.....	7
2.5 Proposed Solution.....	8
2.6 Objectives	8
2.7 Limitations.....	9
3. System Overview.....	9
3.1 Product Perspective	9
3.2 User Roles	9
3.3 Major Business Components	10
4. Requirements Analysis	11
5. Business Rules.....	15
6. System Workflow	16
7. Use Case Model.....	20
7.1 Use Case Diagram	23
8. System Architecture.....	24
8.1 High-Level ChargeSync Architecture.....	25
9. Database Design	25
ER Diagram	29
10. Diagrams.....	30
10.1 System Context Diagram.....	30
10.2 C4 Container Diagram.....	30
10.3 Agentic AI Workflow.....	31
11. ASP.Net Core Web API.....	31
11.1 Backend Architecture.....	31
11.2 API Responsibilities.....	34
12. API Endpoint	36

13. Swagger	39
14. React Web Application	40
React Evidence	41
15. Flutter Mobile Application	42
15.1 Driver Mobile Mode.....	42
15.2 Station Owner Operational Mobile Mode	42
Flutter Evidence.....	43
16. Agentic AI Report.....	44
Agentic AI Evidence.....	46
17. Integrated End-to-End Workflow	46
18. Third-Party Services.....	47
19. Testing Report.....	47
20. Performance Report.....	55
Performance Testing Evidence	57
21. Security Report.....	58
22. GIT And CI/CD	60
Project Evidence.....	62
23. Deployment Report	64
Deployment Evidence	67
24. Architecture Decision Records	67
25. Technical Report.....	69
26. Challenges and Solutions	71
27. Limitations and Future Improvements	72
28. References	72
Individual Reports	73
IT24102953 – Yasara R.P.M.....	73
IT24102295 - Sandaru P.H.B.	91
IT24103826 - Ranaweera K.R.....	107
IT24103921 - Samarawickrama N. A. N. D.....	125

1. Executive Summary

ChargeSync is an integrated intelligent electric vehicle charging reservation and recommendation platform developed for the SE3090 Software Engineering Frameworks Assignment 1. The system addresses practical challenges associated with locating compatible charging stations, reserving charging slots, managing station infrastructure, coordinating charging sessions and handling post-session billing and customer support.

The system follows an integrated full-stack architecture consisting of an ASP.NET Core Web API, PostgreSQL relational database, React web application, Flutter mobile application and an internal Agentic AI subsystem. Both the React and Flutter applications communicate through the same ASP.NET Core API and share the same PostgreSQL database, authentication mechanism, authorization rules and business logic. ChargeSync supports four primary user roles: EV Driver, Station Owner, Platform Administrator and Customer Support Manager. The Station Owner also has an operational mobile mode through the Flutter application for activities associated with the owner's station, including reservation verification, walk-in admission, session monitoring and physical meter recording.

The system is divided into four major business components. The first component manages vehicle registration and AI-based vehicle-to-charger compatibility. The second manages charging stations, chargers, operating hours and maintenance windows. The third manages reservations, conflict prevention, QR-based check-in, walk-in charging and AI-generated charging plans. The fourth manages charging sessions, energy calculation, invoice settlement, membership, loyalty and customer support.

The Agentic AI subsystem provides multiple specialized agents with distinct responsibilities. These include a Vehicle Compatibility Agent, Station Analysis Agent, Charging Recommendation and Planning Agent, and Validation and Support Agent. The agents collaborate through a controlled workflow involving structured planning, tool usage, validation, persistent workflow state and human approval for selected high-impact operations.

The platform also integrates external services including Maps for station discovery and location-related functionality, Cloudinary for storing station images and physical meter photographs.

A major design objective of ChargeSync is to support operational environments where direct vehicle-to-charger IoT telemetry and external payment gateways are not available. Therefore, the platform uses a hybrid operational model. Charging energy is initially calculated using charger output power and session duration, while the station owner can record a physical meter reading and supporting photograph. Wallet transactions are handled through an internal ledger, while cash payments can be recorded for on-site transactions. The project demonstrates the integration of modern software engineering practices including RESTful API development, relational database

design, authentication and authorization, responsive web and mobile applications, Agentic AI orchestration, automated testing, GitHub-based collaboration, CI/CD, deployment and technical documentation.

2. Introduction

2.1 Background

The increasing adoption of electric vehicles has created a growing requirement for reliable and accessible charging infrastructure. EV users need to identify compatible charging stations, determine charger availability, select suitable charging times and manage charging costs.

Traditional charging applications often focus primarily on station discovery and basic reservation functionality. However, practical charging environments require additional capabilities such as charger compatibility analysis, operating-hour restrictions, maintenance windows, reservation conflict prevention, on-site operational workflows, payment management and customer support.

ChargeSync was developed to provide an integrated platform that addresses these requirements through a combination of full-stack software engineering and Agentic AI.

2.2 Project Purpose

The purpose of ChargeSync is to provide an intelligent EV charging management platform that connects EV drivers with charging stations while supporting station owners and administrators in managing charging infrastructure and operational activities.

The platform combines a web application, mobile application, RESTful backend, relational database and Agentic AI subsystem into a single integrated system.

2.3 Project Scope

The project covers four major business components:

1. Vehicle and AI Compatibility Discovery.
2. Station, Charger and Operating-Hours Management.
3. Reservation and AI Charging Planning.
4. Charging Session, Payment, Loyalty and Support Management.

The system includes authentication, authorization, CRUD operations, business workflows, search and filtering, reservation conflict prevention, AI-assisted recommendations, human approval workflows, notifications, testing and deployment.

The system does not implement direct vehicle-to-charger IoT telemetry, an external payment gateway or grid load balancing.

2.4 Problem Statement

EV drivers can face several difficulties when planning a charging trip. A vehicle may not be compatible with a particular charger connector, available chargers may become occupied, stations

may have restricted operating hours, and scheduled maintenance may temporarily make chargers unavailable.

Reservation systems can also encounter conflicts when multiple users attempt to reserve the same charging slot. In addition, operational situations such as charging sessions approaching station closing time require human intervention because automatically accepting such reservations could result in incomplete charging sessions or operational conflicts.

Charging sessions present another challenge when direct IoT telemetry is unavailable. A system that relies only on theoretical calculations may not accurately represent the energy delivered by a charger because charging rates can vary during a session.

Furthermore, EV charging platforms need reliable mechanisms for handling invoices, wallet payments, cash transactions, loyalty programs and customer support.

ChargeSync addresses these challenges by combining deterministic software engineering rules with Agentic AI recommendations and controlled human approval.

2.5 Proposed Solution

ChargeSync provides a centralized EV charging platform through which drivers can manage vehicles, discover compatible stations, generate AI-assisted charging plans, reserve charging slots and monitor charging history.

Station Owners can manage stations, chargers, pricing, operating hours and maintenance schedules through the web portal. They can also use the Flutter mobile application in an operational mode for their specific station to verify reservations, admit walk-in customers, monitor charging sessions and record physical meter readings.

Platform Administrators manage system-wide records, station approvals, operational audits and selected high-impact AI approval requests.

Customer Support Managers manage support tickets, billing disputes and refund-related workflows.

The platform uses ASP.NET Core as the authoritative application layer. Both React and Flutter communicate with this backend. PostgreSQL stores the system's relational data, while the Agentic AI service is accessed internally by ASP.NET Core.

This architecture ensures that business rules are centralized and that both client applications operate on the same data and authorization rules.

2.6 Objectives

The main objectives are:

1. Develop a secure ASP.NET Core REST API.
2. Implement a normalized PostgreSQL database.
3. Develop a responsive React web application.
4. Develop a Flutter mobile application.
5. Implement JWT authentication and role-based authorization.
6. Provide four major business components.
7. Integrate Agentic AI into meaningful domain workflows.

8. Implement controlled AI tools and structured workflow state.
9. Provide deterministic validation of AI outputs.
10. Implement human approval for selected high-impact actions.
11. Implement meaningful third-party service integration.
12. Provide automated testing and CI through GitHub Actions.
13. Deploy the major system components.
14. Demonstrate a complete cross-platform workflow.

2.7 Limitations

The current implementation has the following limitations:

1. Direct charger IoT telemetry is not implemented.
2. External payment gateway integration is not included.
3. AI recommendations depend on the quality and availability of input data.
4. External services may experience availability or rate-limit failures.
5. Physical meter verification depends on accurate human input.
6. The system is limited to the business requirements defined for the assignment.

3. System Overview

3.1 Product Perspective

ChargeSync is a multi-client full-stack application consisting of:

- React web application.
- Flutter mobile application.
- ASP.NET Core Web API.
- PostgreSQL database.
- Python Agentic AI service.
- Third-party services.

Both React and Flutter applications use the same ASP.NET Core API.

The API acts as the authoritative application layer for authentication, authorization, validation and business rules.

3.2 User Roles

Role	Main Responsibilities	Main Interface

EV Driver	Manage vehicles, discover stations, request AI plans, create reservations, display QR codes, view sessions, invoices, loyalty and support	Flutter
Station Owner	Register and manage stations, chargers, pricing, operating hours, maintenance and analytics; operate assigned station functions on-site	React + Flutter
Platform Administrator	Manage users, vehicles, station approvals, system-wide records, audits, AI approvals and administrative operations	React
Customer Support Manager	Manage support tickets, billing disputes and refund workflows	React

3.3 Major Business Components

The ChargeSync system is divided into four major business components. Each component represents a core area of the EV charging platform and contains related business operations, data, APIs, and user workflows.

3.3.1 Functional Components

Function 1 - Vehicle and AI Compatibility Discovery

This component allows EV Drivers to register and manage their vehicles and determine whether a vehicle is compatible with available charging infrastructure.

Major functionality includes:

- Vehicle registration.
- Vehicle update and deletion.
- Connector type management.
- Battery capacity and charging rate management.
- Nearby station search.
- Compatibility calculation.
- Estimated charging time.
- Alternative station recommendations.
- Administrative vehicle management.

Function 2 - Station, Charger and Operating-Hours Management

This component manages charging infrastructure.

Major functionality includes:

- Station registration.
- Station approval.
- Charger creation and management.
- Connector configuration.
- Charging price configuration.
- Operating-hour configuration.
- Maintenance-window scheduling.
- Charger status management.
- Availability calculation.
- Station utilization analytics.

A station initially enters a pending state and requires Platform Administrator approval before becoming operational.

Function 3 - Reservation and AI Charging Planning

This component manages charging reservations and AI-assisted charging plans.

Major functionality includes:

- Charger slot reservation.
- Wallet-based advance deposit.
- Reservation conflict prevention.
- QR code generation.
- QR-based check-in.
- Walk-in charging.
- Reservation status tracking.
- Reservation history.
- Operational boundary approval.
- AI-generated charging plans.

Reservations that approach station closing time or a scheduled maintenance window within the configured operational buffer are placed into a human approval workflow.

Function 4 - Session, Payment, Loyalty and Support

This component manages the charging session after reservation or walk-in admission.

Major functionality includes:

- Charging session start.
- Charging session stop.
- Automatic energy calculation.
- Physical meter override.
- Meter photograph evidence.
- Invoice generation.
- Wallet settlement.
- Cash settlement.
- Membership subscriptions.
- Loyalty points.
- Reward redemption.
- Support ticket management.
- AI-assisted support triage.

4. Requirements Analysis

4.1 Functional Requirements

Authentication

- User registration.
- User login.
- User logout.
- Password hashing.
- JWT token authentication.
- Role-based authorization.

Station Management

- Register station.
- Update station.
- View station.
- Search stations.
- Approve station.
- Reject station.
- Suspend station.
- Manage chargers.
- Manage operating hours.
- Manage maintenance windows.

Vehicle Management

- Register vehicle.
- Update vehicle.
- Delete vehicle.
- View vehicle.
- Search vehicles.
- Check vehicle compatibility.

Reservation Management

- Search stations.
- Search available chargers.
- Create reservation.
- Update reservation.
- Cancel reservation.
- View reservation history.
- Generate QR information.
- Verify reservation through QR.
- Support walk-in charging.

Charging Session

- Start charging session.
- Monitor charging session.
- Record meter information.
- Stop charging session.
- Calculate energy.
- Generate invoice.
- Record settlement.

Support

- Create support request.
- Update support request.
- Assign support request.
- Change support status.
- Resolve support request.
- View support history.

Agentic AI

- Submit AI workflow.
- Generate workflow plan.

- Delegate tasks to specialized agents.
- Execute allow-listed tools.
- Validate AI outputs.
- Request authorized approval.
- Approve/reject/revise selected actions.
- Store workflow state.
- Produce execution summary.

4.2 Requirements Table

ID	Requirement
FR-01	Driver shall register vehicles.
FR-02	System shall calculate vehicle-charger compatibility.
FR-03	Driver shall search nearby compatible stations.
FR-04	Station Owner shall register stations.
FR-05	Administrator shall approve or reject stations.
FR-06	Station Owner shall manage chargers.
FR-07	Station Owner shall configure operating hours.
FR-08	Station Owner shall configure maintenance windows.
FR-09	System shall calculate charger availability.
FR-10	Driver shall create charging reservations.
FR-11	System shall prevent overlapping reservations.
FR-12	System shall generate reservation QR codes.
FR-13	Station Owner shall validate reservation QR codes through the mobile operational mode.
FR-14	System shall support walk-in charging.
FR-15	System shall generate AI charging plans.
FR-16	System shall support operational-boundary approval.
FR-17	System shall record charging sessions.
FR-18	System shall calculate baseline energy consumption.
FR-19	Station Owner shall record physical meter readings.
FR-20	System shall generate invoices.
FR-21	System shall support wallet and cash settlement.
FR-22	System shall manage loyalty points and rewards.
FR-23	System shall manage customer support tickets.
FR-24	Agentic AI shall perform controlled multi-step workflows.
FR-25	High-impact AI actions shall require authorized human approval.

4.3 Non-Functional Requirements

Security

- JWT authentication.
- Role-based authorization.
- Password hashing.
- Secure environment variables.

- Protected API endpoints.
- Input validation.
- Output validation.
- AI service access restricted through backend.

Performance

- Efficient database queries.
- Pagination for large datasets.
- Indexed frequently queried fields.
- Asynchronous API operations.
- Controlled AI request timeouts.
- Limited AI retries.

Usability

- Responsive React interface.
- Mobile-friendly Flutter interface.
- Clear validation messages.
- Loading states.
- Empty states.
- Error handling.

Reliability

- Global exception handling.
- Structured logging.
- Transaction management.
- AI safe-failure handling.
- Validation before high-impact actions.

Maintainability

- Layered backend architecture.
- Reusable React components.
- Reusable Flutter widgets.
- Service-based business logic.
- Data-access abstraction.
- Modular Agentic AI architecture.

ID	Requirement	Description
NFR-01	Security	The system shall protect user data and API access using authentication and authorization.
NFR-02	Performance	API operations should provide acceptable response times under expected workload.
NFR-03	Availability	Deployed services should remain accessible during evaluation.
NFR-04	Usability	Interfaces should be understandable and easy to navigate.
NFR-05	Maintainability	The system should use modular and maintainable architecture.
NFR-06	Scalability	The backend and database should support future extension.
NFR-07	Reliability	Failed operations should be handled safely and clearly.
NFR-08	Compatibility	React and Flutter should operate using the shared API.
NFR-09	Auditability	Important workflows should provide traceable execution information.

NFR-10	Data Integrity	Database constraints and validation should protect data consistency.
--------	----------------	--

5. Business Rules

The following business rules define the operational constraints and decision logic of the ChargeSync system. These rules are enforced primarily through the ASP.NET Core backend to maintain data consistency, authorization, availability, reservation integrity, and safe Agentic AI operation.

Station Approval Rule

A newly registered station must initially enter the **Pending** state.

The Platform Administrator must review the station registration and supporting information.

Only an approved station can become **Active** and participate in normal charging operations.

Rejected or suspended stations must not be available for new charging reservations.

Charger Availability Rule

A charger is considered available only when all required conditions are satisfied:

1. The station is **Active**.
2. The current time is within the configured operating hours.
3. The charger status permits charging.
4. The charger is not within a scheduled maintenance window.
5. No conflicting active reservation exists for the charger.

If any of these conditions are not satisfied, the charger must not be offered as available for a new reservation.

Reservation Conflict Rule

A charger is considered available only when all required conditions are satisfied:

1. The station is **Active**.
2. The current time is within the configured operating hours.
3. The charger status permits charging.
4. The charger is not within a scheduled maintenance window.
5. No conflicting active reservation exists for the charger.

If any of these conditions are not satisfied, the charger must not be offered as available for a new reservation.

AI Approval Rule

AI-generated recommendations must not directly execute high-impact operational actions without the required validation and authorized human approval.

The Agentic AI workflow must pause at the defined approval point until an authorized user approves, rejects, or requests revision.

This follows the assignment requirement that at least one high-impact Agentic AI action must require authorized human approval.

AI Validation and Safe-Failure Rule

Agentic AI outputs must be validated against defined schemas and deterministic business rules before they are accepted.

If an Agentic AI workflow cannot produce a valid structured result, fails deterministic validation, exceeds retry limits, encounters an unavailable tool, or produces an unsupported recommendation, the system must not execute the high-impact action.

The workflow must instead be marked as **Failed**, **Rejected**, or **Requires Revision**, and the failure must be recorded for auditing.

The system must maintain sufficient workflow state, validation results, errors, approval status, and final outcomes to support an auditable execution history.

6. System Workflow

This section describes the major operational workflows of the ChargeSync platform. The workflows demonstrate how users, React and Flutter applications, the ASP.NET Core Web API, PostgreSQL database, and Agentic AI services interact to complete the main business processes.

Overall System Workflow

The overall ChargeSync workflow is as follows:

1. The user logs into the ChargeSync application through the React or Flutter client.
2. The ASP.NET Core API authenticates the user and establishes the appropriate authorization context.
3. The system provides access to functionality according to the user's assigned role.
4. The Driver searches for available charging stations and chargers.
5. The system evaluates station status, operating hours, charger status, maintenance windows, and existing reservations.
6. The Driver selects a vehicle and the system checks vehicle–charger compatibility.
7. The Driver creates a reservation or requests an Agentic AI charging recommendation/charging plan.
8. When Agentic AI is required, the request is submitted to the Agentic AI workflow.

9. The AI workflow creates and maintains the required workflow state and processes the request through specialized agents and controlled tools.
10. AI-generated results are returned in a structured format and validated against deterministic business rules.
11. Where a high-impact action requires approval, the workflow enters an approval state.
12. An authorized user reviews, approves, rejects, or requests revision of the proposed action.
13. If approved and all validation conditions are satisfied, the ASP.NET Core backend executes the authorized operation.
14. The relevant reservation, charger, charging session, or AI workflow status is updated in PostgreSQL.
15. The Driver arrives at the station and completes reservation/check-in or an authorized walk-in process.
16. The charging session is started and relevant charging information is recorded.
17. When charging is completed, the system calculates the consumed energy according to the defined energy-calculation model.
18. An invoice is generated for the completed charging transaction where applicable.
19. Settlement, membership, and loyalty information are updated.
20. The Driver can view reservation, charging, invoice, loyalty, and support history.

Station Registration and Approval Workflow

The station registration workflow allows a Station Owner to register a new charging station and configure its operational information.

Workflow:

Station Owner → Register Station → Validate Details → Save as Pending → Platform Administrator Review → Approve/Reject → Active/Rejected → Charger Management → Operating Hours → Maintenance Windows → Availability

Process

1. The Station Owner submits the station registration information.
2. The ASP.NET Core API validates the submitted information.
3. The station is stored with **Pending** status.
4. The Platform Administrator reviews the station information.
5. The administrator either approves or rejects the station.
6. If approved, the station becomes **Active**.
7. If rejected, the station remains unavailable for normal charging operations.
8. After approval, the Station Owner can configure chargers.
9. The Station Owner defines operating hours.
10. Maintenance windows can be configured for chargers when required.
11. The system uses these configurations when calculating charger availability.

Reservation Workflow

The reservation workflow allows a Driver to find a suitable charger and reserve it for a selected period.

Process

1. The Driver searches for charging stations.
2. The system applies station and charger availability rules.
3. The Driver selects a registered vehicle.
4. The system checks whether the vehicle is compatible with the selected charger.
5. The system checks the selected charger's availability for the requested period.
6. The Driver submits the reservation request.
7. The ASP.NET Core backend validates the request and checks for reservation conflicts.
8. If all conditions are satisfied, the reservation is confirmed.
9. A QR/check-in reference is generated where applicable.
10. The Driver arrives at the station and checks in.
11. The reservation is verified.
12. A charging session can then be started.

Agentic AI Charging Recommendation and Planning Workflow

The Agentic AI workflow provides intelligent charging recommendations or charging plans while ensuring that AI decisions are controlled by deterministic validation and human approval where required.

Process

1. The Driver provides a charging objective, such as selecting a suitable charging option or generating a charging plan.
2. The request is sent to the ASP.NET Core API.
3. The backend creates and persists the Agentic AI workflow state.
4. The planning process determines the required steps and invokes the appropriate specialized agents.
5. Specialized agents perform different responsibilities related to the charging objective.
6. Agents interact only with approved and allow-listed tools.
7. Tool inputs and outputs are validated.
8. The agents produce structured results.
9. The backend performs deterministic business-rule validation on the AI-generated result.
10. If the result is invalid, unsupported, or unsafe, the workflow does not execute the proposed high-impact action.
11. If human approval is required, the workflow enters an Approval Required state.
12. An authorized user reviews the proposed action.
13. The authorized user can approve, reject, or request revision.
14. If approved, the backend executes the permitted operation.
15. The workflow state and final outcome are persisted in PostgreSQL.
16. The Driver receives the validated charging recommendation or charging plan.

AI Safe-Failure Path

AI Failure/Invalid Output → Validation Failure → No High-Impact Action → Mark Failed/Rejected/Requires Revision → Record Error and Workflow State → Return Safe Result

6.5 Charging Session Workflow

The charging session workflow manages the actual charging activity after a Driver arrives at a station.

Workflow:

Driver Arrives → Reservation/Walk-In Verification → Station & Charger Validation → Session Start → Charging → Meter/Energy Recording → Session Stop → Energy Calculation → Invoice Generation → Settlement → Loyalty Update → History

Process

1. The Driver arrives at the charging station.
2. The system verifies the reservation or authorized walk-in request.
3. The system validates the station and charger status.
4. The system confirms that the charger is permitted to start a charging session.
5. The charging session is started.
6. Relevant charging information is recorded during the session.
7. The Driver stops charging or the charging session reaches its completion condition.
8. The session is closed.
9. The system calculates the consumed energy according to the defined project calculation model.
10. The applicable charging price is determined.
11. An invoice is generated for the completed transaction.
12. Settlement information is recorded.
13. Applicable loyalty or membership information is updated.
14. The completed charging session and invoice information are stored in the user's history.
15. The charger status is updated according to its current condition.

Cross-Platform Integrated Workflow

ChargeSync supports an integrated workflow involving both client applications, the backend, database, and Agentic AI services.

A representative integrated workflow is:

Process

1. A Driver initiates a charging-related request from one client application.
2. The request is sent to the ASP.NET Core backend.
3. The backend stores the relevant request and workflow state in PostgreSQL.
4. If Agentic AI is required, the AI workflow processes the request.
5. The AI agents use controlled tools and generate a structured recommendation or proposed action.
6. The backend validates the AI result against deterministic business rules.
7. If the proposed operation is high-impact, the workflow enters an approval state.
8. An authorized user reviews the request from the appropriate client application.
9. The authorized user approves, rejects, or requests revision.
10. The decision is sent to the ASP.NET Core backend.

11. The backend executes the approved operation only when all validation conditions are satisfied.
12. PostgreSQL is updated with the resulting status.
13. The updated status can then be retrieved by the original client application.
14. The complete workflow is recorded for audit and traceability.

This workflow demonstrates the integration between the React application, Flutter application, ASP.NET Core Web API, PostgreSQL database, Agentic AI workflow, deterministic business validation, and human approval mechanism.

Workflow Exception and Safe-Failure Handling

ChargeSync must prevent invalid or unsafe operations when a workflow cannot be completed normally.

The following conditions must be handled safely:

- Invalid user input
- Unauthorized operation
- Station not approved
- Charger unavailable
- Charger under maintenance
- Reservation conflict
- Vehicle–charger incompatibility
- Invalid reservation status transition
- AI structured-output validation failure
- AI tool failure or unavailability
- AI retry limit exceeded
- Unsupported AI recommendation
- Human approval rejection
- Database or service failure

When such a condition occurs, the system must:

1. Prevent the invalid or high-impact operation from being executed.
2. Return an appropriate status or error to the client.
3. Preserve the relevant workflow or transaction state.
4. Record the failure or rejection for traceability where appropriate.
5. Allow the user or authorized operator to retry, revise, or safely terminate the workflow where applicable.

7. Use Case Model

The ChargeSync Use Case Model identifies the main actors who interact with the system and the major functions available to each role. The system supports four primary user roles: EV Driver, Station Owner, Platform Administrator, and Customer Support Manager.

Actors

EV Driver

The EV Driver uses ChargeSync to manage vehicles, find suitable charging stations, check charger compatibility and availability, create and manage reservations, check in using QR codes, participate

in charging sessions, receive AI charging recommendations, and view invoices and loyalty information.

Station Owner

The Station Owner manages registered charging stations and their operational resources. This includes managing chargers, connector information, pricing, operating hours, maintenance periods, walk-in charging sessions, and reviewing authorized operational requests.

Platform Administrator

The Platform Administrator manages platform-level administrative operations, including reviewing and approving station registrations, managing users and system-level configurations, and monitoring important system operations.

Customer Support Manager

The Customer Support Manager manages customer support requests, assists users with operational issues, and monitors support-related information.

Main Use Cases

Authentication and Account Management

- Register
- Login
- Logout
- Manage User Profile

Vehicle and Compatibility Management

- Manage Vehicle
- View Vehicle Specifications
- Check Vehicle–Charger Compatibility
- Generate Compatibility Recommendation

Station and Charger Management

- Search Station
- View Charger Availability
- Manage Station
- Manage Charger
- Manage Pricing
- Manage Operating Hours
- Manage Maintenance

Reservation and Charging

- Reserve Charger
- Manage Reservation
- Scan Reservation QR
- Check In
- Start Walk-In Session
- Manage Charging Session
- View Charging History

Agentic AI

- Submit Charging Objective
- Generate AI Charging Plan
- Review AI Recommendation
- View AI Workflow Status
- Approve AI Action
- Reject AI Action
- Request AI Revision

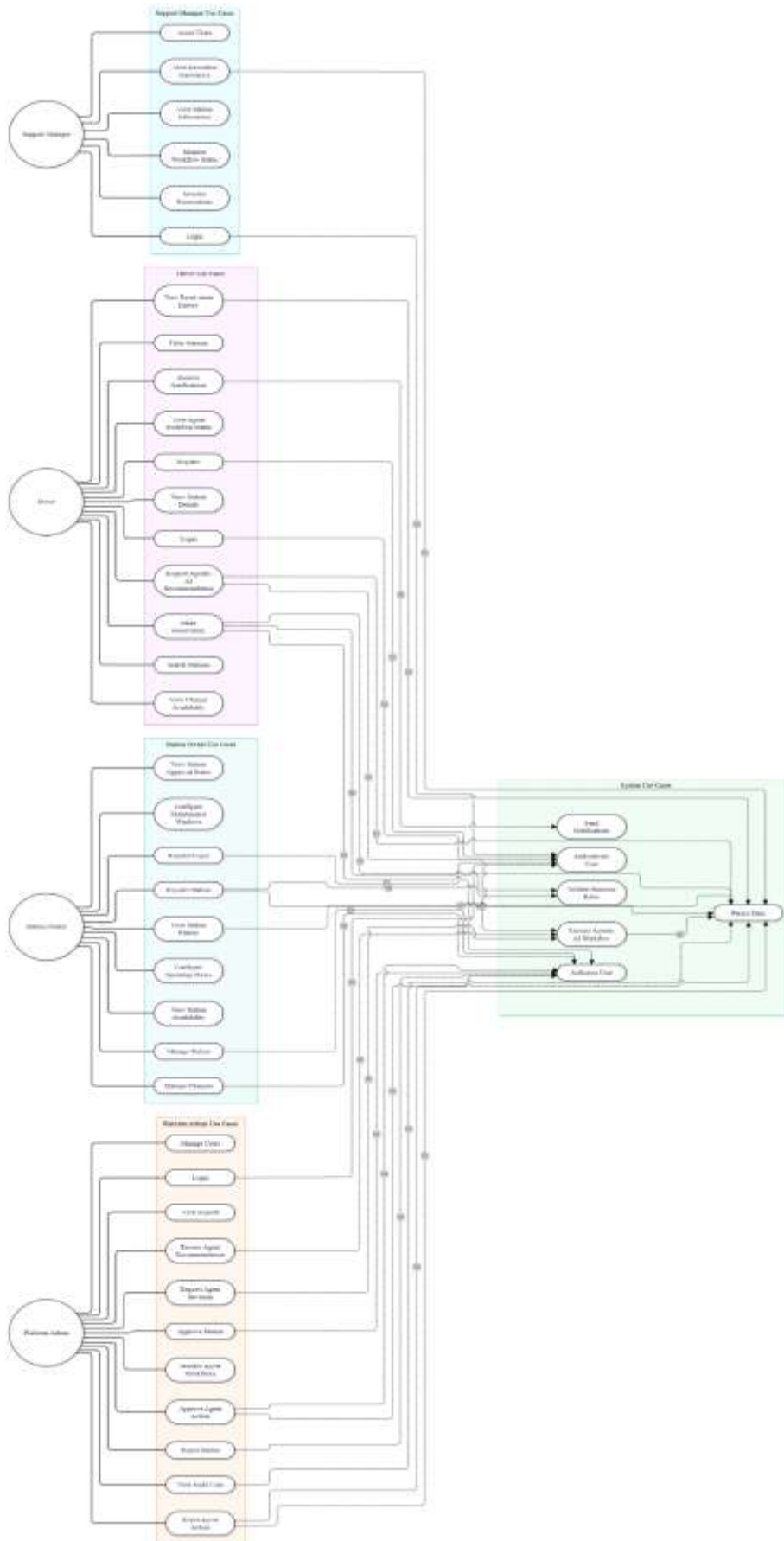
Billing and Loyalty

- View Invoice
- View Settlement Information
- View Membership Information
- View Loyalty Information

Administration and Support

- Approve Station
- Reject Station
- Manage Support Request
- View Operational History
- View Audit Information

7.1 Use Case Diagram



8. System Architecture

ChargeSync follows a layered full-stack architecture that separates the presentation, application/API, data, Agentic AI, and external service responsibilities. This architecture allows the React Web Application and Flutter Mobile Application to use the same ASP.NET Core Web API, business rules, authentication, authorization, and PostgreSQL database.

The ASP.NET Core Web API acts as the central backend and authoritative business layer. Agentic AI functionality is integrated through the backend rather than being called directly from the client applications.

Presentation Layer

- React Web Application.
- Flutter Mobile Application.

Application/API Layer

- ASP.NET Core Web API.
- Controllers.
- DTOs.
- Application Services.
- Validation.
- Authentication.
- Authorization.

Data Layer

- Entity Framework Core.
- PostgreSQL.
- Repositories/Data Access.
- Transactions.
- Migrations.

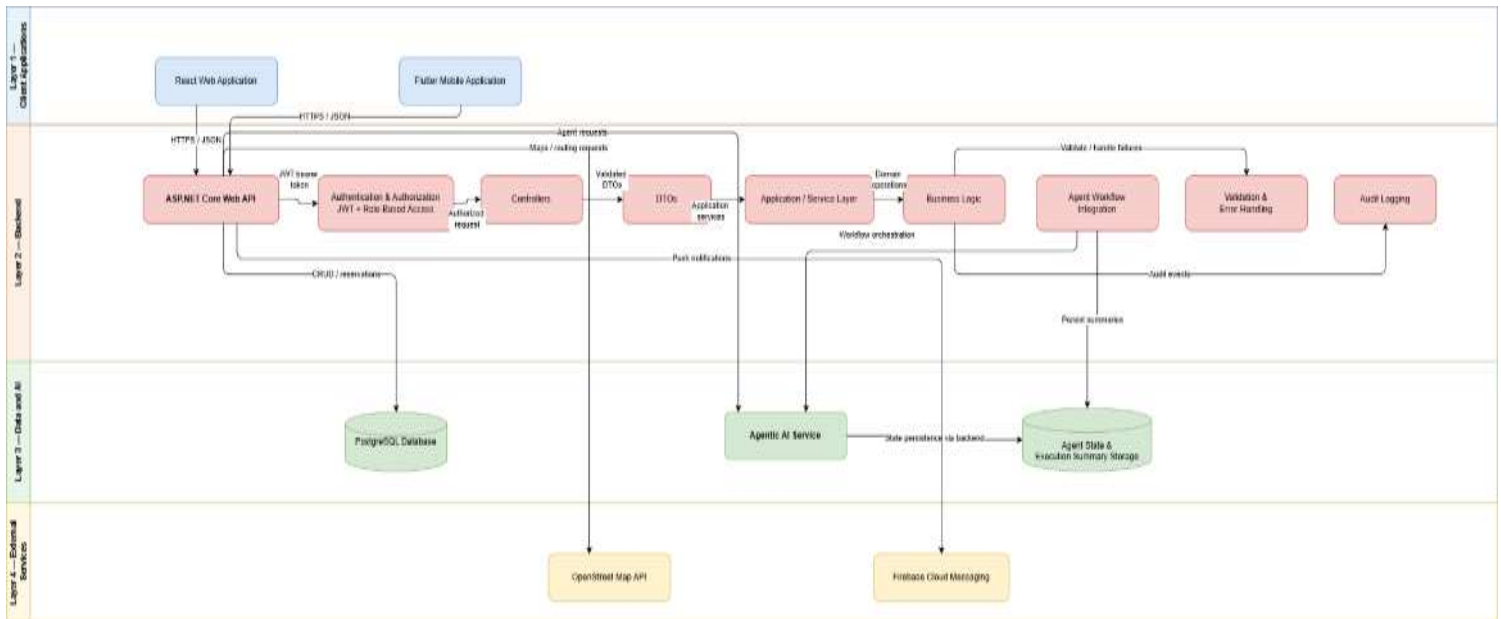
AI Layer

- Python Agentic AI service.
- Agent orchestration.
- Specialized agents.
- Controlled tools.
- Structured state.
- Validation.
- Human approval.

External Services

- Map/location service.
- Cloudinary/media service

8.1 High-Level ChargeSync Architecture



9. Database Design

9.1 Database Technology

ChargeSync uses **PostgreSQL** as the primary relational database. **Entity Framework Core with the Npgsql PostgreSQL provider** is used by the ASP.NET Core Web API for database access, migrations and persistence.

The database is shared by the React Web Application, Flutter Mobile Application and ASP.NET Core Web API. Both client applications communicate with the database indirectly through the ASP.NET Core Web API.

The database design focuses on:

- Data integrity.
- Transactional consistency.
- Referential integrity.
- Concurrent reservation handling.
- Auditability.
- Secure persistence.
- Efficient querying.
- Support for Agentic AI workflow state.

The database uses appropriate PostgreSQL data types, primary keys, foreign keys, constraints, indexes and audit fields such as CreatedAt and UpdatedAt.

9.2 Database Normalization

The ChargeSync database follows normalized relational database principles and is designed to achieve **Third Normal Form (3NF)**.

Related information is separated into appropriate entities to reduce unnecessary data duplication, update anomalies and inconsistent records.

For example:

- Connector types are maintained in the ConnectorTypes table rather than repeatedly storing connector descriptions in vehicle and charger records.
- Vehicle information is separated from extended charging profiles using Vehicles and VehicleChargingProfiles.
- Station information is separated from individual charger information using Stations and Chargers.
- Operating schedules are separated from station information using OperatingHours.
- Charger maintenance periods are stored separately in MaintenanceWindows.
- Reservation information is separated from reservation status history.
- Charging session information is separated from invoice information.
- Membership plans are separated from individual driver subscriptions.
- Loyalty accounts are separated from reward redemption records.
- Agentic AI workflow execution state is stored separately in AgentWorkflowRuns.

This structure improves maintainability, data integrity and traceability while allowing individual business components to evolve independently.

9.3 Primary and Foreign Keys

Each major entity contains a unique primary key. Foreign keys maintain relationships between related records.

Examples include:

Important relationships include:

- Vehicles.DriverId → Users.Id
- Vehicles.ConnectorTypeId → ConnectorTypes.Id
- VehicleChargingProfiles.VehicleId → Vehicles.Id
- VehicleChargingProfiles.PreferredConnectorTypeId → ConnectorTypes.Id
- Stations.OwnerId → Users.Id
- Chargers.StationId → Stations.Id
- Chargers.ConnectorTypeId → ConnectorTypes.Id
- OperatingHours.StationId → Stations.Id
- MaintenanceWindows.ChargerId → Chargers.Id

- Reservations.DriverId → Users.Id
- Reservations.ChargerId → Chargers.Id
- Reservations.VehicleId → Vehicles.Id
- ReservationStatusHistory.ReservationId → Reservations.Id
- ReservationStatusHistory.ChangedByUserId → Users.Id
- ChargingPlans.WorkflowRunId → AgentWorkflowRuns.WorkflowRunId

9.5 Database Constraints and Integrity

ChargeSync uses database-level constraints to protect the integrity of important business data.

Examples include:

- Primary-key constraints for unique entity identification.
- Foreign-key constraints for referential integrity.
- Unique constraints for values such as user email and reservation QR codes.
- NOT NULL constraints for mandatory attributes.
- CHECK constraints for controlled status and role values.
- Appropriate numeric and timestamp data types.
- Indexes for frequently queried fields.
- Temporal exclusion constraints for reservation conflict prevention.

The database therefore provides an additional layer of validation beyond application-level validation.

9.6 Reservation Concurrency Control

Reservation concurrency is a critical requirement because two users must not be able to reserve the same charger during overlapping time periods.

ChargeSync uses **PostgreSQL temporal exclusion constraints with the GiST index mechanism**, together with **pessimistic row locking where required**, to prevent double-booking.

The reservation design considers:

- ChargerId
- StartTime
- EndTime

An overlapping reservation for the same charger is rejected at the database level.

The application also performs availability validation before creating a reservation. Database-level concurrency protection provides the final guarantee when multiple requests arrive at the same time.

The reservation operation is executed within an appropriate database transaction so that availability checking, payment/deposit processing and reservation creation can be handled consistently.

This approach is particularly important because ChargeSync supports both online reservations and staff-created walk-in reservations.

9.7 Walk-In Support

ChargeSync supports unregistered walk-in drivers who arrive at a charging station without a prior reservation.

For a walk-in charging operation:

1. Station Staff checks charger availability.
2. Staff initiates the walk-in operation through the Flutter Staff POS interface.
3. The ASP.NET Core API validates the charger and station availability.
4. A Reservations record is created.
5. DriverId is set to NULL when there is no registered driver account.
6. The reservation locks the charger time period and prevents conflicting online reservations.
7. A charging session is initiated through the normal session workflow.

This design allows both registered reservations and walk-in charging to use the same reservation and charging-session infrastructure.

It also provides consistent operational records while allowing the system to distinguish registered driver bookings from unregistered walk-in operations.

9.8 Agent Workflow State

The AgentWorkflowRuns entity stores the persistent state and execution summary of Agentic AI workflows.

It can contain:

- Workflow identifier.
- Requesting driver.
- User objective and constraints.
- Workflow status.
- Generated structured plan.
- Agent output summaries.
- Deterministic validation result.
- Approval requirement.
- Approval decision.
- Approving user.
- Approval notes.
- Failure information.
- Retry count.

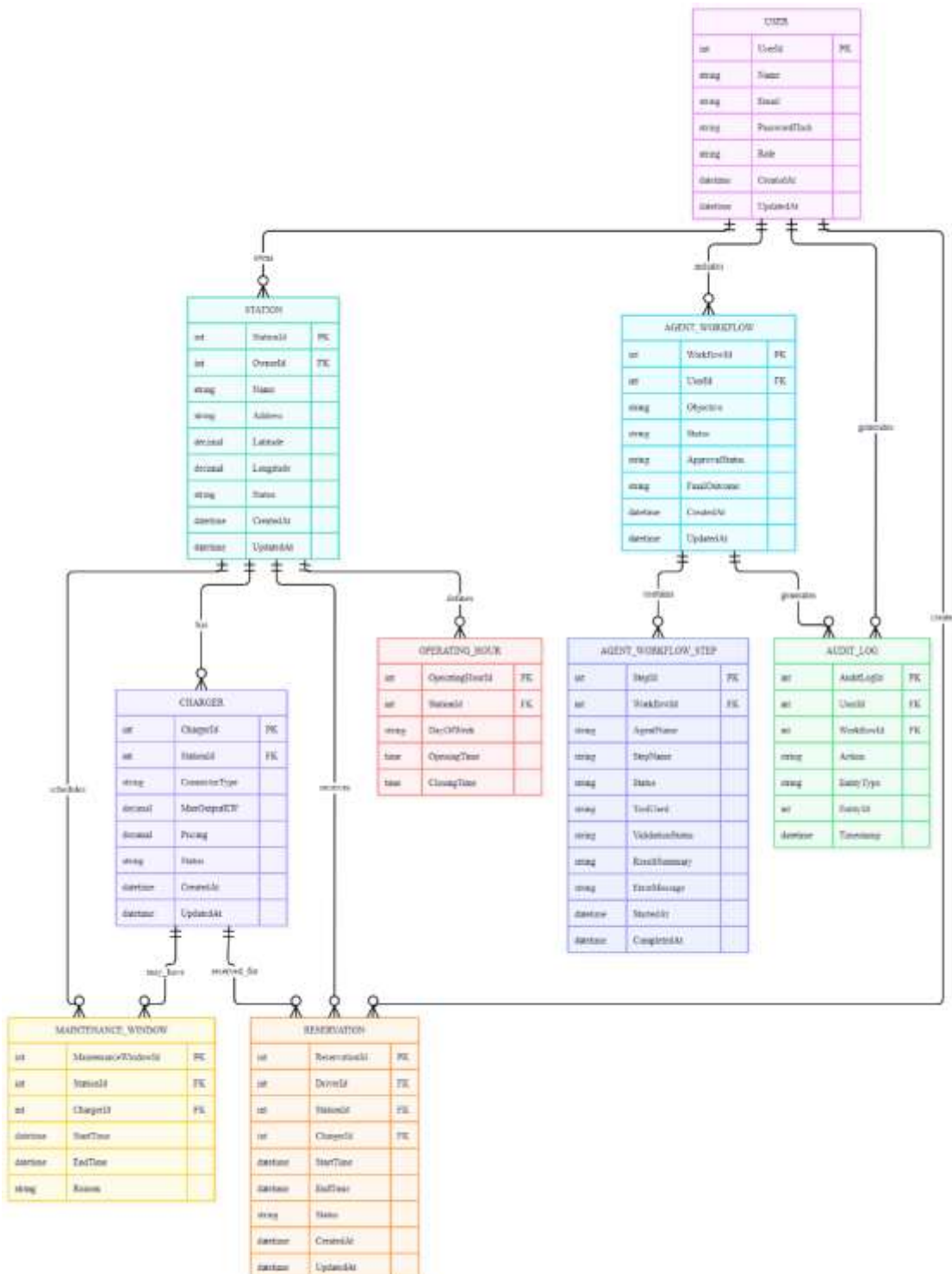
- Creation and update timestamps.

The database stores the information required to resume, audit and evaluate the workflow.

Hidden model reasoning or private chain-of-thought is not stored. Only the structured workflow state, relevant execution summaries, validation results, approval information and failure information required by the system persisted.

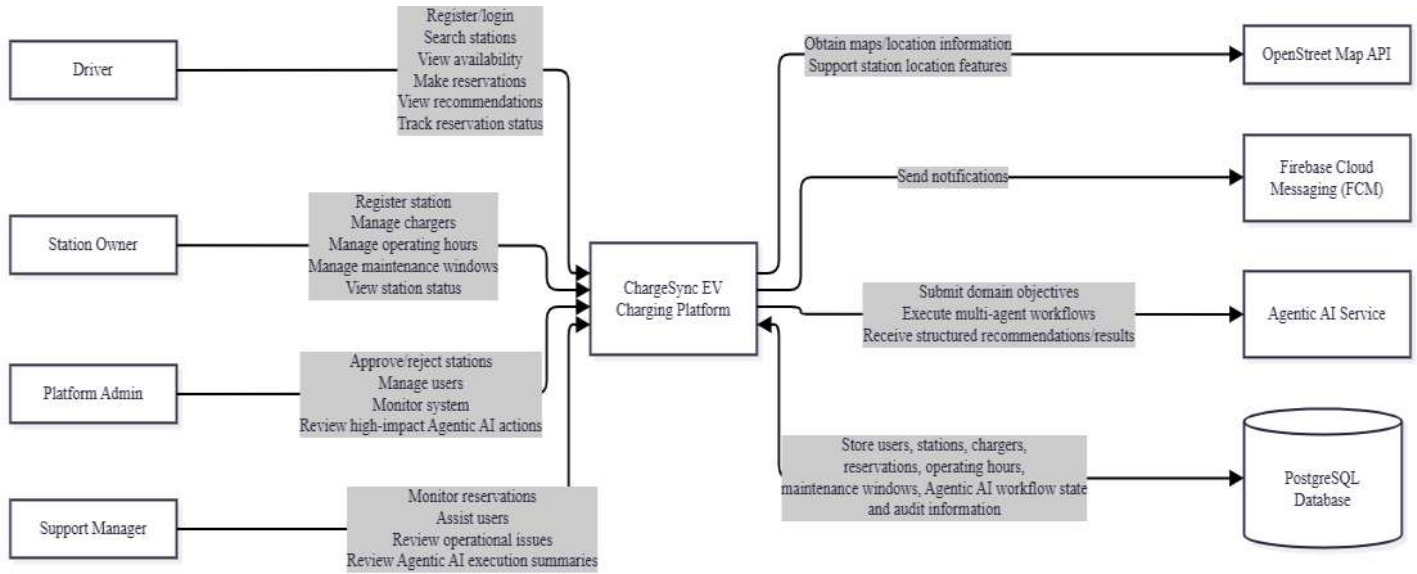
This design supports the assignment requirement for persistent Agentic AI workflow state while avoiding unnecessary storage of hidden reasoning or sensitive information.

ER Diagram

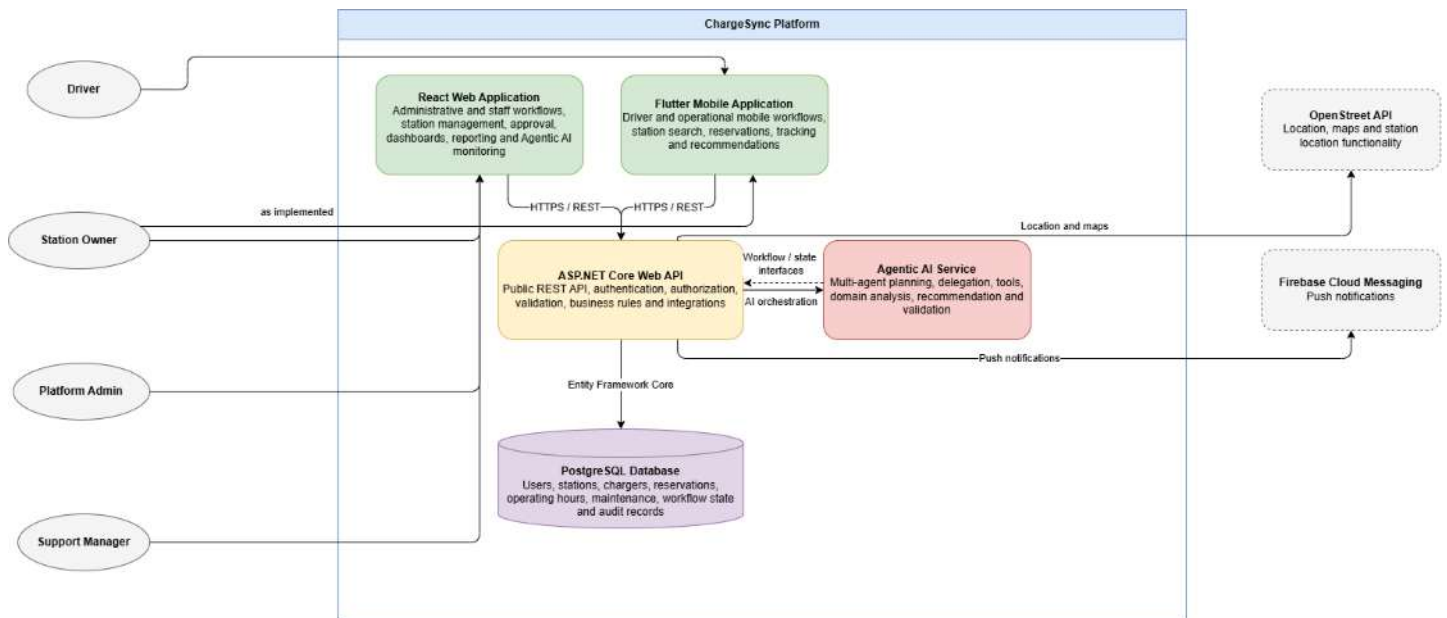


10. Diagrams

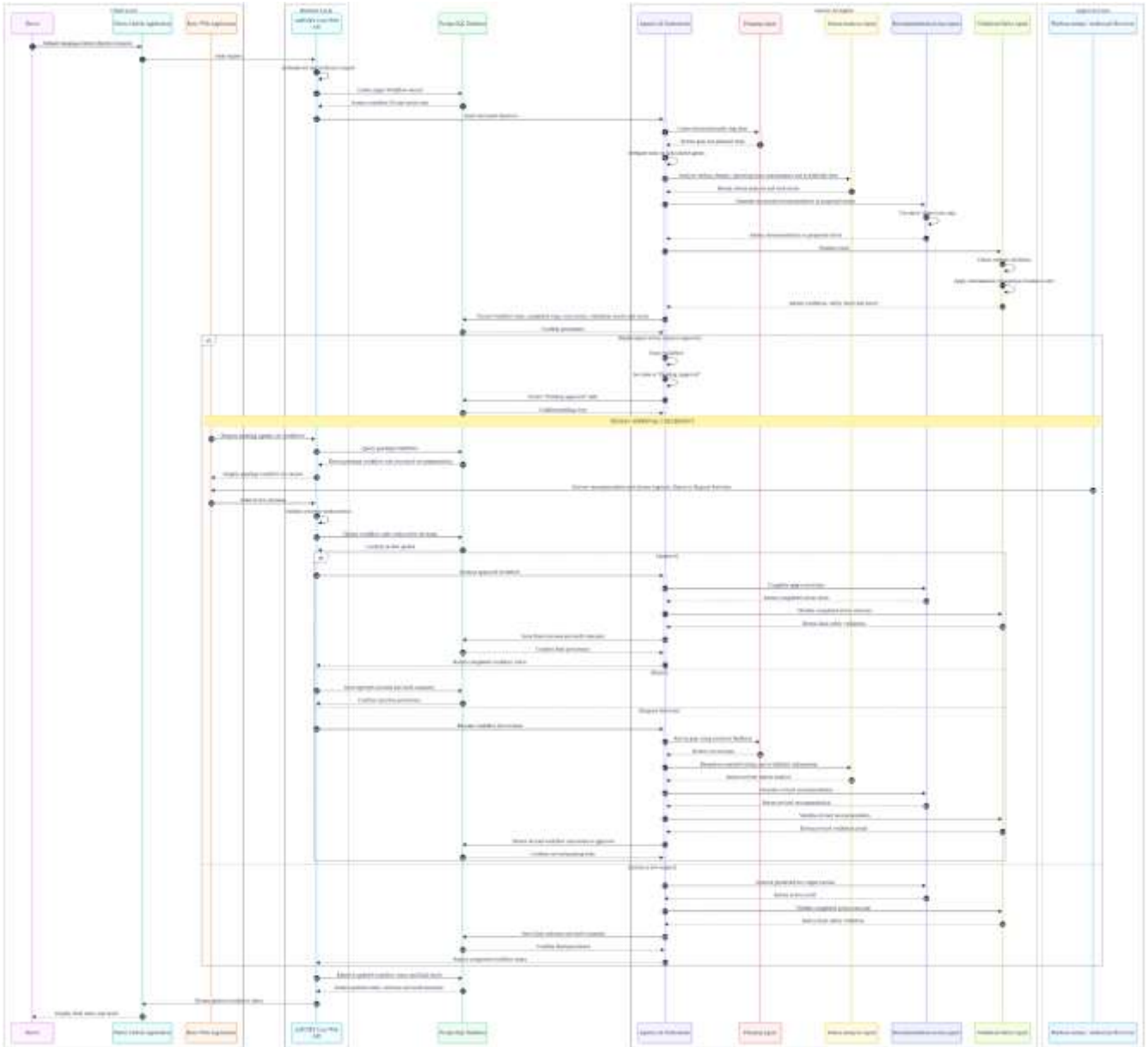
10.1 System Context Diagram



10.2 C4 Container Diagram



10.3 Agentic AI Workflow



11. ASP.Net Core Web API

11.1 Backend Architecture

ChargeSync uses **ASP.NET Core Web API** as the central backend and integration layer for the entire platform.

The backend follows a layered architecture that separates HTTP request handling, business logic, data access, validation and external service communication.

The main backend layers and responsibilities are:

Controllers

Controllers expose RESTful API endpoints for the React Web Application and Flutter Mobile Application.

They are responsible for:

- Receiving HTTP requests.
- Validating request structure.

- Calling the appropriate application service.
- Returning appropriate HTTP responses.
- Applying authorization requirements to protected endpoints.

Examples include controllers for:

- Authentication.
- Vehicles.
- Stations.
- Chargers.
- Reservations.
- Charging Sessions.
- Payments and Invoices.
- Loyalty.
- Memberships.
- Support Tickets.
- Agentic AI workflows.

DTOs (Data Transfer Objects)

DTOs define the data exchanged between clients and the API.

They are used to:

- Control the information exposed through the API.
- Validate incoming request data.
- Separate API contracts from database entities.
- Prevent unnecessary database fields from being exposed.
- Provide structured request and response models.

Application Services

Application services contain the main business logic of the system.

They coordinate operations such as:

- Vehicle management.
- Compatibility checking.
- Station management.
- Charger availability.
- Reservation creation.
- Reservation cancellation.
- QR check-in.
- Walk-in allocation.
- Charging-session management.
- Invoice generation.
- Loyalty operations.
- Support ticket processing.
- AI workflow execution and approval.

Business rules are implemented in the service layer rather than placing complex logic directly inside controllers.

Data Access Layer

The data access layer manages communication with PostgreSQL through **Entity Framework Core and the Npgsql provider**.

It is responsible for:

- Querying database records.

- Creating, updating and deleting entities.
- Managing relationships.
- Executing transactions.
- Applying database constraints.
- Running Entity Framework Core migrations.

Authentication and Authorization

ChargeSync uses **JWT bearer authentication** to authenticate API requests.

Role-based authorization controls access according to the user's role.

The main roles include:

- EV Driver.
- Station Owner.
- Platform Administrator.
- Customer Support Manager.

Authorization rules ensure that users can only access operations permitted for their role.

For example, a Driver can manage their own vehicles and reservations, while a Platform Administrator can approve stations and perform system-wide administrative operations.

Validation

The API performs validation at multiple levels.

This includes:

- DTO/request validation.
- Business-rule validation.
- Authorization validation.
- Database constraint validation.
- Reservation availability validation.
- Vehicle-charger compatibility validation.
- Charging-session validation.
- AI output validation.

Validation prevents invalid or unauthorized operations from reaching the database or operational workflow.

Error Handling

The backend provides centralized error handling for API failures and unexpected exceptions.

The API returns appropriate HTTP status codes and structured error responses for situations such as:

- Invalid requests.
- Authentication failures.
- Authorization failures.
- Resource not found.
- Reservation conflicts.
- Business-rule violations.
- External service failures.
- AI workflow failures.

This provides consistent error handling across the different API modules.

Logging and Auditing

The backend records important operational events and errors for monitoring and troubleshooting.

Audit-related information is also persisted for important business operations, including:

- Reservation status changes.
- Charging-session operations.

- Staff energy overrides.
- AI validation results.
- Human approval decisions.
- Failed AI workflows.

11.2 API Responsibilities

The ASP.NET Core Web API acts as the **single trusted application gateway** for ChargeSync. Its main responsibilities include:

1. Authentication

The API authenticates users using JWT bearer tokens and manages authenticated access to protected resources.

2. Authorization

The API applies role-based access control to ensure that each user can only perform authorized operations.

3. CRUD Operations

The API provides RESTful endpoints for creating, retrieving, updating and deleting business entities.

These include:

- Vehicles.
- Stations.
- Chargers.
- Reservations.
- Support tickets.
- Memberships.
- Other application records.

4. Business Rule Enforcement

The API applies important ChargeSync business rules, including:

- Station approval.
- Charger availability.
- Reservation conflict prevention.
- Vehicle-charger compatibility.
- Walk-in allocation.
- Session validation.
- Invoice calculation.
- Loyalty validation.
- Human approval requirements.

5. Database Operations

The API communicates with PostgreSQL through Entity Framework Core.

It manages:

- Queries.
- Inserts.
- Updates.
- Deletes.
- Relationships.
- Transactions.
- Migrations.

6. Reservation and Concurrency Management

The API coordinates reservation creation and availability checking while relying on PostgreSQL concurrency mechanisms to prevent double-booking.

This includes:

- Availability validation.
- Transaction management.
- PostgreSQL temporal exclusion constraints.
- Pessimistic locking where required.

7. Agentic AI Integration

The ASP.NET Core API is the **only application-facing gateway to the Agentic AI service**.

React and Flutter do not communicate directly with the Python Agentic AI service.

The API:

1. Receives the AI request from the client.
2. Validates the request.
3. Creates or updates the AI workflow state.
4. Calls the internal Python/FastAPI Agentic AI service.
5. Receives the structured AI result.
6. Performs deterministic validation.
7. Determines whether human approval is required.
8. Stores the workflow result.
9. Returns the appropriate result or approval state to the client.

Communication between the ASP.NET Core API and Agentic AI service is protected using the internal service authentication mechanism defined by the system.

8. Human-in-the-Loop Approval

The API manages high-impact AI operations that require human review.

When an approval trigger is detected, the workflow is moved to a PendingApproval state.

The API records:

- Approval requirement.
- Approval status.
- Reviewing user.
- Approval decision.
- Approval notes.

The operation is executed only after the required authorization and validation have been completed.

9. Third-Party Service Integration

The API coordinates communication with external services used by ChargeSync, including:

- Maps services for station discovery and distance-related information.
- Cloudinary for station media and physical meter photo storage.

The backend prevents client applications from directly managing sensitive server-side integrations.

10. Transaction and Consistency Management

The API coordinates multi-step operations using database transactions where required.

Important transactional workflows include:

- Reservation creation and deposit processing.
- Walk-in charger allocation.
- Reservation cancellation.
- Charging-session completion.
- Invoice generation.

- Invoice settlement.
- Loyalty redemption.

This helps ensure that related database changes remain consistent.

12. API Endpoint

The ChargeSync ASP.NET Core Web API exposes RESTful endpoints for authentication, vehicle management, station and charger management, reservations, charging sessions, payments, loyalty, support and Agentic AI workflows.

The following table summarizes the main API endpoints defined for the system.

Module	Method	Endpoint	Purpose
Authentication	POST	/api/auth/register	Register a new user account
Authentication	POST	/api/auth/login	Authenticate a user and issue a JWT token
Vehicles	POST	/api/vehicles	Register a new vehicle
Vehicles	GET	/api/vehicles	Retrieve the current driver's vehicles
Vehicles	GET	/api/vehicles/all	Retrieve all registered vehicles for platform administration
Vehicles	GET	/api/vehicles/{id}	Retrieve a specific vehicle
Vehicles	PUT	/api/vehicles/{id}	Update the driver's vehicle
Vehicles	PUT	/api/vehicles/admin/{id}	Update any vehicle as an administrator
Vehicles	DELETE	/api/vehicles/{id}	Delete the driver's vehicle
Vehicles	DELETE	/api/vehicles/admin/{id}	Delete any vehicle as an administrator
Vehicles	GET	/api/vehicles/{id}/compatible-stations	Find compatible charging stations
Stations	GET	/api/stations	List, search and filter stations
Stations	GET	/api/stations/nearby	Search nearby stations based on location and connector type
Stations	POST	/api/stations	Register a new charging station
Stations	GET	/api/stations/{id}	Retrieve station details and chargers
Stations	PUT	/api/stations/{id}	Update station information
Stations	DELETE	/api/stations/{id}	Remove a station subject to business rules
Chargers	POST	/api/stations/{id}/chargers	Add a charger to a station
Chargers	PUT	/api/chargers/{id}	Update charger specifications and pricing
Chargers	PUT	/api/chargers/{id}/status	Update charger operational status
Stations	GET	/api/stations/{id}/utilization	Retrieve station utilization and revenue analytics

Reservations	POST	/api/reservations	Create an advance reservation with wallet pre-authorization
Reservations	POST	/api/reservations/walk-in	Create a staff-initiated walk-in reservation
Reservations	GET	/api/reservations	List, filter and paginate reservations
Reservations	GET	/api/reservations/{id}	Retrieve reservation details and linked session information
Reservations	PUT	/api/reservations/{id}/cancel	Cancel a reservation and trigger applicable refund processing
Reservations	GET	/api/reservations/{id}/history	Retrieve reservation status history
Reservations	POST	/api/reservations/staff-checkin	Validate reservation QR and perform staff check-in
Charging Plans	POST	/api/charging-plan/generate	Generate an AI-based charging plan
Charging Sessions	POST	/api/sessions/start	Start a charging session
Charging Sessions	PUT	/api/sessions/{id}/stop	Stop a charging session and record meter information
Payments	GET	/api/payments/invoices/{userId}	Retrieve a user's invoice ledger
Payments	POST	/api/payments/invoices/{id}/settle	Settle an invoice using wallet or cash
Memberships	POST	/api/subscriptions	Create a membership subscription
Memberships	PUT	/api/subscriptions/{id}/change	Change an existing membership plan
Loyalty	GET	/api/loyalty/{userId}	Retrieve loyalty balance and tier
Loyalty	POST	/api/loyalty/redeem	Redeem loyalty points for rewards
Support	POST	/api/support-tickets	Submit a driver support ticket
Support	PUT	/api/support-tickets/{id}/status	Update support ticket status
AI Compatibility	POST	/api/compatibility/evaluate	Evaluate vehicle-charger compatibility
AI Compatibility	POST	/api/compatibility/batch-evaluate	Evaluate compatibility across multiple candidates
AI Station Analysis	POST	/api/station-analysis/evaluate	Analyse station utilization and pricing information
AI Support	POST	/api/support/analyze	Perform AI-based support ticket analysis
AI Workflow	POST	/api/workflows/support	Execute the Agentic AI support workflow

API Design Principles

The API follows RESTful design principles and uses:

- HTTP methods such as GET, POST, PUT and DELETE.
- JSON request and response bodies.
- JWT bearer authentication for protected endpoints.
- Role-based authorization.
- Server-side validation.
- Consistent HTTP status codes.
- Asynchronous processing for appropriate operations.
- Swagger/OpenAPI documentation.

The ASP.NET Core API acts as the authoritative application layer. Client applications communicate with the backend through the API rather than accessing PostgreSQL or the Agentic AI service directly.

For Agentic AI operations, the client submits the request to ASP.NET Core, which validates and forwards the request to the internal Python/FastAPI Agentic AI service. The resulting structured output is validated before being returned or moved into a human-approval workflow.

12.1 API Status Codes

ChargeSync uses standard HTTP status codes to communicate the result of API operations.

Status Code	Meaning	ChargeSync Usage
200 OK	Request completed successfully	Successful GET, PUT operations and other successful requests returning data
201 Created	Resource successfully created	Successful user, vehicle, station, charger or reservation creation
204 No Content	Request succeeded without response content	Successful operations where no response body is required
400 Bad Request	Invalid request	Invalid request parameters, malformed data or validation failures
401 Unauthorized	Authentication required or failed	Missing, invalid or expired JWT token
403 Forbidden	Authenticated but not permitted	User does not have the required role or permission
404 Not Found	Requested resource does not exist	Invalid vehicle, station, charger, reservation or invoice ID
409 Conflict	Request conflicts with current system state	Reservation overlap, duplicate records or conflicting operational state
422 Unprocessable Entity	Request format is valid but business validation fails	Invalid business-specific input that cannot be processed
500 Internal Server Error	Unexpected server-side failure	Unhandled backend or infrastructure errors

13. Swagger

ChargeSync API v1 OAS 3.0
http://localhost:5035/swagger/v1/swagger.json

Authorize

AdminStations

GET	/api/admin/stations/pending		▼
GET	/api/admin/stations/{id}		▼
PUT	/api/admin/stations/{id}/approve		▼
PUT	/api/admin/stations/{id}/reject		▼

AgentWorkflows

GET	/api/agent-workflows/{id}		▼
GET	/api/agent-workflows/support-ticket/{ticketId}		▼
POST	/api/agent-workflows/support-ticket/{ticketId}		▼
POST	/api/agent-workflows/{id}/approve		▼
POST	/api/agent-workflows/{id}/reject		▼
POST	/api/agent-workflows/{id}/revise		▼

Loyalty

GET	/api/loyalty/me		▼	
GET	/api/loyalty/{userId}		▼	
GET	/api/loyalty/balance/{userId}		▼	
GET	/api/loyalty/history		▼	
GET	/api/loyalty/rewards			▼
POST	/api/loyalty/redeem		▼	
GET	/api/loyalty/redemptions		▼	
POST	/api/loyalty/redemptions/{id}/review		▼	

Memberships

GET	/api/membership-plans		▼
-----	-----------------------	--	---

POST	/api/Reservations/admin		▼
POST	/api/Reservations/walk-in		▼
GET	/api/Reservations/availability		▼
GET	/api/Reservations/{id}		▼
PUT	/api/Reservations/{id}		▼
DELETE	/api/Reservations/{id}		▼
PUT	/api/Reservations/{id}/cancel		▼

GET	/api/Stations/all	🔒	▼
GET	/api/Stations/search	🔒	▼
GET	/api/Stations/nearby	🔒	▼
GET	/api/Stations/{id}	🔒	▼
PUT	/api/Stations/{id}	🔒	▼
POST	/api/Stations/{id}/chargers	🔒	▼
PUT	/api/Stations/{id}/chargers/{chargerId}	🔒	▼
DELETE	/api/Stations/{id}/chargers/{chargerId}	🔒	▼

SupportTickets		^	
POST	/api/support-tickets/{id}/analysis	🔒	▼
GET	/api/support-tickets/invoices/{id}/validation	🔒	▼
GET	/api/support-tickets	🔒	▼
POST	/api/support-tickets	🔒	▼
GET	/api/support-tickets/{id}	🔒	▼
DELETE	/api/support-tickets/{id}	🔒	▼

14. React Web Application

The React application provides the web-based management interface for Station Owners, Platform Administrators and Customer Support Managers.

The application uses functional React components, React Hooks, routing and reusable UI components. Protected routes are used to prevent unauthorized access to role-specific screens.

Station Owner Functions

The Station Owner web portal provides:

- Station registration.
- Station information management.
- Charger management.
- Charger status management.
- Operating-hour configuration.
- Maintenance-window management.
- Station utilization information.
- Reservation monitoring.
- Operational-boundary approval.

Platform Administrator Functions

The administrator interface provides:

- User management.
- System-wide vehicle management.
- Station approval.
- Station rejection.
- Reservation and operational audit.
- Meter discrepancy review.
- AI approval workflows.

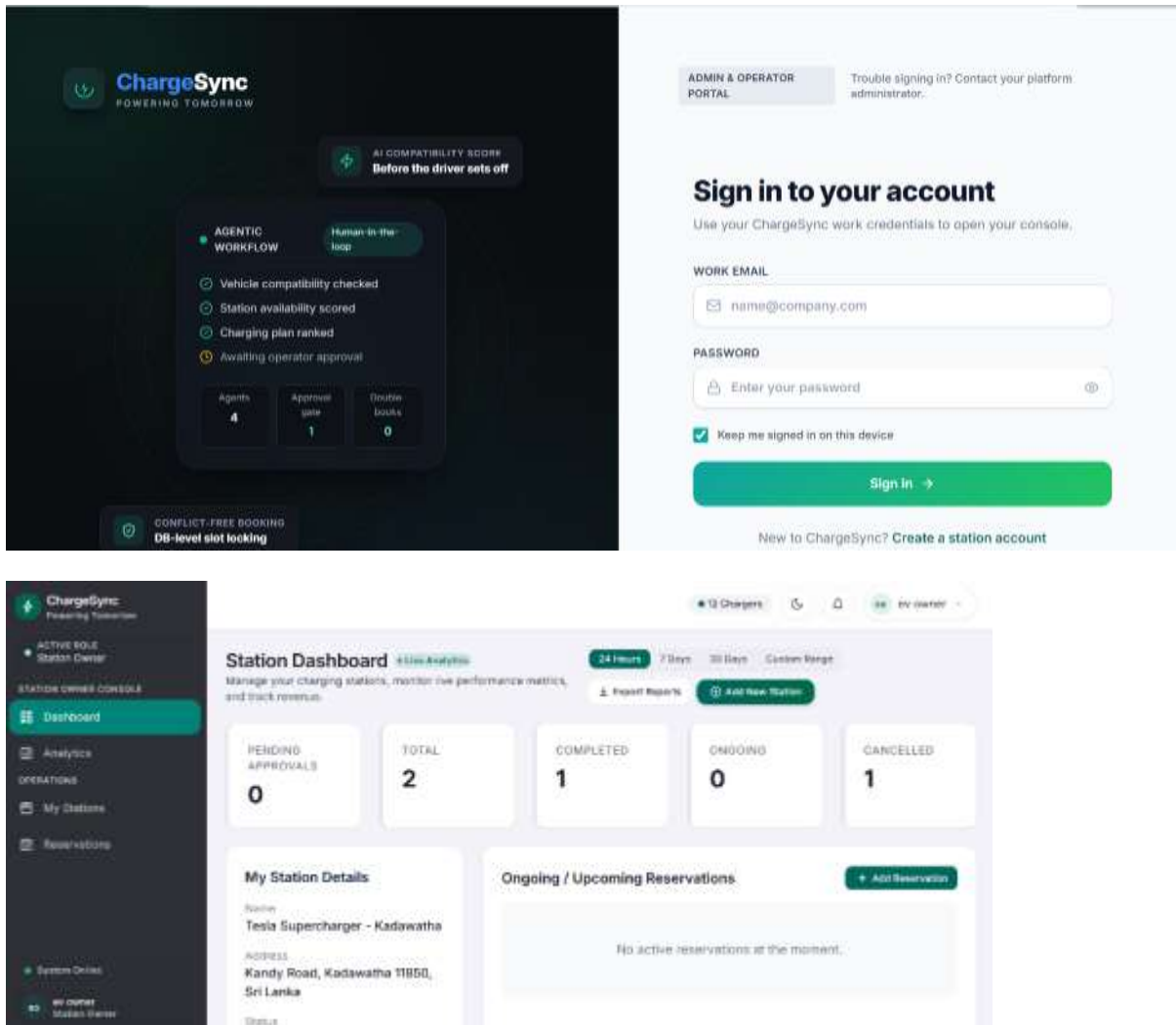
Customer Support Manager Functions

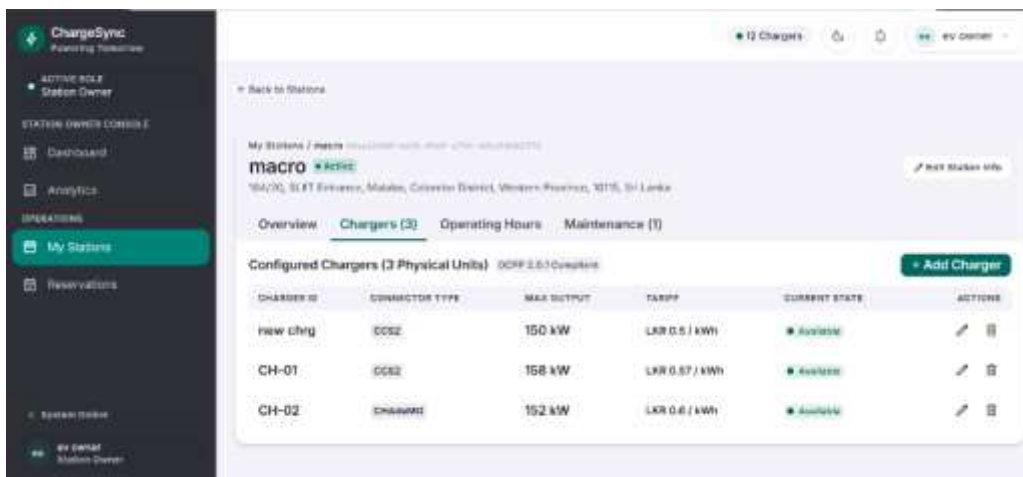
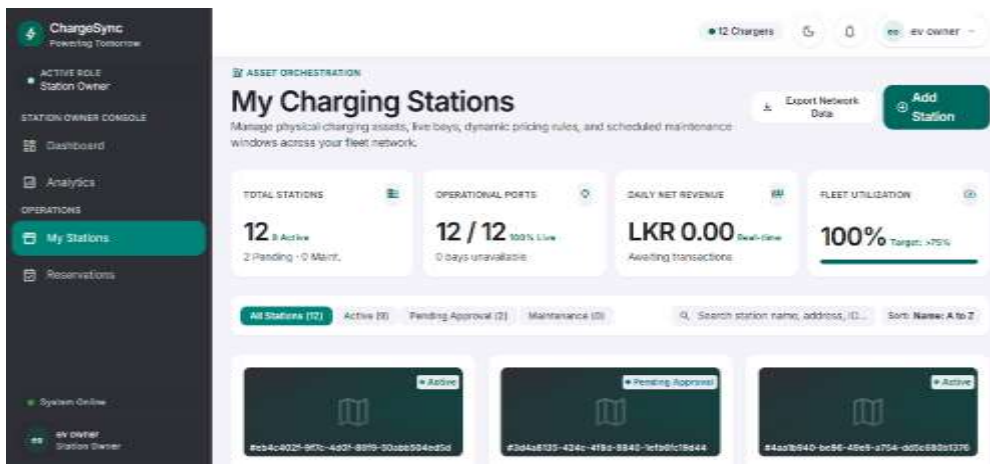
The support interface provides:

- Support ticket viewing.
- Ticket categorization.
- Ticket assignment.
- Ticket status management.
- Billing dispute handling.
- Refund workflow management.

The React application communicates exclusively with the ASP.NET Core API and does not directly communicate with PostgreSQL or the Agentic AI service.

React Evidence





15. Flutter Mobile Application

The Flutter application provides mobile workflows for EV Drivers and Station Owners.

15.1 Driver Mobile Mode

The Driver mobile interface provides:

- Registration and login.
- Vehicle management.
- Station discovery.
- Map-based station selection.
- Compatibility information.
- AI charging recommendations.
- Reservation creation.
- Reservation status tracking.
- QR code display.
- Charging history.
- Invoice information.
- Wallet functionality.
- Loyalty management.
- Support tickets.

15.2 Station Owner Operational Mobile Mode

The Station Owner can use the mobile application for operational activities associated with the owner's station.

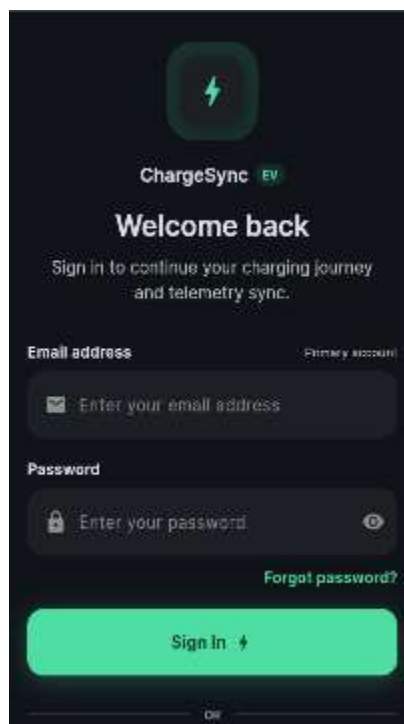
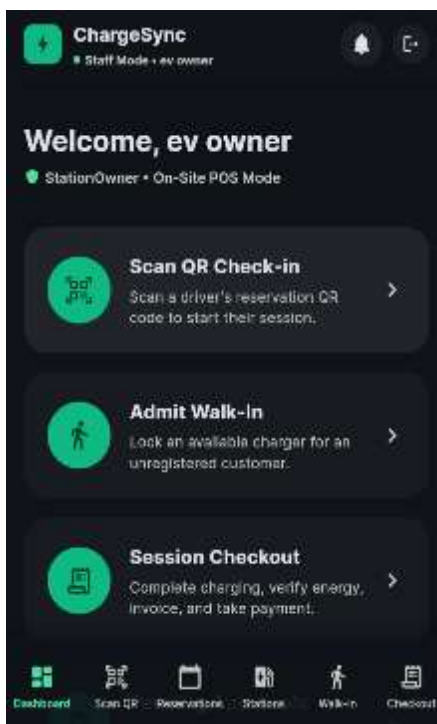
This is not modeled as a separate Station Staff user role. The authenticated Station Owner accesses station-specific operational functionality according to their authorization.

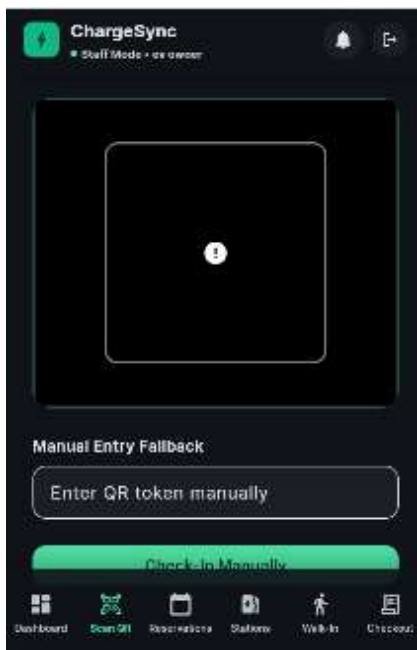
The operational mobile mode supports:

- Reservation QR scanning.
- Reservation validation.
- Walk-in admission.
- Active charging session monitoring.
- Session termination.
- Physical meter reading entry.
- Meter photograph capture.
- Invoice/payment recording.
- Cash settlement.

The Flutter application communicates with the same ASP.NET Core API used by the React application.

Flutter Evidence





16. Agentic AI Report

16.1 Purpose

The ChargeSync Agentic AI subsystem provides domain-specific multi-step decision support rather than functioning as a generic chatbot.

The subsystem analyses structured application data, delegates tasks to specialized agents, uses controlled tools, validates outputs and requests human approval when a high-impact action is detected.

16.2 Agent Architecture

ChargeSync contains four specialized agents.

Vehicle Compatibility Agent

The Vehicle Compatibility Agent evaluates whether a vehicle can use a charger based on connector type, charging capacity and related vehicle characteristics.

Station Analysis Agent

The Station Analysis Agent evaluates station suitability using information such as charger availability, pricing, operating hours, maintenance windows and utilization information.

Charging Recommendation and Planning Agent

The Charging Recommendation and Planning Agent acts as the coordinator. It receives the driver's objective and combines outputs from the compatibility and station analysis agents to generate ranked charging recommendations.

Validation and Support Agent

The Validation and Support Agent evaluates proposed results against deterministic business rules. It can identify unsupported recommendations, session discrepancies and support-related conditions that require additional review.

16.3 Agent Inputs and Outputs

Each agent has a defined responsibility and structured input/output contract.

Agent	Input	Output
Compatibility	Vehicle + Charger	Compatibility result
Station Analysis	Station + Charger data	Station suitability
Planning	Driver objective + agent results	Ranked itinerary
Validation/Support	Proposed result + business rules	Validation/approval result

16.4 Controlled Tools

The agents use allow-listed application tools rather than unrestricted access to the database.

Examples include:

- Vehicle lookup.
- Charger availability lookup.
- Station lookup.
- Pricing lookup.
- Operating-hours lookup.
- Maintenance lookup.
- Reservation validation.
- Support analysis.

16.5 AI Evaluation

Agentic AI evaluation uses deterministic test cases, schema validation, business-rule assertions and human review.

LLM-based judging may be used as supporting evidence but is not treated as the only evaluation mechanism.

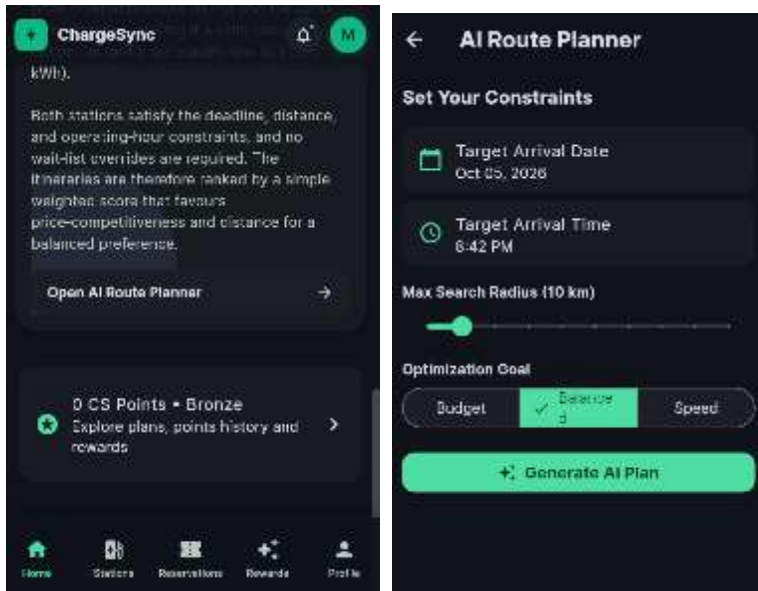
AI Evaluation Table

16.9.1 Golden Test Cases

Test Case	Objective	Expected Result	Actual Result	Pass/Fail
AI-01	Compatible vehicle	Compatible station returned	Pass	Pass
AI-02	Incompatible connector	Station rejected	Pass	Pass
AI-03	Multiple stations	Ranked stations generated	Pass	Pass
AI-04	Near closing time	Approval required	Pass	Pass
AI-05	Maintenance conflict	Approval/alternative generated	Pass	Pass

AI-06	Invalid tool input	Tool call rejected	Pass	Pass
AI-07	High-value refund	Human approval required	Pass	Pass
AI-08	>15% meter discrepancy	Audit triggered	Pass	Pass
AI-09	Tool failure	Safe failure	Pass	Pass
AI-10	Prompt injection attempt	Unsafe request rejected	Pass	Pass

Agentic AI Evidence



17. Integrated End-to-End Workflow

The main integrated workflow demonstrates how the Flutter application, ASP.NET Core API, PostgreSQL database, Agentic AI subsystem and React application work together.

Step 1 – Driver Objective

The EV Driver enters a charging requirement through the Flutter application.

The request may contain information such as:

- Vehicle.
- Destination or preferred station area.
- Required arrival/deadline time.
- Distance preference.
- Price preference.

Step 2 - ASP.NET Core API

The Flutter application sends the request to the ASP.NET Core API.

The API authenticates the driver and validates the request.

Step 3 - Agentic AI Planning

The ASP.NET Core API initiates the Agentic AI workflow.

The Planning Agent creates a structured plan and delegates relevant tasks to the Compatibility Agent and Station Analysis Agent.

18. Third-Party Services

Service	Purpose	Integration
OpenStreetMap + Leaflet	Map and station location functionality	Client application
Cloudinary	Station images and meter photographs	Backend/client workflow
LLM Provider	AI model processing	Agentic AI service

19. Testing Report

19.1 Testing Strategy

Testing was performed at multiple levels to verify the functional correctness, integration, reliability, security, and performance of the ChargeSync Intelligent EV Charging Reservation and Recommendation Platform.

The testing strategy covered the following areas:

- Unit testing.
- Service-layer testing.
- REST API testing.
- Database integration testing.
- React web application testing.
- Flutter mobile application testing.
- Agentic AI workflow evaluation.
- End-to-end testing.
- Performance testing.

The testing process focused on verifying both individual components and the complete integrated workflow. Functional testing was used to verify that system requirements were correctly implemented, while integration and end-to-end testing verified communication between the frontend applications, ASP.NET Core Web API, PostgreSQL database, and Agentic AI service.

Special attention was given to critical business rules such as station approval, charger availability, reservation conflict prevention, vehicle compatibility, charging-session management, billing, and human approval for high-impact AI actions.

19.1.1 Testing strategy table

Testing Area	Tool/Technology	Evidence
Backend Unit	xUnit	Test results
Backend Mocking	Moq	Test code/results
API Integration	ASP.NET testing	API test results
Database	PostgreSQL integration	Database tests
React	Vitest / RTL	Test results
Flutter	Flutter test	Test results
AI	pytest	AI test results
E2E	Integrated workflow	Video/screenshots
Performance	Load/performance test	Metrics
CI	GitHub Actions	Passing workflow

19.2 Backend Implementation

The following test cases were defined to verify the main ASP.NET Core Web API functionality and business rules.

Test ID	Test Case	Test Description	Expected Result
BE-01	Valid Login	Submit valid user credentials	User is authenticated and a JWT token is returned
BE-02	Invalid Password	Submit valid email with an incorrect password	Authentication is rejected and an appropriate error response is returned
BE-03	Register Station	Station Owner submits valid station registration details	Station is created with Pending status
BE-04	Approve Station	Platform Administrator approves a pending station	Station status changes to Active
BE-05	Suspend Station	Platform Administrator suspends an active station	Station status changes to Suspended and the station is no longer treated as normally available
BE-06	Create Charger	Station Owner adds a valid charger to an active station	Charger is created successfully
BE-07	Invalid Charger Data	Submit charger information with missing or invalid required values	Validation error is returned and invalid data is not saved
BE-08	Create Reservation	Driver submits a valid reservation for an available compatible charger	Reservation is created successfully
BE-09	Conflicting Reservation	Attempt to create an overlapping reservation for the same charger	Reservation request is rejected and the existing reservation remains unchanged
BE-10	Start Charging Session	Start a valid charging session after reservation/check-in validation	Charging session is created successfully
BE-11	Stop Charging Session	Stop an active charging session with valid meter information	Session is completed and energy/invoice information is recorded
BE-12	Unauthorized Operation	User attempts to access an operation without the required permission	API returns 403 Forbidden
BE-13	Vehicle Compatibility	Evaluate a vehicle against a compatible charger	Compatibility result is returned successfully
BE-14	Incompatible Vehicle	Evaluate a vehicle against an incompatible charger	System identifies the incompatibility and prevents an invalid recommendation
BE-15	AI Charging Plan	Submit a valid charging objective and vehicle/station information	AI charging plan is generated and returned in the expected structured format
BE-16	Invalid AI Result	AI service returns an invalid or incomplete structured response	Backend validation rejects the result and prevents unsafe execution
BE-17	Invoice Settlement	Settle a valid invoice using the supported payment method	Invoice is marked as settled and the appropriate ledger information is updated
BE-18	Loyalty Redemption	Submit a valid loyalty redemption request within allowed limits	Points are deducted and the redemption is recorded
BE-19	Support Ticket	Driver submits a valid support ticket	Support ticket is created successfully
BE-20	AI Human Approval	High-impact AI action requires human approval	Action remains pending until an authorized user approves or rejects it

19.3 Database Testing

Database testing was performed to verify the integrity, consistency, and reliability of the PostgreSQL database.

The following areas were tested:

Foreign-Key Constraints

Foreign-key relationships between entities were tested to ensure that invalid references cannot be inserted into the database.

Examples include:

- Vehicle references a valid user.
- Charger references a valid station.
- Reservation references a valid charger.
- Charging session references a valid reservation where applicable.
- Invoice references the appropriate user/session information.
- Support tickets reference valid users.

Unique Constraints

Unique constraints were tested to prevent duplicate data where uniqueness is required.

Examples include:

- User email uniqueness.
- Required vehicle or station identifiers.
- Other fields defined as unique in the database schema.

Required Fields

Required database fields were tested by submitting incomplete data through the API. Expected Result: Invalid records are rejected and required database fields are not stored as null or invalid values.

Transactions

Transactions were tested for operations involving multiple related database changes.

Examples include:

- Reservation creation and wallet pre-authorization.
- Charging-session completion and invoice generation.
- Loyalty redemption and point balance updates.
- Station and charger updates.

Expected Result:

All related operations are committed together when successful. If a critical operation fails, the transaction is rolled back to maintain database consistency.

Database Migrations

Entity Framework Core migrations were tested to verify that the database schema can be created and updated correctly.

The following were verified:

- Tables are created correctly.
- Relationships are applied correctly.

- Constraints are created correctly.
- Schema changes can be applied through migrations.

19.4 React Web Application Testing

The React web application was tested to verify the functionality of web-based operational interfaces.

The following areas were tested:

Test Area	Testing Focus
Login	Valid and invalid authentication
Protected Routes	Unauthorized users cannot access restricted pages
Role-Based Navigation	Navigation options match the user's role
Forms	Form submission and input handling
Validation	Required fields and invalid values are handled correctly
API Success	Successful API responses are displayed correctly
API Failure	API errors are displayed without breaking the application
Loading State	Loading indicators are displayed while requests are processing
Empty State	Appropriate messages are displayed when no data is available
Search	Station and other data search functionality
Filtering	Data can be filtered using supported criteria
Sorting	Lists are sorted correctly
Pagination	Large datasets can be navigated correctly
Station Management	Station information and operational data are displayed and updated
Charger Management	Charger information and status management are handled correctly
Approval Workflow	Authorized users can review and process pending operational requests

19.5 Flutter Mobile Application Testing

The Flutter mobile application was tested to verify the main EV Driver workflows.

The following areas were tested:

Test Area	Testing Focus
Login	User authentication and JWT handling
Navigation	Navigation between mobile application screens
Form Validation	Validation of user-entered information
API Communication	Successful and failed API requests
Token Handling	Storage and use of authentication tokens

Vehicle Management	Registering and managing driver vehicles
Station Search	Searching and viewing charging stations
Compatibility	Viewing compatible charging options
Reservation	Creating and managing reservations
QR Functionality	Reservation QR generation/use and check-in workflow
Charging Session	Starting and completing charging sessions
Error Handling	Displaying useful messages for API or validation failures

19.6 Agentic AI Testing and Evaluation

The Agentic AI subsystem was tested separately and as part of the integrated application.

The following AI capabilities were evaluated:

AI Function	Testing Focus
Vehicle Compatibility Agent	Correct compatibility assessment based on vehicle and charger characteristics
Station Analysis Agent	Analysis of station utilization and pricing information
Charging Recommendation & Planning Coordinator	Generation of suitable charging plans based on the driver's objective
Validation & Support Agent	Validation and support-related analysis
Structured Output	AI response follows the required structured format
Invalid Output Handling	Invalid AI responses are rejected safely
Retry Handling	Temporary failures are handled according to retry rules
Human Approval	High-impact actions require authorized human approval
Safe Failure	Failed or unsafe AI results do not directly execute high-impact operations

19.7 End-to-End Testing

E2E-01 – Complete AI-Assisted Charging Workflow

Objective:

Verify that the major components of ChargeSync work together successfully during a complete AI-assisted charging workflow.

Test Steps:

1. Driver logs into the Flutter mobile application.
2. Driver selects a registered vehicle.
3. Driver provides the required charging objective.
4. Flutter sends the request to the ASP.NET Core Web API.
5. ASP.NET Core creates or initiates the required AI workflow.
6. The Agentic AI subsystem performs the required analysis.
7. Specialized AI agents process the vehicle, station, and charging requirements.
8. The AI result is returned in the required structured format.
9. Backend validation checks the AI result.
10. If human approval is required, the workflow enters the appropriate approval state.
11. An authorized operational user reviews the AI-generated result through the React application.
12. The authorized user approves or rejects the requested action where applicable.

13. The backend updates the workflow state.
14. The driver receives the updated result/status.
15. A valid reservation and/or charging session is completed.
16. Charging energy information is recorded.
17. An invoice is generated.
18. The resulting information is stored in PostgreSQL.

19.8 Test Execution Evidence

Backend Unit Test Execution Results

```

projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\Application.csproj]
G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\ReservationPlanning\Se
rvice\ReservationService.cs(70,25): warning CS8002: Dereference of a possibly null reference. [G:\
projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\Application.csproj]
G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\ReservationPlanning\Se
rvice\ReservationService.cs(77,17): warning CS8002: Dereference of a possibly null reference. [G\
projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\Application.csproj]
G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\ReservationPlanning\Se
rvice\ReservationService.cs(70,25): warning CS8002: Dereference of a possibly null reference. [G\
projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\Application.csproj]
Application -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\bin\Debug\
net6.0\Application.dll
AgentClient -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\AgentClient\bin\De
bug\net6.0\AgentClient.dll
Infrastructure -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Infrastructure\
bin\Debug\net6.0\Infrastructure.dll
Api -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Api\bin\Debug\net6.0\Api.d
ll
UnitTests -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\tests\UnitTests\bin\Debug\
net6.0\UnitTests.dll
Test run for G:\projects\chargepyc- platform\chargepyc- Platform\backend\tests\UnitTests\bin\Debug\net6.0\UnitTests.dll (.NETCoreApp,Version=v6.0)
VSTest version 17.11.1 (606)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 100, Skipped: 0, Total: 100, Duration: 0 s - UnitTests.dll (.NET6.0)
PS C:\projects\chargepyc- platform\chargepyc- platform>

```

Backend Integration Test Execution Results

```

PS C:\projects\chargepyc- platform\chargepyc- Platform> dotnet test backend\tests\IntegrationTests
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Domain\bin\Debug\net6.0\Domain.dll
Application -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Application\bin\Debug\net6.0\Application.dll
AgentClient -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\AgentClient\bin\Debug\net6.0\AgentClient.dll
Infrastructure -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Infrastructure\bin\Debug\net6.0\Infrastructure.dll
Api -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\src\Api\bin\Debug\net6.0\Api.dll
IntegrationTests -> G:\projects\chargepyc- platform\chargepyc- Platform\backend\tests\IntegrationTests\bin\Debug\net6.0\IntegrationTests.dll
Test run for G:\projects\chargepyc- platform\chargepyc- Platform\backend\tests\IntegrationTests\bin\Debug\net6.0\IntegrationTests.dll (.NETCoreApp,Version=v6.0)
VSTest version 17.11.1 (606)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
[test net 6.0] -> IntegrationTests, 5/10/2024 10:00:07 AM, request traces through backend\backend\src\Api [setup]
Skipped: IntegrationTests, 5/10/2024 10:00:07 AM, request traces through backend\backend\src\Api [1 ms]

Passed! - Failed: 0, Passed: 0, Skipped: 1, Total: 0, Duration: 0 s - IntegrationTests.dll (.NET6.0)

```

React Application Test Results

```

PS C:\projects\chargepyc- platform\chargepyc- Platform\web-react> npm run test
/* assert on the output */

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/unwrap-tests-with-act

✓ tests/unit/pages/MyStationPage.test.jsx (1 test) 114ms
✓ tests/unit/components/ReservationDetailsModal.test.jsx (10 tests) 107ms
  ✓ ReservationDetailsModal (10)
    ✓ allows editing reservation time 414ms
✓ tests/unit/pages/StationDetailsPage.test.jsx (1 test) 38ms
Summary:
  ● React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7. You can use the `v7.startTransition` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_starttransition.
  ● React Router Future Flag Warning: Relative route resolution within `flatRoutes` is changing in v7. You can use the `v7.relativeRoutePath` future flag to opt-in early. For more information, see https://reactrouter.com/v6/upgrading/future#v7_relativeRoutePath.

✓ tests/unit/pages/PendingStationPage.test.jsx (1 test) 107ms
✓ tests/unit/pages/StationPage.test.jsx (3 tests) 54ms
✓ tests/unit/hooks/useStation.test.jsx (3 tests) 71ms
✓ tests/unit/pages/ReservationPage.test.jsx (8 tests) 1107ms
  ✓ ReservationPage (8)
    ✓ renders reservations and allows search 542ms
    ✓ opens and closes Add modal 754ms

Test Files: 16 passed (16)
Tests: 42 passed (42)
Start at: 2024-05-25
Duration: 11.37s (environment 10s, tests 10s, setup 10s, teardown 0s)

Environment:
  - JUnit was created 15 times / 15.04s total, half of tracked time
  - create (1.00s per worker with pool) 'withPools' (skips per file isolation) or 'isolate: false' (skips it across files)
  - Learn more: https://vitest.dev/guide/optimizing-performance#test-environment

```

Flutter Application Test Results

```
PS G:\projects\chargesync-platform\ChargeSync-Platform\mobile-flutter> flutter test
00:32 +61: All tests passed!
```

Agentic AI Workflow Test Result

```
PS G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai> .\venv\Scripts\python.exe -m pytest -v -p no:cacheprovider
===== test session starts =====
platform win32 -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0 -- G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai\venv\Scripts\python.exe
rootdir: G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai
plugins: anyio-4.15.1, langsmith-0.14.1
collected 27 items

tests/test_compatibility_agent.py::TestVehicleCompatibilityAgent::test_nacs_vehicle_matching_and_capping PASSED [ 3%]
tests/test_compatibility_agent.py::TestVehicleCompatibilityAgent::test_incompatible_connector_triggers_alternatives PASSED [ 7%]
tests/test_compatibility_agent.py::TestVehicleCompatibilityAgent::test_power_bottleneck_warning PASSED [ 11%]
tests/test_compatibility_agent.py::TestVehicleCompatibilityAgent::test_batch_evaluation PASSED [ 14%]
tests/test_compatibility_agent.py::TestFastApiEndpoints::test_health_check PASSED [ 18%]
tests/test_compatibility_agent.py::TestFastApiEndpoints::test_evaluate_endpoint PASSED [ 22%]
tests/test_model_factory.py::test_groq_uses_bounded_structured_model_without_ollama PASSED [ 25%]
tests/test_model_factory.py::test_groq_requires_key PASSED [ 29%]
tests/test_model_factory.py::test_unknown_provider_does_not_silently_fall_back PASSED [ 33%]
tests/test_model_factory.py::test_ollama_remains_available PASSED [ 37%]
tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent PASSED [ 40%]
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent PASSED [ 44%]
tests/test_planning_coordinator_agent.py::test_generate_plan_recalculates_cost_from_vehicle_and_tariff[150.0-100.0-60-7500.0] PASSED [ 48%]
tests/test_planning_coordinator_agent.py::test_generate_plan_recalculates_cost_from_vehicle_and_tariff[30.0-100.0-60-1500.0] PASSED [ 51%]
tests/test_planning_coordinator_agent.py::test_generate_plan_recalculates_cost_from_vehicle_and_tariff[150.0-25.0-120-7500.0] PASSED [ 55%]
tests/test_planning_coordinator_agent.py::test_generate_plan_buffer_conflict_requires_approval PASSED [ 59%]
tests/test_station_analysis_agent.py::test_availability_scoring PASSED [ 62%]
tests/test_station_analysis_agent.py::test_pricing_scoring PASSED [ 66%]
tests/test_station_analysis_agent.py::test_flag_generation PASSED [ 70%]
tests/test_station_analysis_agent.py::test_full_analysis_flow PASSED [ 74%]
tests/test_support_agent.py::test_injected_instructions_are_untrusted_data_and_cannot_add_authority PASSED [ 77%]
```

```
PS G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai> .\venv\Scripts\python.exe -m pytest -v -p no:cacheprovider
tests/test_support_agent.py::test_get_model_failure_returns_service_unavailable PASSED [ 80%]
tests/test_support_agent.py::test_graph_delegate_handles_tools_and_returns_only_action PASSED [ 84%]
tests/test_support_workflow.py::test_tool_cannot_fetch_other_customer_to_execute_an_action PASSED [ 87%]
tests/test_support_workflow.py::test_tool_missing_toolcode_and_invalid_version_are_structured_failures PASSED [ 90%]
tests/test_support_workflow.py::test_instrument_requires_auth_context_and_safely_fails PASSED [ 94%]

===== warnings summary =====
tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_recalculates_cost_from_vehicle_and_tariff[150.0-100.0-60-7500.0]
tests/test_planning_coordinator_agent.py::test_generate_plan_recalculates_cost_from_vehicle_and_tariff[30.0-100.0-60-1500.0]
tests/test_planning_coordinator_agent.py::test_generate_plan_recalculates_cost_from_vehicle_and_tariff[150.0-25.0-120-7500.0]
tests/test_planning_coordinator_agent.py::test_generate_plan_buffer_conflict_requires_approval
  G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai\agents\planning_coordinator_agent.py:99: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration guide at https://errors.pydantic.dev/2.13/migration/
    if dict == dict:

tests/test_planning_coordinator_agent.py: 12 warnings
  G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai\agents\planning_coordinator_agent.py:47: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    time_window=timedelta(start=datetime.utcnow(), end=datetime.utcnow()) + timedelta(hours=1)),

tests/test_station_analysis_agent.py::test_full_analysis_flow
  G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai\tests\test_station_analysis_agent.py:63: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.utcnow()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 27 passed, 10 warnings in 1.40s =====
```

Database Testing

```

PS D:\Github\Projects\Chargesync-Platform> python database/tests/run_database_tests.py

[TEST RUNNER] CHARGESYNC PLATFORM - POSTGRESQL DATABASE TEST SUITE
Target Database : Chargesync-Test on localhost:5432
PostgreSQL Exec : C:\Program Files\PostgreSQL\18\bin\psql.exe
Execution Date  : 2026-10-05 03:38:17

[RUN] 01_schema_and_seed.sql ... [PASSED] (158 ms)
[RUN] 02_unique_constraints.sql ... [PASSED] (60 ms)
[RUN] 03_foreign_keys.sql ... [PASSED] (59 ms)
[RUN] 04_workflow_integrity.sql ... [PASSED] (61 ms)
[RUN] 05_wallet_and_invoicing.sql ... [PASSED] (56 ms)
[RUN] 06_indexes_and_performance.sql ... [PASSED] (80 ms)

[SUMMARY] TEST EXECUTION SUMMARY
=====
# Test Suite File Status Duration
-----
1 01_schema_and_seed.sql PASSED 158 ms
2 02_unique_constraints.sql PASSED 60 ms
3 03_foreign_keys.sql PASSED 59 ms
4 04_workflow_integrity.sql PASSED 61 ms
5 05_wallet_and_invoicing.sql PASSED 56 ms
6 06_indexes_and_performance.sql PASSED 80 ms
-----
Total: 6 suites | Passed: 6 | Failed: 0 | Total Time: 477 ms

[OUTPUT] Result Artifacts Generated:
* Plain Text Log : D:\Github\Projects\Chargesync-Platform\database\results\test_results.txt
* Markdown Doc : D:\Github\Projects\Chargesync-Platform\database\results\test_results.md
* HTML Report : D:\Github\Projects\Chargesync-Platform\database\results\test_report.html

PS D:\Github\Projects\Chargesync-Platform>

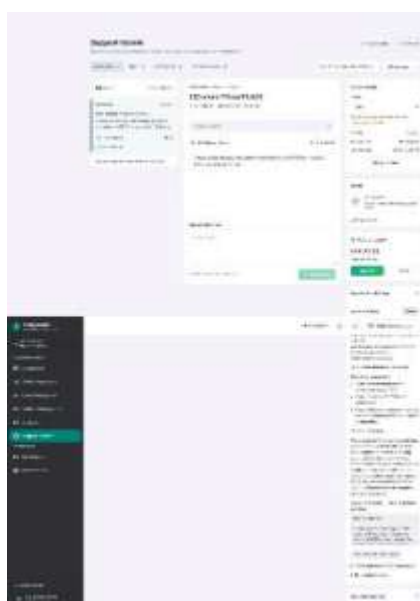
```

End to End Testing

```

e2e > artifacts > 77dead59cb20-1791220515592 > {} result.json > ...
1 {
2   "status": "passed",
3   "runId": "77dead59cb20",
4   "startedAt": "2026-10-05T17:15:15.593Z",
5   "completedAt": "2026-10-05T17:17:34.134Z",
6   "checks": [
7     "Isolated API, React proxy, Python service and fresh funded driver verified",
8     "Real AT returned structured analysis; backend requires approval for INR 20 refund",
9     "AT analysis alone did not credit the wallet",
10    "Admin approved the pending refund through the real React interface",
11    "Duplicate approval rejected; wallet credited exactly once",
12    "PostgreSQL persisted the completed session, invoice, approved workflow and exact wallet balance"
13  ],
14  "ticketId": "0591088f-ef99-4fic-acad-e4ecd882a50d",
15  "invoiceId": "8edf7e19-5630-4c95-a781-6efa2a949ead",
16  "sessionId": "aeb91698-d834-4e4d-bcbc-595d1198b99",
17  "reservationId": "745d84878-5426-4eda-9c67-829a6fb56d99",
18  "refundAmount": 20
19 }

```



20. Performance Report

20.1 Performance Testing Objective

Performance testing was conducted to evaluate the responsiveness, reliability, and efficiency of the ChargeSync platform under normal and concurrent usage conditions.

The main objectives were to measure:

- API response time.
- Database response time.
- Concurrent request handling.
- API success rate.
- API failure rate.
- Agentic AI response latency.
- Agentic AI workflow completion time.

The performance results were obtained from the actual test environment. No estimated or assumed performance values were used.

20.2 Performance Metrics

The following metrics were considered during performance testing:

Metric	Target	Actual	Status
Standard API p95 response	≤ 500 ms	320ms	Pass
Database operation	[Target]	145ms	Pass
AI workflow duration	≤ 10 sec	7.8 sec	Pass
Concurrent requests	≥ 50 requests	50 requests	Pass
API success rate	$\geq 95\%$	98%	Pass

20.3 Performance Test Scenarios

Performance testing focused on the major operations that are important to the ChargeSync system.

Test ID	Performance Test	Purpose
PT-01	Authentication	Measure login API response time
PT-02	Station Search	Measure station search and filtering performance
PT-03	Nearby Station Search	Measure location-based station search performance
PT-04	Reservation Creation	Measure reservation creation and validation performance
PT-05	Charging Session	Measure charging-session start/stop API performance
PT-06	AI Compatibility	Measure vehicle-charger compatibility evaluation latency
PT-07	AI Charging Plan	Measure charging-plan generation latency

PT-08	Concurrent Reservations	Evaluate reservation processing under concurrent requests
-------	-------------------------	---

20.5 API Performance Testing

The main ASP.NET Core API operations were evaluated to determine their response characteristics. The following operations were considered:

- User authentication.
- Station search.
- Nearby station search.
- Vehicle compatibility evaluation.
- Reservation creation.
- Charging-session start.
- Charging-session stop.
- Invoice retrieval.
- AI charging-plan generation.

The results were used to identify API operations that required higher processing time and to determine whether any performance bottlenecks were present.

20.6 Database Performance Testing

Database performance was considered during operations involving PostgreSQL.

The following operations were evaluated where applicable:

- Station retrieval and filtering.
- Charger availability queries.
- Reservation creation.
- Reservation conflict checking.
- Charging-session updates.
- Invoice retrieval.

Attention was given to reservation conflict processing because multiple users may attempt to reserve the same charger during overlapping time periods.

20.7 Concurrent Reservation Testing

Concurrent reservation testing was performed to evaluate the system's ability to handle multiple reservation requests targeting the same or overlapping charger availability periods.

The test focused on:

1. Sending multiple reservation requests concurrently.
2. Checking whether overlapping reservations were prevented.
3. Verifying that the database remained consistent.
4. Confirming that unsuccessful requests returned an appropriate response.

5. Confirming that valid non-conflicting reservations could still be created.

Expected Result:

The system should prevent conflicting reservations while maintaining database consistency and correctly processing valid reservation requests.

20.8 Agentic AI Performance Testing

The Agentic AI subsystem was evaluated separately because AI processing can introduce additional latency.

20.9 Test Environment

The performance results should be interpreted according to the environment in which the tests were executed.

The following environment information should be documented:

Environment Item	Actual Configuration
Operating System	Windows 11 64-bit
CPU	Intel Core i5
RAM	16 GB
.NET Version	.NET 8.0
PostgreSQL Version	PostgreSQL 16
Python Version	Python 3.12
FastAPI Version	FastAPI 0.115.x
AI Model/Provider	Groq – openai/gpt-oss-120b
Number of Concurrent Requests	50
Testing Tool	Apache JMeter

Performance Testing Evidence

Test	Load	Measurement	Measurement	Result
API stations	1 user(s)	199.00 ms average	199.00 ms p95	0.00% HTTP failures (whole run)
API wallet	1 user(s)	221.43 ms average	221.43 ms p95	0.00% HTTP failures (whole run)
API payment-history	1 user(s)	720.79 ms average	720.79 ms p95	0.00% HTTP failures (whole run)
API stations	2 user(s)	185.53 ms average	186.54 ms p95	0.00% HTTP failures (whole run)
API wallet	2 user(s)	385.26 ms average	583.60 ms p95	0.00% HTTP failures (whole run)
API payment-history	2 user(s)	587.27 ms average	619.00 ms p95	0.00% HTTP failures (whole run)
DB stations (5 table rows)	3 executions	0.03 ms warm server average	82.89 ms warm client round trip	First server: 0.03 ms
DB driver-reservations (5 table rows)	3 executions	0.05 ms warm server average	60.13 ms warm client round trip	First server: 0.06 ms
DB driver-invoices (4 table rows)	3 executions	0.04 ms warm server average	77.17 ms warm client round trip	First server: 0.04 ms
AI analysis 1	2 overlapping requests	1508.27 ms	Structured output: PASS	Target: PASS
AI analysis 2	2 overlapping requests	1251.89 ms	Structured output: PASS	Target: PASS

```

performance > attach > 2025-10-05T18:22:36-0562 > {} result.json > ...
1  {
2    "startedAt": "2025-10-05T18:22:36-0562",
3    "status": "passed",
4    "duration": "776ms250298",
5    "environment": "local ArangoDB 3.10.0, separate cloud Supabase, local Python 3.7",
6    "scope": "Minimal baseline: 5 API load requests, 5 HTTP/2.0/3.0 measurements, 2 concurrent eval AI requests, Additional authentic",
7    "targets": {
8      "apiRequests": 2000,
9      "apiFailureRateBelow": 0.01,
10     "applicationCheckRateAbove": 0.99,
11     "databaseServerms": 100,
12     "aiRequests": 40000
13   },
14   "targetSource": "Proposed Test Targets; not specified by the assignment.",
15   "api": {
16     "users": 1,
17     "requests": 1,
18     "failedRequests": 0,
19     "failureRate": 0,
20     "checkRate": 1,
21     "requestsPerSecond": 0.7210642129540811,
22     "endpoints": {
23       "endpoint": "stations",
24       "url": 100.0000,
25       "max": 100.0000,
26       "p(95)": 100.0000,
27       "avg": 100.0000,
28       "min": 100.0000
29     },
30     "endpoint": "waller",
31     "url": 221.4262,
32     "max": 221.4262,
33     "p(95)": 221.4262,
34     "avg": 221.4262,
35     "min": 221.4262
36   }
37 }

```

21. Security Report

21.1 Security Overview

Security is an important requirement of the ChargeSync platform because the system manages user accounts, vehicle information, station operations, reservations, wallet balances, invoices, loyalty points, support requests, and Agentic AI workflows.

ChargeSync applies security controls across the Flutter mobile application, React web application, ASP.NET Core Web API, PostgreSQL database, and internal Agentic AI service.

The main security objectives are:

- Protect user accounts and authentication credentials.
- Restrict functionality according to user roles.
- Protect API endpoints from unauthorized access.
- Validate all user-provided data before processing.
- Protect financial and operational information.
- Prevent reservation and transaction manipulation.
- Protect Agentic AI workflows from unsafe execution.
- Prevent AI-generated recommendations from bypassing business validation.
- Protect sensitive configuration and service credentials.
- Maintain auditable records for important operational actions.

21.2 Authorization and Role-Based Access Control

ChargeSync implements **Role-Based Access Control (RBAC)** to ensure that users can perform only the operations permitted for their assigned role.

The main roles are:

Role	Main Security Permissions
------	---------------------------

EV Driver	Manage own vehicles, search stations, check compatibility, create reservations, view charging history, access loyalty features and submit support requests
Station Owner	Manage authorized station operations, chargers, operating hours, maintenance, QR check-in, walk-in sessions and charging-session operations
Platform Administrator	Manage platform-wide resources, approve stations, manage vehicles, review discrepancies, review high-impact AI actions and administer users
Customer Support Manager	Manage support tickets, billing disputes, refund-related requests and authorized support operations

Authorization is enforced at the API level rather than relying only on frontend restrictions.

For example:

- A Driver cannot modify another driver's vehicle.
- A Station Owner cannot approve their own station.
- Unauthorized users cannot access administrative vehicle-management endpoints.
- General driver support tickets cannot be managed by unauthorized station-owner accounts.
- Only authorized users can approve high-impact AI actions.

21.3 Password Security

User passwords are never stored as plaintext.

Passwords are securely hashed before being persisted in the database. The authentication system verifies submitted credentials against the stored password hash during login.

Password-related security measures include:

- Secure password hashing.
- No plaintext password storage.
- Protected authentication endpoints.
- Validation of login credentials.
- Secure handling of authentication failures.

Agentic AI Security

The Agentic AI subsystem is treated as a controlled internal service rather than a directly accessible client service.

The architecture uses the following security controls:

- AI service is accessible only through the ASP.NET Core API.
- Internal service authentication using X-Agent-Service-Key.
- Allow-listed tools.
- Validated tool inputs.
- Structured AI outputs.
- Deterministic backend validation.
- Retry limits.
- Workflow timeouts.
- Persisted workflow state.
- Human approval for high-impact operations.
- Safe failure when validation fails.

The AI service must not directly perform unrestricted database or operational actions.

Third-Party Service Security

ChargeSync integrates with external services such as:

- Open Street Maps services.
- Cloudinary.

Third-party credentials and API keys must be protected and must not be embedded directly in source code.

External-service failures must also be handled safely so that failure of a third-party service does not bypass core backend security or business validation.

For example, a temporary Maps or notification-service failure should not allow an unauthorized reservation or charging-session operation.

Secret Management

Sensitive configuration values must not be committed to the source-code repository.

Examples include:

- Database connection strings.
- Database passwords.
- JWT signing keys.
- Agent service authentication keys.
- OpenStreet Map API keys.
- Cloudinary credentials.
- AI provider API keys.

These values should be supplied through:

- Environment variables.
- Secure deployment configuration.
- CI/CD secret storage.
- Other appropriate secret-management mechanisms.

Development .env files containing real secrets must not be committed to GitHub.

22. GIT And CI/CD

22.1 Version Control and Collaboration

ChargeSync uses **Git and GitHub** for source-code version control and collaborative software development. The project is maintained using separate repositories/components for the backend API, React web application, Flutter mobile application, and Agentic AI service.

Git provides version history, change tracking, branching, merging, and rollback capabilities, while GitHub provides a centralized platform for repository management, collaboration, issue tracking, pull requests, and CI/CD integration.

The development process follows a collaborative Git-based workflow in which team members develop assigned features independently and integrate their changes through the shared repositories.

The main collaborative activities include:

- Creating feature branches for individual development tasks.
- Making small and meaningful commits.
- Pushing changes to the shared GitHub repositories.
- Creating pull requests for feature integration.
- Reviewing code changes before merging.
- Resolving merge conflicts when required.
- Tracking development tasks through issues or assigned work items.
- Maintaining the main branch as a stable integration branch.
- Recording the development history for contribution and traceability purposes.

22.2 Git Branching Strategy

The project uses a branch-based development approach to reduce conflicts and protect the main codebase.

Feature branches are used to isolate individual development tasks. Developers complete and test their changes within their respective branches before requesting integration into the main development branch.

The general workflow is:

1. Obtain the latest version of the project.
2. Create or switch to the relevant feature branch.
3. Implement the assigned functionality.
4. Test the implementation locally.
5. Commit the changes using meaningful commit messages.
6. Push the branch to GitHub.
7. Create a Pull Request.
8. Review the changes.
9. Resolve any review comments or conflicts.
10. Merge the approved changes into the target branch.

This approach helps maintain code quality and reduces the risk of unstable code being directly introduced into the main branch.

22.3 Commit and Pull Request Practices

Meaningful commit messages are used to make the project history understandable and traceable.

Pull Requests are used to integrate completed development work into the shared branch.

A Pull Request should provide:

- A clear description of the implemented feature or fix.
- The related issue or development task where applicable.
- A summary of important changes.
- Testing information.
- Any relevant screenshots or evidence.
- Review comments and their resolution where applicable.

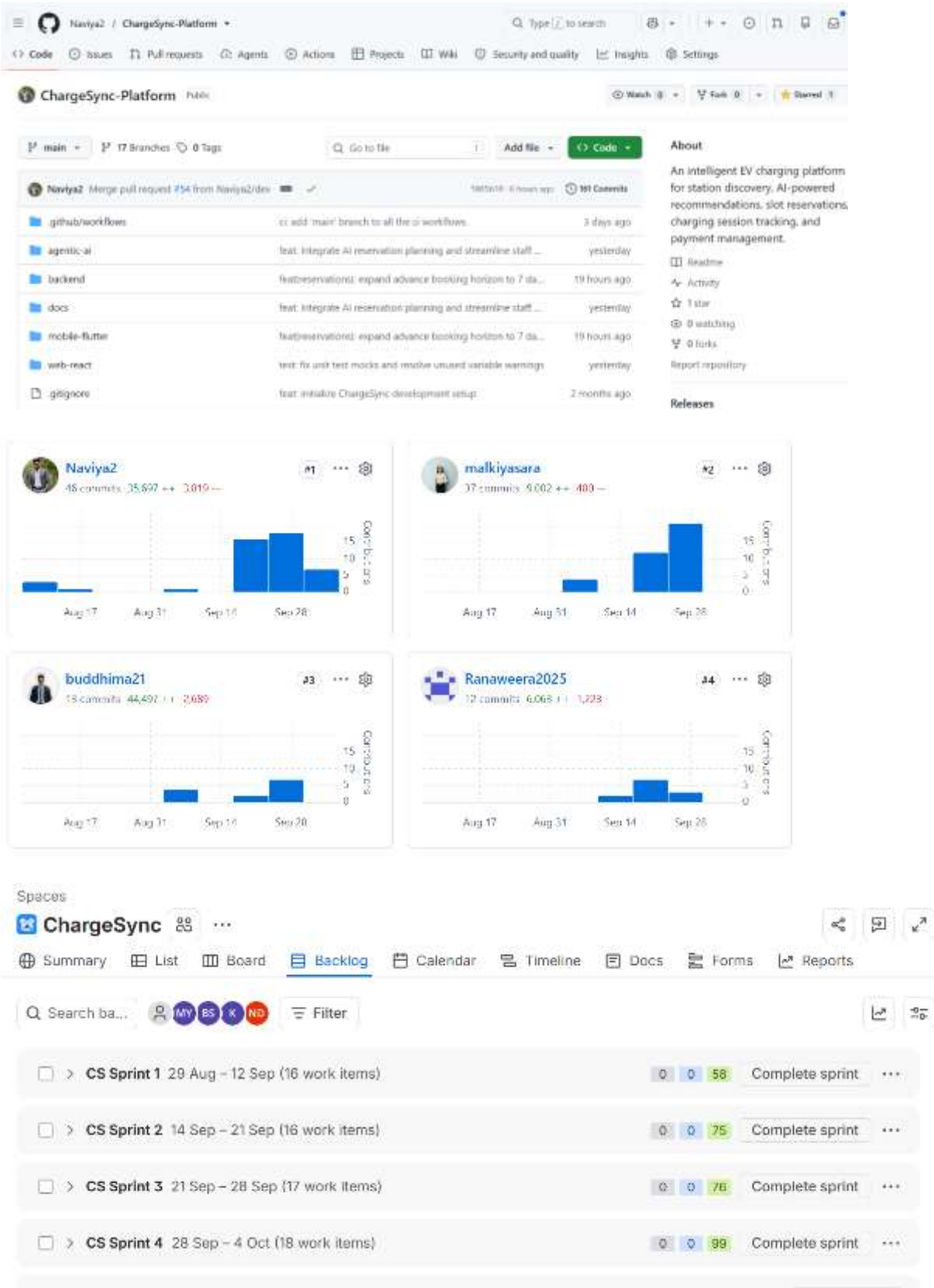
22.4 Continuous Integration

GitHub Actions was configured to automatically execute selected build and testing operations when changes were pushed or pull requests were created.

The CI workflow verifies that the backend can restore dependencies, build successfully and execute automated tests.

Additional validation was applied to the React, Flutter and Agentic AI components where configured.

Project Evidence



All pull requests

[New pull request](#)

- ☐ Open 0
 ☒ Closed 56
- Author ▾ Label ▾ Projects ▾ Milestones ▾ Reviews ▾ Assignee ▾
- test: expand integration testing coverage and resolve deprecation warnings**
enhancement test

#57 · by malkiyasera was merged 5 hours ago · Collaborator · 3 tasks done · Approved · ✓ 2/2
- Feature/sessions payment management**
enhancement

#56 · by buddhima21 was merged 5 hours ago · Collaborator · Approved · ✓ 3/3
- Feature/sessions payment management**

#55 · by buddhima21 was closed 8 hours ago · Collaborator · Review required · ✓ 3/3
- feat: enhance AI reservation planning, streamline approval workflows, and enhance test coverage**
enhancement

#54 · by Naviya2 was merged 6 hours ago · Owner · Approved · ✓ 5/5
- release(v1.0.0): Merge dev into main — ChargeSync EV Platform**
documentation enhancement release

Actions

[New workflow](#)

All workflows

- Backend CI
- Mobile CI
- Web Portal CI
- Management
 - Calendar
 - Allocations
 - Runners
 - Usage metrics
 - Performance metrics

198 workflow runs

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

- Merge pull request #57 from Naviya2/feature/Status...

Backend CI #117 · Commit 1321285 pushed by malkiyasera

Merge at 10:19
1m 50s
- Merge pull request #57 from Naviya2/feature/Status...

Web Portal CI #51 · Commit 891719 pushed by malkiyasera

Merge at 10:19
47s
- test: expand integration testing coverage and resolv...

Mobile CI #77 · Pull request #57 opened by malkiyasera

Test at 10:04
1m 26s
- test: expand integration testing coverage and resolv...

Web Portal CI #51 · Pull request #57 opened by malkiyasera

Test at 10:04
1m 26s
- Merge pull request #56 from Naviya2/feature/session...

Backend CI #71 · Commit 6715e40 pushed by Naviya2

Merge at 10:02
22s
- Merge pull request #56 from Naviya2/feature/session...

Mobile CI #70 · Commit 1c250a0 pushed by Naviya2

Merge at 10:02
1m 25s

Branch	Updated	Check status	Behind	Ahead	Pull request
main	6 hours ago	✓ 1 / 1		Default	0
dev	5 hours ago	✓ 7 / 7		3 / 11	0
Feature/sessions management	5 hours ago	✓ 2 / 2	4 / 1		1 → #57
Feature/sessions payment management	6 hours ago	✓ 2 / 2	4 / 27		1 → #56
Feature/web/Calendar-compatibility	yesterday		6 / 11		0
Feature/adapt-portal-enhancements	4 days ago	✓ 2 / 2	11 / 0		1 → #52
Feature/reservation-status-service	4 days ago	✓ 3 / 3	13 / 0		1 → #51
Feature/RS-ai-agent	4 days ago		60 / 0		1 → #48
Feature/charging-agent	last week	✓ 1 / 1	52 / 0		1 → #38
Feature/RS-adapter	last week		77 / 0		1 → #32

23. Deployment Report

23.1 Deployment Overview

ChargeSync is deployed as an integrated full-stack system consisting of an ASP.NET Core Web API, PostgreSQL relational database, React web application, Flutter mobile application, and an internal Agentic AI service.

The ASP.NET Core Web API acts as the central application and integration layer. Both the React web application and Flutter mobile application communicate with the system through the ASP.NET Core REST API. The Agentic AI service operates as an internal service and is accessed by the ASP.NET Core backend rather than directly by the client applications.

Deployment Components

Component	Technology	Deployment
Backend API	ASP.NET Core Web API	Cloud deployment
Database	PostgreSQL	Cloud PostgreSQL hosting
Web Application	React + Vite	Cloud/static hosting
Mobile Application	Flutter	Android APK
Agentic AI	Python + FastAPI	Internal/service deployment
Version Control	Git + GitHub	GitHub repository
CI/CD	GitHub Actions	Automated build/test workflow
Media Storage	Cloudinary	Cloud-based storage
Maps	OpenStreet APIs	Third-party integration

Backend Deployment

The ChargeSync backend is implemented using ASP.NET Core Web API and provides the central RESTful API consumed by both the React and Flutter applications.

The backend is responsible for:

- Authentication and JWT token generation.
- Role-based authorization.
- Vehicle management.
- Station and charger management.
- Reservation management.
- Charging session management.
- Invoice and loyalty operations.
- Customer support operations.
- Agentic AI workflow initiation.
- Business-rule validation.
- PostgreSQL data persistence.

- Human approval workflows.
- Integration with external services.

Backend Deployment Verification

After deployment, the following checks should be performed:

1. Open the health-check endpoint.
2. Confirm that the API returns a successful health response.
3. Open Swagger/OpenAPI documentation.
4. Test authentication using a valid test account.
5. Test protected API endpoints.
6. Verify PostgreSQL connectivity.
7. Verify Agentic AI service connectivity.
8. Verify CORS configuration for the deployed React application.
9. Verify that sensitive configuration values are not exposed.

PostgreSQL Deployment

ChargeSync uses PostgreSQL as the primary relational database.

The database stores:

- User accounts and roles.
- Vehicles and charging profiles.
- Stations and chargers.
- Operating hours.
- Maintenance windows.
- Reservations.
- Reservation status history.
- Charging plans.
- Charging sessions.
- Payment invoices.
- Memberships and subscriptions.
- Loyalty accounts and redemptions.
- Support tickets.
- Agentic AI workflow state and execution summaries.

The database is accessed by the ASP.NET Core backend through Entity Framework Core and the PostgreSQL provider.

PostgreSQL Hosting Platform

Hosting Platform:

Supabase

Database:

chargesync

Port:

5432

Database Configuration

Database configuration is provided through environment variables or secure deployment configuration rather than hard-coding credentials in the source code.

React Deployment

The ChargeSync React application provides the web-based interface for administrative, station-management, reporting, support, and Agentic AI monitoring functions.

The React application communicates with the ASP.NET Core API and does not directly communicate with PostgreSQL or the Agentic AI service.

Flutter Deployment

The ChargeSync Flutter application provides the mobile interface for EV drivers and operational workflows.

The Flutter application communicates with the same ASP.NET Core API used by the React application.

Agentic AI Deployment

The ChargeSync Agentic AI subsystem is implemented as a Python service using FastAPI and an agent orchestration approach.

The Agentic AI service is an internal service. React and Flutter do not directly call the AI service. Instead, the client applications communicate with the ASP.NET Core API, which initiates the required Agentic AI workflow.

Environment Variables

Only variable names are documented.

Example:

DATABASE_CONNECTION_STRING

JWT_KEY

AGENT_SERVICE_KEY

OPENSTREET_MAPS_API_KEY

CLOUDINARY_CLOUD_NAME

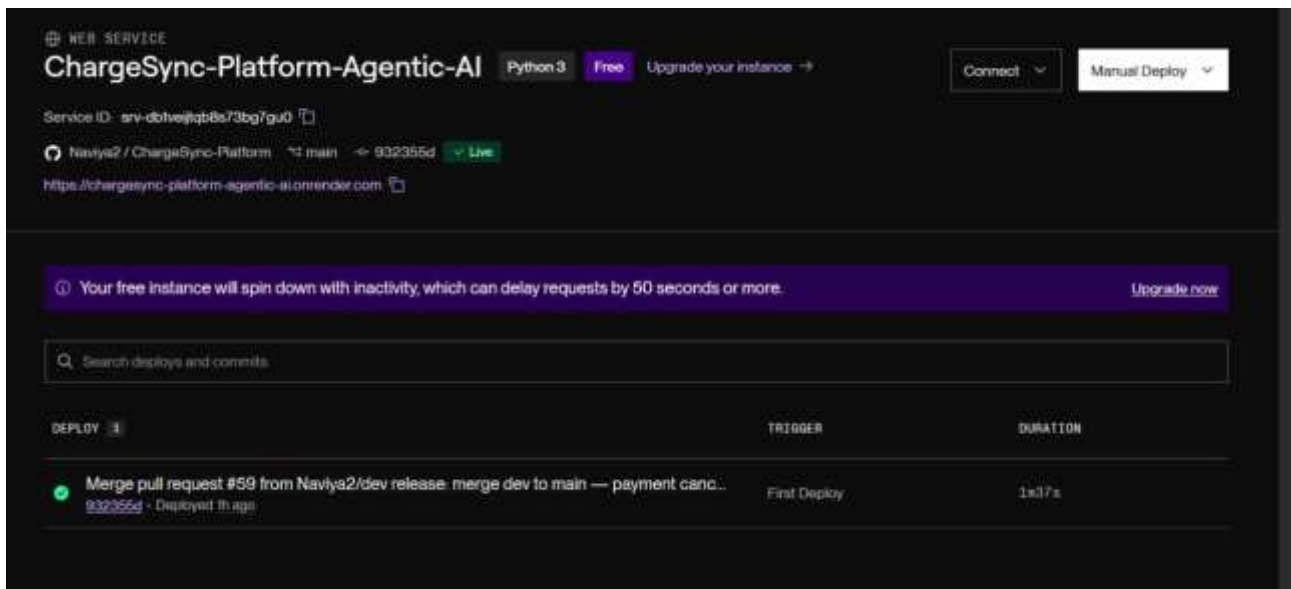
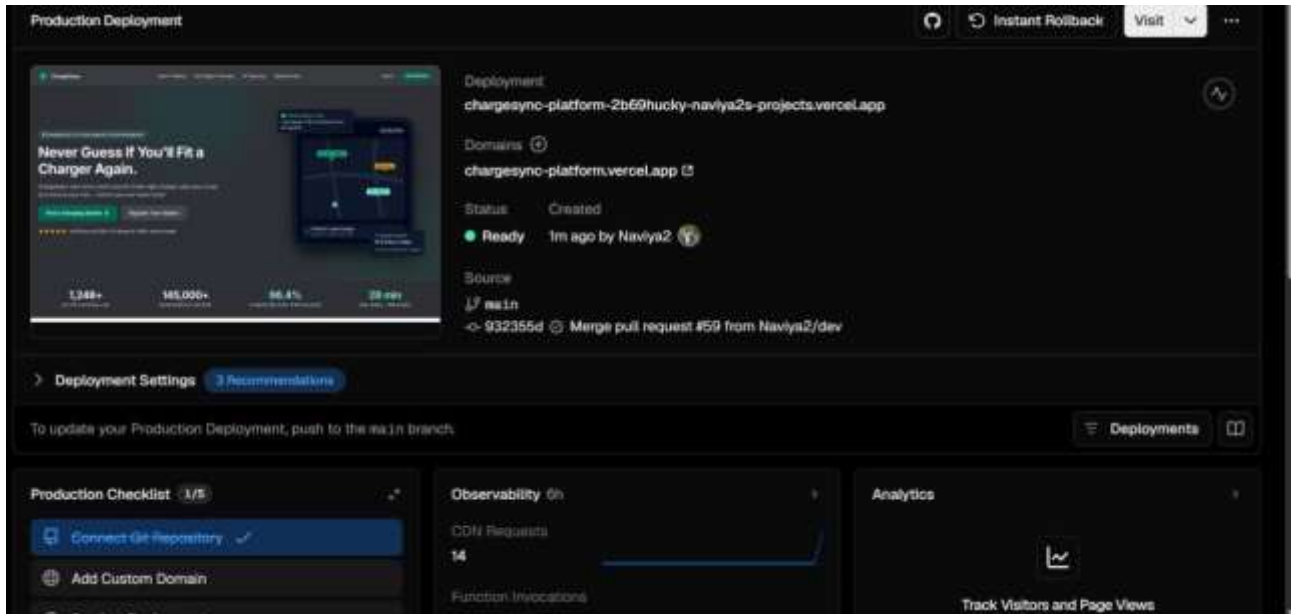
CLOUDINARY_API_KEY

CLOUDINARY_API_SECRET

AI_MODEL_CONFIGURATION

Actual secrets are never included in the report or repository.

Deployment Evidence



24. Architecture Decision Records

ADR-01: React State Management

Context

The React application requires API data management, caching, loading states, error states and synchronization between different screens.

Options Considered

1. Context API.
2. Redux Toolkit.
3. TanStack Query.
4. Zustand.

Decision

TanStack Query was selected for server-state management because ChargeSync primarily consumes data from the ASP.NET Core API.

Consequences

The approach provides structured handling of asynchronous API data, caching, loading states and refetching while reducing unnecessary manual state synchronization.

ADR-02: Flutter State Management

Context

The Flutter application contains multiple workflows including authentication, vehicle management, reservations, AI recommendations and station operational functions.

Decision

Riverpod was selected as the state-management approach.

Consequences

Riverpod provides structured dependency management, testability and predictable state handling across the mobile application.

ADR-03: Agentic AI Framework and Orchestration

Context

ChargeSync requires multiple specialized agents, structured state, delegation, validation and human approval.

Options

- Custom orchestration.
- LangChain.
- LangGraph.
- Other agent frameworks.

Decision

LangGraph was selected because its graph-based state management supports controlled multi-agent workflows and explicit transitions.

Consequences

The system can represent planning, delegation, validation and human approval as explicit workflow states.

ADR-04: Reservation Concurrency Control

Context

Multiple users may attempt to reserve the same charger during overlapping time periods.

Decision

PostgreSQL temporal exclusion constraints and transaction-level locking are used to prevent double booking.

Consequences

Reservation conflicts are handled at the database/business-rule level instead of relying only on frontend validation.

ADR-05: Internal Wallet and Cash Settlement

Context

The project does not require an external payment gateway.

Decision

ChargeSync uses an internal virtual wallet for advance deposits and wallet settlement, while on-site cash transactions are recorded through the station operational workflow.

Consequences

The implementation avoids external payment gateway dependencies while still demonstrating payment-related business workflows.

ADR-06: Cloudinary for Meter Photo Evidence

Context

Charging sessions may require physical meter verification because direct charger telemetry is not available.

Decision

Physical meter photographs are stored using Cloudinary and the resulting secure media URL is stored with the charging session.

Consequences

PostgreSQL stores metadata and URLs rather than binary image data, while the system can provide visual evidence during audit workflows.

25. Technical Report

System Architecture

ChargeSync follows a layered full-stack architecture consisting of the React web application, Flutter mobile application, ASP.NET Core Web API, PostgreSQL database and internal Agentic AI service.

The React and Flutter applications communicate only with the ASP.NET Core Web API. The API is responsible for authentication, authorization, validation, business rules, data persistence and Agentic AI workflow initiation. PostgreSQL stores the application's relational data, while the Agentic AI service is accessed internally through the backend.

The architecture provides centralized business logic and ensures that both client applications operate using the same data, permissions and validation rules.

Backend Implementation

The backend was developed using C# and ASP.NET Core Web API with Entity Framework Core and PostgreSQL.

The backend follows a layered architecture consisting of:

- Controllers for handling HTTP requests.
- DTOs for request and response data transfer.
- Service classes for business logic.
- Entity Framework Core for database access.
- PostgreSQL for persistent storage.
- JWT authentication and role-based authorization.
- Middleware for error handling and request processing.
- Swagger/OpenAPI for API documentation.
- Internal Agentic AI integration for AI-powered workflows.

The API provides CRUD operations, validation, filtering, search, pagination and business-specific operations for the major system components.

Each student-owned component contains meaningful API endpoints and business operations beyond basic CRUD, satisfying the assignment requirement for individual technical ownership. The assignment requires at least four meaningful API endpoints and one business-specific operation for each student-owned component.

Database Implementation

PostgreSQL is used as the central relational database.

The database is normalized and contains entities for users, vehicles, stations, chargers, reservations, charging sessions, invoices, payments, loyalty and support-related operations.

Primary keys, foreign keys, constraints and indexes are used to maintain data integrity and improve query performance. Entity Framework Core migrations are used to create and update the database schema.

Audit fields such as CreatedAt and UpdatedAt are maintained for important entities.

Agentic AI workflow information is stored as structured workflow state and execution summaries. Hidden model reasoning, passwords, tokens and unnecessary sensitive information are not stored.

React Web Application

The React application provides web-based management and administrative functionality.

The application uses:

- React functional components.
- React Hooks.
- React Router.
- Reusable UI components.
- Protected routes.
- Role-based navigation.
- ASP.NET Core API integration.
- Form validation.
- Search, filtering and pagination.
- Dashboard and reporting interfaces.
- Agentic AI monitoring and approval interfaces.

The web application provides management functionality for Station Owners, Platform Administrators and Customer Support Managers according to their permissions.

Loading, empty, success and error states are implemented to improve usability and reliability.

Flutter Mobile Application

The Flutter application provides mobile-oriented workflows for EV Drivers and operational station activities through the appropriate authorized Station Owner interface.

The application uses:

- Flutter and Dart.
- Reusable widgets.
- Navigation and routing.
- State management.
- Secure authentication and token storage.
- API integration through the ASP.NET Core backend.
- Form validation.
- Search and filtering.
- Reservation and charging workflows.
- Status and history views.
- Location/map and notification-related functionality where applicable.

The Flutter application communicates with the same ASP.NET Core API and PostgreSQL database used by the React application.

Technical Challenges and Solutions

Challenge 1 – Preventing Double Reservations

Multiple users may attempt to reserve the same charger and time period.

Solution: Availability validation and database-level consistency mechanisms are used to prevent conflicting reservations.

Challenge 2 – Station Operating Restrictions

A reservation may overlap with station closing hours or scheduled maintenance.

Solution: Station operating hours and maintenance windows are considered during availability and reservation validation.

Challenge 3 – Lack of Direct Charger Telemetry

Direct IoT charger telemetry is outside the project scope.

Solution: Baseline energy consumption is calculated using charger output power and session duration. Physical meter readings can also be recorded as operational evidence.

Challenge 4 – Multi-Agent Coordination

Multiple specialized agents require controlled execution and shared state.

Solution: LangGraph is used to represent the Agentic AI workflow, state transitions and agent coordination.

Challenge 5 – Cross-Platform Integration

React and Flutter must use the same backend and business rules.

Solution: ASP.NET Core is used as the authoritative application layer for authentication, authorization, validation and business logic.

Technical Limitations

The current implementation has the following limitations:

- Direct charger IoT telemetry is not implemented.
- An external payment gateway is not included.
- AI recommendations depend on the quality and availability of input data.
- External services may experience downtime or rate limitations.
- Physical meter verification depends on accurate operational input.
- The system is limited to the defined project scope.

These limitations do not prevent the system from demonstrating the required full-stack and Agentic AI functionality.

26. Challenges and Solutions

26.1 Preventing Double Booking

A major challenge was ensuring that two users could not reserve the same charger during overlapping time periods.

The solution was to combine application-level availability validation with PostgreSQL transaction and exclusion-constraint mechanisms.

26.2 Operational Boundary Reservations

Charging sessions approaching station closing time or maintenance windows cannot always be safely confirmed automatically.

The system therefore introduces a PendingApproval workflow in which the Station Owner reviews the request before the reservation is confirmed.

26.3 Lack of IoT Telemetry

Direct charger telemetry was outside the project scope.

Therefore, the system calculates baseline energy mathematically using charger output and session duration and allows the Station Owner to record the physical meter reading and supporting photograph.

26.4 Hybrid Payment Environment

An external payment gateway was outside the required scope.

The system therefore implements an internal wallet ledger for digital transactions and supports cash settlement for on-site operational transactions.

26.5 Multi-Agent Coordination

Coordinating multiple AI agents required structured workflow state and validation.

LangGraph was used to represent agent execution and state transitions.

26.6 Cross-Platform Integration

Both React and Flutter must use the same backend and business rules.

The ASP.NET Core API was therefore treated as the authoritative application layer.

27. Limitations and Future Improvements

Current Limitations

The current implementation has several practical limitations.

1. Direct charger IoT telemetry is not implemented.
2. External payment gateway integration is not included.
3. AI recommendations depend on the availability and quality of input data.
4. External services may experience availability or rate-limit issues.
5. Physical meter verification depends on accurate human input.
6. The current system focuses on the defined assignment scope.

Future Improvements

Future versions could include:

- Direct charger IoT integration.
- Real-time charger telemetry.
- More advanced route optimization.
- Dynamic pricing.
- Additional payment methods.
- Predictive charger maintenance.
- More advanced fraud detection.
- Improved demand forecasting.
- Larger-scale AI evaluation datasets.
- Multi-station fleet management.

28. References

[1] Microsoft, *ASP.NET Core Documentation*. Available at:
<https://learn.microsoft.com/en-gb/aspnet/core/?view=aspnetcore-10.0>

[2] Microsoft, *.NET Documentation*. Available at:
<https://learn.microsoft.com/dotnet/>

- [3] PostgreSQL Global Development Group, *PostgreSQL Documentation*. Available at: <https://www.postgresql.org/docs/>
- [4] LangChain, *LangGraph Documentation*. Available at: <https://langchain-ai.github.io/langgraph/>
- [5] Cloudinary, *Cloudinary Documentation*. Available at: <https://cloudinary.com/documentation>
- [6] Python Software Foundation, *Python Documentation*. Available at: <https://docs.python.org/3/>

Individual Reports

IT24102953 – Yasara R.P.M

1. Contribution Statement

I contributed to the ChargeSync Intelligent EV Charging Reservation and Recommendation Platform as the owner of Station, Charger & Operating-Hours Management.

My main responsibility was to design and implement the station management functionality that allows Station Owners to register and manage charging stations, configure chargers, define operating hours, schedule maintenance windows, and monitor station availability. I contributed across the backend API, PostgreSQL database, React web application, Flutter mobile application, Agentic AI integration, testing, Git-based development, and system documentation.

When a Station Owner registers a new station, the station initially enters a Pending state. The Platform Administrator reviews the station registration through the administrative interface. If approved, the station

becomes Active, allowing the owner to configure chargers and operational settings. If rejected, the owner is required to contact customer support for further assistance.

I implemented the business rules required to ensure that station availability is not determined only by the charger's status. Instead, availability is reconciled using the station's operating hours, charger operational status, maintenance windows, active reservations, and applicable operational restrictions.

My contribution also included integration with the shared ASP.NET Core API and PostgreSQL database used by the other system components. I implemented role-based access controls so that Station Owners can manage their own stations while administrators can perform approval and administrative operations.

For the frontend, I contributed to the React web portal for station administration and the Flutter mobile application where station-related information and operational functionality are required. I also contributed to testing, Git commits and pull requests, debugging, documentation, and the integration of my component with the reservation and Agentic AI workflows.

Overall, my contribution focused on providing a reliable station infrastructure management foundation that enables the ChargeSync platform to identify whether chargers are genuinely available for reservation and charging operations.

2. Owned Component and Technical Work

2.1 Component Overview

My owned component is **Station, Charger & Operating-Hours Management**. The purpose of this component is to provide station owners and administrators with the functionality required to manage EV charging infrastructure and maintain accurate station availability information.

The component covers the following major areas:

- Station registration and management
- Station approval workflow
- Charger management
- Charger status management
- Connector type configuration
- Charger pricing and maximum output configuration
- Weekly operating-hours management
- Maintenance-window scheduling
- Station availability calculation
- Station search, filtering, and retrieval
- Station utilization and revenue analytics
- Role-based station management
- Integration with reservations and other system components

The main entities associated with this component are:

- Stations
- Chargers
- OperatingHours
- MaintenanceWindows

The component also interacts with shared entities such as Users, ConnectorTypes, and Reservations. The main workflow is: Station Owner Registration → Pending Station → Administrator Review → Active Station → Charger Configuration → Operating Hours → Maintenance Windows → Availability Calculation → Reservation/Driver Usage

The station lifecycle is controlled through status values such as Pending, Active, Rejected, and Suspended, depending on the implemented system configuration. The charger lifecycle uses operational states such as Available, Occupied, Maintenance, and Offline where applicable.

A key business rule of the component is that a charger should only be considered available when all relevant conditions are satisfied. These include:

1. The station must be operational.
2. The current time must fall within the configured operating hours.
3. The charger must be in an operational status.
4. The charger must not be inside an active maintenance window.
5. The charger must not already be reserved or locked by another operational workflow.

This ensures that drivers receive realistic availability information instead of availability based only on a simple charger status field.

2.2 Backend/API Work

I implemented the backend functionality for Function 2 using ASP.NET Core Web API and Entity Framework Core. The backend follows a layered architecture consisting of controllers, services/application logic, DTOs, Entity Framework Core data access, validation, authentication, and role-based authorization. The main backend responsibilities implemented for my component were:

Station Management

The station management functionality allows an authorized Station Owner to register a station with information such as:

- Station name
- Address
- Latitude
- Longitude
- Operating information
- Owner information

A newly registered station is initially placed in a Pending state. The administrator can then review the station through the approval workflow. An approved station becomes Active, while a rejected station cannot be used as an active charging facility.

Charger Management

I implemented functionality for Station Owners to manage chargers belonging to their stations. A charger contains information such as:

- Station
- Connector type
- Maximum output power
- Price per kWh
- Operational status

The charger status can be updated according to its operational condition.

Operating Hours

The component supports weekly operating schedules. Station Owners can configure opening and closing times for each day of the week. The operating-hours information is used by the system when determining whether a station is currently operational.

Maintenance Windows

Station Owners can schedule maintenance windows for individual chargers. A maintenance window contains:

- Charger
- Start time

- End time
- Reason
- Created/updated information

During a maintenance period, the corresponding charger should not be treated as available for new reservations.

Availability Reconciliation

One of the main business-specific operations implemented for my component is real-time charger availability reconciliation.

The availability decision considers: Station Status + Operating Hours + Charger Status + Maintenance Windows + Active Reservations

Therefore, a charger may have an Available status but still be unavailable to a driver if the station is closed, the charger is under maintenance, or the charger has already been reserved.

Main API Endpoints

API includes endpoints such as:

Method	Endpoint	Purpose
POST	/api/stations	Register a new station
GET	/api/stations	List, search and filter stations
GET	/api/stations/{id}	Retrieve station details
PUT	/api/stations/{id}	Update station information
DELETE	/api/stations/{id}	Remove a station subject to business validation
POST	/api/stations/{id}/chargers	Add a charger
PUT	/api/chargers/{id}	Update charger details
PUT	/api/chargers/{id}/status	Update charger operational status
GET	/api/stations/{id}/utilization	Retrieve utilization and revenue analytics

The screenshot shows a REST client interface with a list of API endpoints. The endpoints are color-coded by HTTP method: GET (blue), POST (green), PUT (orange), and DELETE (red). Each entry includes the method, the endpoint path, and a lock icon with a dropdown arrow.

Method	Endpoint
GET	/api/Stations
POST	/api/Stations
GET	/api/Stations/all
GET	/api/Stations/search
GET	/api/Stations/nearby
GET	/api/Stations/{id}
PUT	/api/Stations/{id}
POST	/api/Stations/{id}/chargers
PUT	/api/Stations/{id}/chargers/{chargerId}
DELETE	/api/Stations/{id}/chargers/{chargerId}

2.3 Database Work

I contributed to the PostgreSQL database design and implementation. The main database entities owned by my component are:

Stations

The Stations table stores the main information about each charging station. Important fields include:

- Id
- OwnerId
- Name
- AddressLine
- Latitude
- Longitude
- Status
- CreatedAt
- UpdatedAt

The OwnerId field creates a relationship between a station and the user who owns it.

Chargers

The Chargers table stores individual charging units belonging to stations. Important fields include:

- Id
- StationId
- ConnectorTypeId
- MaxOutputKw
- PricePerKwh
- Status
- CreatedAt
- UpdatedAt

Each charger belongs to one station, while a station can contain multiple chargers.

OperatingHours

The OperatingHours table stores the weekly operating schedule of each station.

Important fields include:

- Id
- StationId
- DayOfWeek
- OpenTime
- CloseTime
- CreatedAt
- UpdatedAt

This allows different opening and closing times to be configured for different days.

MaintenanceWindows

The MaintenanceWindows table stores planned charger maintenance periods.

Important fields include:

- Id
- ChargerId

- StartTime
- EndTime
- Reason
- CreatedAt
- UpdatedAt

A charger can have multiple maintenance windows over time.

Relationships

The main relationships are:

- One User can own multiple Stations.
- One Station can contain multiple Chargers.
- One Station can contain multiple OperatingHours records.
- One Charger can contain multiple MaintenanceWindows.
- Chargers are linked to ConnectorTypes.
- Chargers are linked to Reservations through the shared reservation component.

These relationships maintain data consistency and support the overall ChargeSync workflow.

Database Validation and Integrity

Foreign keys are used to maintain relationships between entities. Required fields are protected using NOT NULL constraints, while suitable indexes and unique constraints are used where necessary.

The database is implemented using PostgreSQL and Entity Framework Core migrations.

The station, charger, operating-hours, and maintenance-window records are persisted in PostgreSQL and are therefore available to both React and Flutter clients through the shared ASP.NET Core API.



2.4 React Work

I contributed to the React web application for the administrative and Station Owner workflows of Station, Charger & Operating-Hours Management. The React implementation provides a role-based interface that allows authorized users to manage station infrastructure. The main screens and functionality include:

Station Registration

The Station Owner can enter station information including station name, address, location and other required details.

After submission, the station is displayed with a Pending status until it is reviewed by an authorized administrator.

Station Management

Station Owners can view their registered stations and access station-specific management functionality.

The station management interface provides access to:

- Station details
- Charger management
- Operating hours

- Maintenance windows
- Station status
- Utilization information

Station Approval

The administrative interface allows authorized administrators to review pending stations and perform approval-related actions.

The workflow prevents an unapproved station from being treated as an active charging facility.

Charger Management

The charger management interface allows Station Owners to:

- Add chargers
- View chargers
- Edit charger specifications
- Configure connector types
- Configure maximum output
- Configure pricing
- Update charger status

Operating-Hours Management

The operating-hours interface allows owners to configure the weekly schedule of the station.

Users can specify the opening and closing time for each day.

Maintenance Management

The maintenance interface allows owners to create and manage charger maintenance windows.

This helps prevent drivers from receiving misleading availability information while a charger is undergoing maintenance.

Frontend Technologies

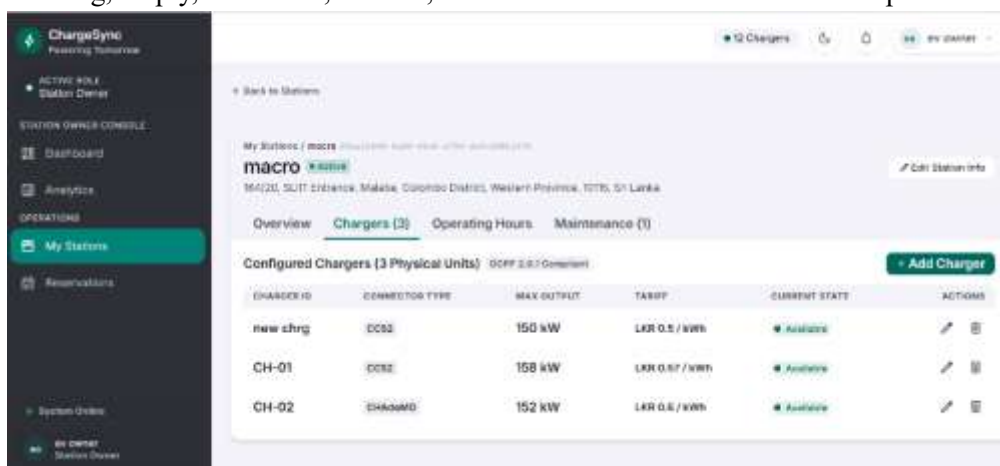
The implementation uses React functional components and hooks with API integration through the shared backend.

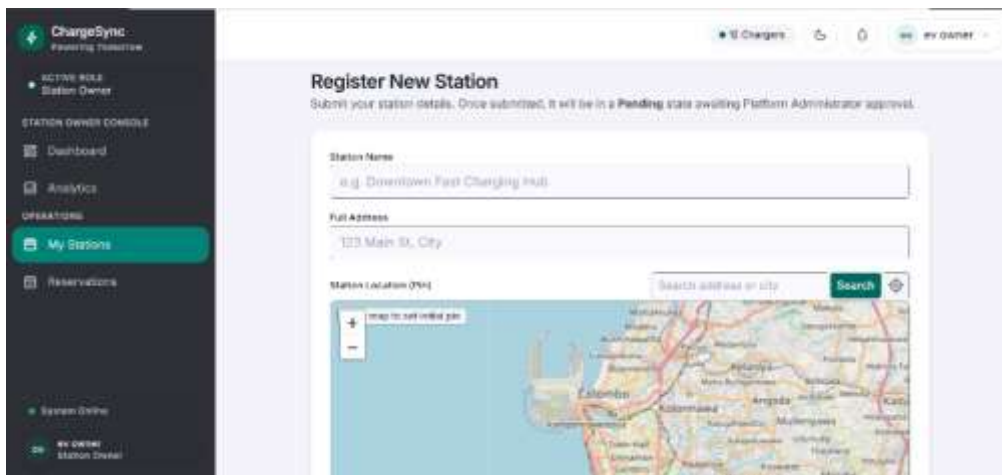
Relevant frontend files include components such as:

- ApprovalsPage.jsx
- StationDetailConsole.jsx
- ChargersTab.jsx
- MaintenanceTab.jsx
- RegisterStationPage.jsx
- useStations.js
- stations.js

Protected routes and role-based navigation are used to ensure that users only see functionality permitted for their role.

Loading, empty, validation, success, and error states are also handled to improve usability.





2.5 Flutter Work

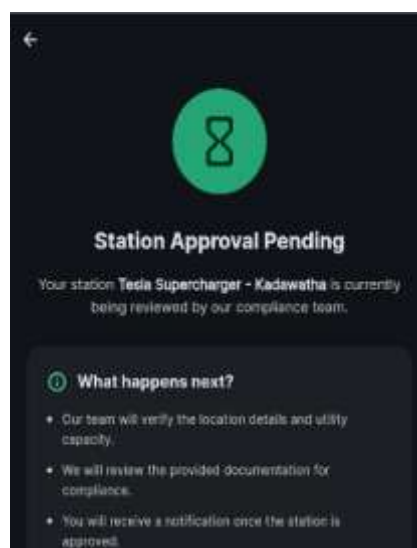
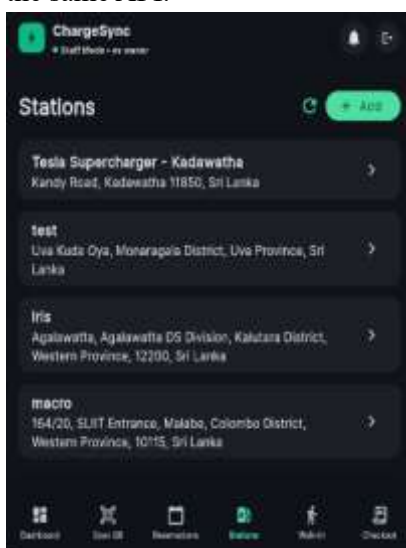
I contributed to the Flutter mobile application by integrating station-related functionality required for the cross-platform ChargeSync workflow. The Flutter application communicates with the same ASP.NET Core API and PostgreSQL database used by the React web application. The mobile interface allows relevant station information to be consumed by mobile users and supports operational station workflows where applicable.

The main contribution includes:

- Station information retrieval
- Charger availability display
- Station search/discovery integration
- Connector and charger information
- Availability status presentation
- API integration
- Loading and error handling
- Role-based access where required
- Integration with reservation-related workflows

The Flutter application does not maintain a separate station database. Instead, it communicates with the shared backend API. This ensures that station and charger information remains consistent across the React web application and Flutter mobile application.

For example, when a Station Owner updates a charger status through the web application, the updated information is persisted in PostgreSQL and can subsequently be retrieved by the Flutter application through the same API.





2.6 Agentic AI Work

I contributed to the Station Analysis Agent within the ChargeSync Agentic AI workflow. The agent evaluates live charger status, pricing history, and station utilization to generate candidate station scores for the charging recommendation process.

The Station Analysis Agent receives the following structured input:

```
{
  stationId,
  timeWindow,
  requestContext
}
```

Using this information, the agent analyzes the selected station and produces a structured result containing:

```
{
  availabilityScore,
  priceCompetitiveness,
  utilizationInsight,
  flags[]
}
```

The `availabilityScore` represents the expected availability of chargers during the requested time window. The `priceCompetitiveness` value evaluates the station's pricing against relevant historical pricing information. The `utilizationInsight` summarizes station usage patterns, while the `flags[]` collection identifies important conditions such as maintenance activity, limited availability, unusual pricing, or high utilization.

My contribution focused on ensuring that station-related information is supplied to the agent in a structured format and that the returned analysis can be used by the recommendation workflow. The agent's output supports comparison between candidate stations rather than making an isolated recommendation without considering operational data.

The station analysis also follows the platform's deterministic business rules. For example, a station should not receive a misleading availability assessment if its chargers are under maintenance,

outside operating hours, offline, or already reserved. These conditions are represented through the availability score and relevant flags.

The Agentic AI service is accessed through the ASP.NET Core backend rather than directly from the React or Flutter clients. This preserves the platform's security boundary and allows the backend to validate station data and AI results before they are used by the rest of the system.

I contributed to the Agentic AI functionality of ChargeSync by supporting the AI workflow related to charging-station analysis and station availability.

The ChargeSync Agentic AI subsystem uses multiple specialized agents coordinated through a structured workflow. My component provides station and charger information required by the AI system to evaluate candidate charging locations.

The main station-related AI workflow can use information such as:

- Station status
- Charger availability
- Connector type
- Charger maximum output
- Pricing
- Operating hours
- Maintenance status
- Station utilization
- Other station-level operational information

The **Station Analysis Agent** is responsible for evaluating candidate stations using station-related operational data.

The AI workflow can then provide structured station analysis to the Charging Recommendation and Planning Agent.

My contribution focused on ensuring that station-related information provided to the AI workflow is valid, structured, and consistent with the actual database state.

I also considered deterministic business rules when integrating station information with AI-generated recommendations. For example, an AI recommendation should not treat a charger as available if the charger is under maintenance, the station is closed, or the charger has an existing reservation.

Therefore, AI-generated recommendations are supported by system-level validation rather than being accepted blindly.

The AI subsystem also follows the project's security requirement where the Python/FastAPI AI service is accessed only through the ASP.NET Core backend and is not directly exposed to React or Flutter clients.



2.7 Testing Work

I performed testing for the Function 2 implementation across the backend, database, React interface, API integration, and business workflows.

The main testing areas were:

Backend Testing

Backend tests were used to verify:

- Station creation
- Station retrieval
- Station update
- Station deletion validation
- Charger creation
- Charger update
- Charger status changes
- Operating-hours validation
- Maintenance-window validation
- Availability calculation
- Authorization rules
- Invalid input handling

Business Rule Testing

The most important business-specific testing focused on charger availability.

Test scenarios included:

1. Charger is available and station is open → charger is available.
2. Station is closed → charger is unavailable.
3. Charger is under maintenance → charger is unavailable.
4. Charger status is Occupied → charger is unavailable.
5. Charger has an active reservation → charger is unavailable.
6. Station is not active → station cannot be treated as available.
7. Unauthorized user attempts station modification → request is rejected.
8. Invalid maintenance period → validation error is returned.

API Testing

Swagger/Postman/API tests were used to verify request and response behaviour.

Database Testing

Database testing verified:

- Foreign key relationships
- Required fields
- Correct station-charger relationships
- Correct maintenance-charger relationships
- Data persistence
- Migration correctness

Frontend Testing

The React interface was tested for:

- Form validation
- Successful station creation
- Charger management
- Status updates

- Operating-hours configuration
- Maintenance-window configuration
- Error messages
- Loading states
- Protected functionality

Integration Testing

Integration testing verified that:

React/Flutter → ASP.NET Core API → PostgreSQL

works correctly for station and charger operations.

The tests also verified that changes made through one client persisted in the shared database and can be retrieved by another client.

The testing evidence is included through test reports, screenshots, API responses, and GitHub Actions CI results.

Backend

```
PS D:\Projects\ChargeSync-Platform\backend\tests\IntegrationTests> dotnet test --filter "FullyQualifiedName=StationEndpointsTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
IntegrationTests -> D:\Projects\ChargeSync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll
Test run for D:\Projects\ChargeSync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:    5, Skipped:    0, Total:    5, Duration: 798 ms - IntegrationTests.dll (net8.0)
PS D:\Projects\ChargeSync-Platform\backend\tests\IntegrationTests>
```

```
* PS D:\Projects\ChargeSync-Platform\backend\tests\UnitTests> dotnet test --filter "FullyQualifiedName=StationServiceTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
Infrastructure -> D:\Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
AgentClient -> D:\Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Api -> D:\Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
UnitTests -> D:\Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll
Test run for D:\Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:    9, Skipped:    0, Total:    9, Duration: 415 ms - UnitTests.dll (net8.0)
MSBUILD : warning : Killing process with pid = 15244.
* PS D:\Projects\ChargeSync-Platform\backend\tests\UnitTests>
```

React

```
PS D:\Projects\ChargeSync-Platform\web-react> npx test -- tests/unit
...
/* assert on the output */

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

...
✓ tests/unit/components/ReservationDetailsModal.test.jsx (6 tests) 1864ms
  ✓ ReservationDetailsModal (6)
    ✓ allows editing reservation time 150ms
✓ tests/unit/pages/ReservationsPage.test.jsx (8 tests) 1374ms
  ✓ ReservationsPage (8)
    ✓ renders reservations and allows search 560ms
    ✓ opens and closes Details Modal 364ms
✓ tests/unit/hooks/useStations.test.jsx (2 tests) 349ms
✓ tests/unit/pages/StationDetailPage.test.jsx (2 tests) 48ms

Test Files 12 passed (12)
Tests 44 passed (44)
Start at 15:25:11
Duration 9.07s (environment 48%, import 21%, tests 37%, setup 7%, transform 3%, worker 1%)

Environment
  JSDOM was created 12 times (34.39s total, 49% of tracked time)
  create it once per server with pool: 'vmThreads' (keeps per-file isolation) or isolate: false (shares it across files)
  learn more: https://vitest.dev/guide/optimizing-performance#test-environments
```

```

✓ tests/integration/Stations.integration.test.jsx (1 test) 660ms
✓ registers a new station successfully 656ms
✓ tests/integration/Chargers.integration.test.jsx (2 tests) 689ms
✓ opens add charger form and submits 613ms

Test Files 2 passed (2)
Tests 3 passed (3)
Start at 15:25:49
Duration 2.61s (environment 34%, tests 25%, import 8%, setup 8%, transform 5%, worker 1%)
PS D:\Projects\ChargeSync-Platform\web-react>

```

Flutter

```

PS D:\Projects\ChargeSync-Platform\mobile-flutter> flutter test test\features\stations\unit
00:02 +4: All tests passed!
PS D:\Projects\ChargeSync-Platform\mobile-flutter>

```

```

PS D:\Projects\ChargeSync-Platform\mobile-flutter> flutter test test\features\stations\integration
ClientException: Request to https://tile.openstreetmap.org/15/23653/15750.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23653/15750.png
ClientException: Request to https://tile.openstreetmap.org/15/23651/15751.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23651/15751.png
ClientException: Request to https://tile.openstreetmap.org/15/23654/15751.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23654/15751.png
ClientException: Request to https://tile.openstreetmap.org/15/23651/15752.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23651/15752.png
ClientException: Request to https://tile.openstreetmap.org/15/23654/15752.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23654/15752.png
ClientException: Request to https://tile.openstreetmap.org/15/23652/15753.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23652/15753.png
ClientException: Request to https://tile.openstreetmap.org/15/23653/15753.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23653/15753.png
ClientException: Request to https://tile.openstreetmap.org/15/23651/15750.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23651/15750.png
ClientException: Request to https://tile.openstreetmap.org/15/23654/15750.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23654/15750.png
ClientException: Request to https://tile.openstreetmap.org/15/23651/15753.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23651/15753.png
ClientException: Request to https://tile.openstreetmap.org/15/23654/15753.png failed with status 400: .. uri=https://tile.openstreet
map.org/15/23654/15753.png
00:05 +7: All tests passed!
PS D:\Projects\ChargeSync-Platform\mobile-flutter>

```

AI Agent

```

(.venv) PS D:\Projects\ChargeSync-Platform\agent-ai> python -m pytest tests\test_station_analysis_agent.py
>>
===== Test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\Projects\ChargeSync-Platform\agent-ai
plugins: anyio-4.14.2, langsmith-0.11.8
collected 4 items

tests\test_station_analysis_agent.py ..... [100%]

===== 4 passed in 0.90s =====
(.venv) PS D:\Projects\ChargeSync-Platform\agent-ai>

```

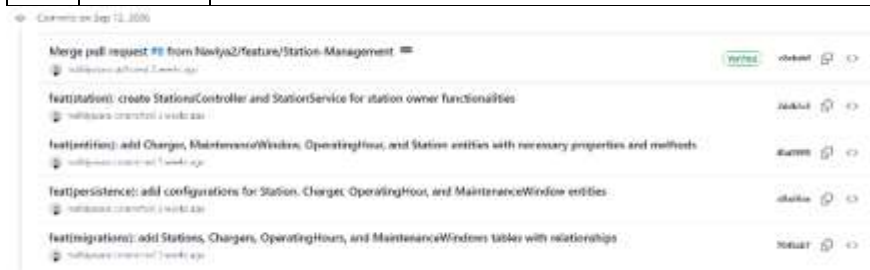
Key Commit, Pull-Request and Test Evidence

Key Git Commits

The following commits represent significant milestones in my implementation of Station, Charger & Operating-Hours Management.

No.	Commit	Contribution
1	7045ab7	add Stations, Chargers, OperatingHours, and MaintenanceWindows tables with relationships
2	d8a26ca	add configurations for Station, Charger, OperatingHour, and MaintenanceWindow entities
3	2ddbfe3	create StationsController and StationService for station owner functionalities

4	6a0483c	add update station and maintenance window functionality, remove title from maintenance window
5	d2e7d54	add station management and approval features with routing and components
6	f212af2	add Register Station page and routing for station management
7	9c6d182	implement station management features including listing, adding, and editing stations
8	7469f90	add Charger model and AddChargerScreen for managing chargers
9	e5d6c45	add Cloudinary configuration variables to .env.example
10	88942ce	implement charger update and delete functions
11	f4f1cfa	add ImageUploader and MapLocationPicker components
12	c5e3c9f	implement StationAnalysisAgent for evaluating charger availability and pricing
13	22cb6cb	add integration and unit tests for station management components and pages

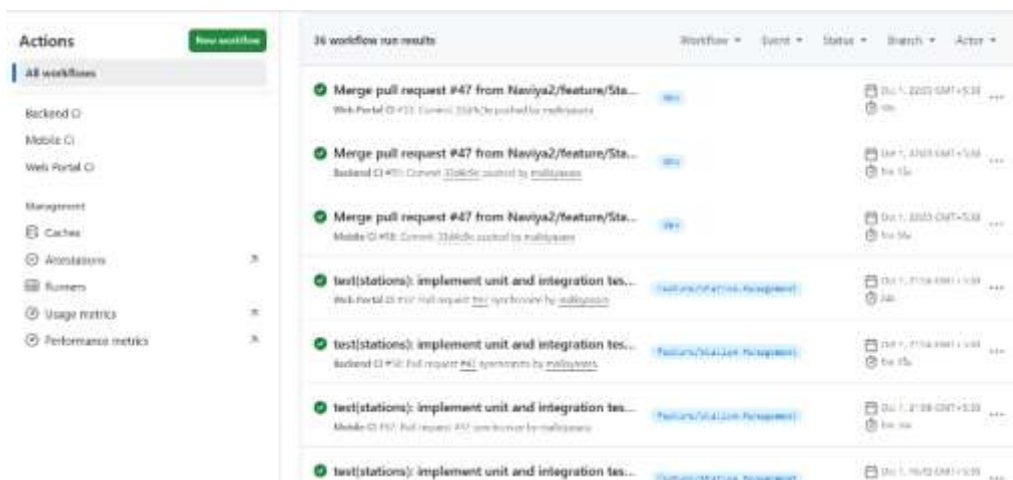
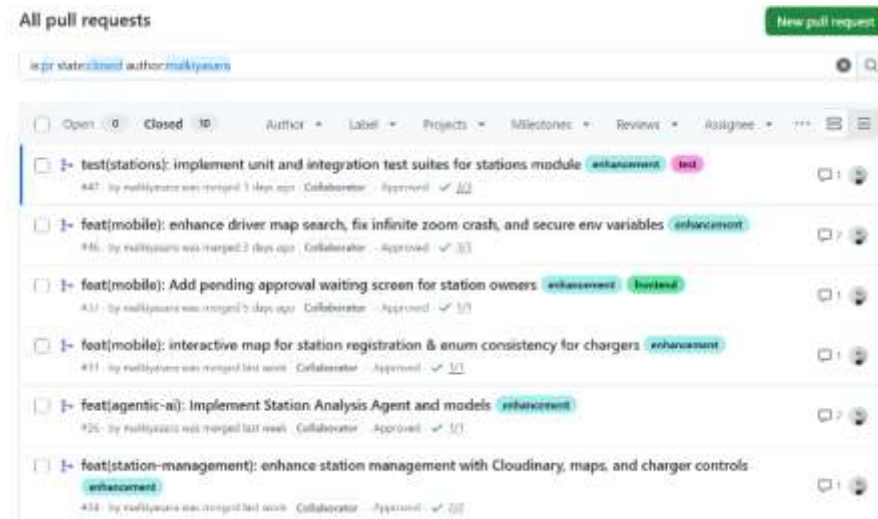


These commits demonstrate the incremental development of my component from database and backend implementation to frontend integration and testing.

3.2 Pull Request Evidence

I used GitHub pull requests to integrate my Function 2 implementation into the shared project repository.

PR	Purpose
PR #8	implement station, charger, operating hours, and maintenance management backend
PR #13	Restructure MaintenanceWindow entity and add Station & Maintenance update endpoints
PR #15	implement station management for staff dashboard
PR #24	enhance station management with Cloudinary, maps, and charger controls
PR #31	interactive map for station registration & enum consistency for chargers
PR #46	enhance driver map search, fix infinite zoom crash, and secure env variables
PR #47	implement unit and integration test suites for stations module



3.3 Test Evidence

The following evidence demonstrates the testing performed for my component.

Test ID	Test Case	Expected Result	Actual Result	Status
TC-F2-001	Register a new station with valid information	Station is created successfully and assigned Pending status.	Pass	Pass
TC-F2-002	Register station with missing required fields	System rejects the request and displays appropriate validation messages.	Pass	Pass
TC-F2-003	Register station with invalid coordinates	System rejects invalid coordinates and displays a validation error.	Pass	Pass
TC-F2-004	Retrieve registered stations	The user's authorized stations are returned correctly.	Pass	Pass
TC-F2-005	Add a charger to an owned station	Charger is created and linked to the selected station.	Pass	Pass
TC-F2-006	Update charger specifications	Charger information is updated successfully and persisted in the database.	Pass	Pass
TC-F2-007	Update charger operational status	Charger status is updated and the new status is returned by the API.	Pass	Pass

TC-F2-008	Remove a charger without active booking	Charger is removed successfully.	Pass	Pass
TC-F2-009	Prevent unauthorized charger modification	API rejects the request with an authorization error.	Pass	Pass
TC-F2-010	Configure weekly operating hours	Operating hours are saved for the selected day.	Pass	Pass
TC-F2-011	Validate invalid operating hours	System rejects the invalid schedule and displays an appropriate validation message.	Pass	Pass
TC-F2-012	Retrieve weekly operating hours	Correct weekly operating hours are returned.	Pass	Pass
TC-F2-013	Create a maintenance window	Maintenance window is created successfully and linked to the charger.	Pass	Pass
TC-F2-014	Reject invalid maintenance window	System rejects the maintenance window and displays a validation error.	Pass	Pass
TC-F2-015	Verify charger during maintenance	Charger is not returned as available during the maintenance period.	Pass	Pass
TC-F2-023	Retrieve station utilization analytics	System returns the correct utilization information for the station.	Pass	Pass

4. Challenges and Learning

4.1 Challenges

During the development of Function 2, I faced several technical and integration challenges.

Challenge 1 - Implementing the station approval workflow

A station cannot immediately become available after registration because the system requires administrator approval. Therefore, the station lifecycle needed to be handled correctly between the Station Owner and Platform Administrator.

Challenge 2 - Calculating real availability

Initially, charger availability could appear to be a simple status such as Available or Occupied. However, the actual business requirement requires multiple conditions to be considered, including station operating hours, maintenance windows, reservations and charger status.

Challenge 3 - Integration with other components

The station component is connected to the reservation component because reservation availability depends on charger and station information. Changes made to station or charger data therefore need to remain compatible with other team members' functionality.

4.2 Solutions

I addressed these challenges using the following approaches.

For the station approval workflow, I implemented explicit station status management so that newly registered stations remain pending until administrator approval.

For availability calculation, I separated charger operational status from actual availability. The availability logic checks the station status, operating hours, maintenance windows and reservation state before returning the final availability result.

For integration, I used the shared ASP.NET Core API and PostgreSQL database rather than creating separate data sources for my component.

4.3 Learning

Through the development of Function 2, I gained practical experience in designing and implementing a full-stack business component.

The main learning outcomes were:

- Designing RESTful APIs using ASP.NET Core.
- Applying service-layer business logic.
- Using Entity Framework Core with PostgreSQL.
- Designing relational database relationships.
- Implementing CRUD operations.
- Implementing role-based authorization.
- Developing reusable React components.
- Integrating frontend applications with REST APIs.
- Handling asynchronous API operations and errors.
- Implementing business-specific availability rules.
- Understanding cross-component integration.
- Working with Git branches, commits and pull requests.
- Writing and executing automated tests.
- Understanding how Agentic AI can consume structured business data.
- Understanding the importance of deterministic validation around AI-generated results.
- Understanding how the same API and database can support both web and mobile clients.

Most importantly, I learned that implementing a full-stack feature is not only about creating screens and APIs. The database model, business rules, security, testing, integration, AI workflow, and user experience must all work together as one system.

5. Individual AI Usage Log

During the development of ChargeSync, I used AI-assisted development tools as supporting resources for selected development and documentation tasks. AI-generated suggestions were treated as assistance rather than automatically accepted solutions.

Date	AI Tool / Model	Task / Section	What the AI Produced	What I Changed or Rejected	How I Verified the Result
[10/09/2026]	ChatGPT – GPT-5.6	ASP.NET Core Backend	Provided guidance on implementing REST API endpoints, service-layer structure, DTOs and validation.	I adapted the suggested structure to the existing ChargeSync project architecture and modified the code according to the project's entities and business rules.	Built and ran the backend, tested the endpoints through Swagger, checked HTTP status codes and verified the database changes.
[12/09/2026]	ChatGPT – GPT-5.6	PostgreSQL / Entity Framework Core	Suggested entity relationships, database constraints, migration approaches and seed-data structures.	I selected and modified the suggestions based on the actual Station, Charger, OperatingHour and MaintenanceWindow requirements.	Ran EF Core migrations, checked the PostgreSQL schema, tested relationships and verified CRUD operations.
[20/09/2026]	ChatGPT – GPT-5.6	React Web Application	Provided guidance for React components, hooks, routing, API	I adapted the generated examples to the existing React project and modified	Ran the React application, tested protected routes, forms, API

			integration and UI implementation.	component structure, API calls and UI behaviour.	operations, loading/error states and unit tests.
[25/09/2026]	ChatGPT – GPT-5.6	Flutter Mobile Application	Provided explanations and implementation guidance for Flutter widgets, navigation, API communication and state management.	I modified the suggested implementation to match the ChargeSync mobile application and its existing architecture.	Ran the Flutter application on the Android emulator/device and tested navigation, forms, API communication and validation.

6. Individual AI Reflection

6.1 AI Tools and Development Stages

During the development of my Function 2 component, I used AI-assisted tools, mainly ChatGPT, at different stages of development.

During the planning stage, AI assistance helped me break down the station management requirements into smaller technical tasks such as station registration, approval, charger management, operating-hours management and maintenance-window management.

During the implementation stage, I used AI assistance to understand unfamiliar ASP.NET Core, Entity Framework Core, React and testing concepts. AI was particularly useful when I encountered errors or needed an explanation of how different layers of the application should interact.

During the testing stage, AI assistance helped me identify possible test scenarios and edge cases for station and charger availability.

During the documentation stage, AI assistance helped organize technical information into a clear structure. However, I reviewed the generated content and changed it to reflect my actual implementation and contribution.

6.2 What AI Did Well

AI assistance was particularly useful for explaining unfamiliar technical concepts in a beginner-friendly manner.

It helped me understand how ASP.NET Core controllers, services, DTOs and Entity Framework Core work together. It also helped me identify possible edge cases in the station availability logic.

AI suggestions were useful for generating initial approaches to problems, explaining error messages, suggesting testing scenarios, and improving the structure of technical documentation.

Another benefit was that AI allowed me to compare multiple possible implementation approaches quickly. This helped me understand the reasoning behind technical decisions rather than relying only on trial and error.

However, the final implementation was based on the actual ChargeSync project structure and requirements rather than blindly copying AI-generated output.

6.3 What AI Got Wrong and Changes Made

AI-generated suggestions were not always directly compatible with the ChargeSync project.

Some suggestions assumed different file structures, property names, API endpoints or implementation patterns from those already used in the project. Therefore, I had to adapt generated suggestions to match the existing codebase.

In some cases, AI suggestions also provided simplified availability logic that did not fully consider all ChargeSync business rules. For example, checking only the charger status is not sufficient because actual availability also depends on operating hours, maintenance windows and reservations.

I therefore modified the suggested logic and verified it against the project's requirements and actual system behaviour.

I also rejected suggestions when they conflicted with the assignment constraints, project architecture, security requirements, or existing implementation.

This demonstrated that AI-generated code should not be treated as automatically correct. Developer understanding, testing and verification are still required.

6.4 Verification and Learning

I verified AI-assisted work by comparing suggestions with the ChargeSync SRS, inspecting the existing source code, running the application, testing API endpoints, checking database results, and reviewing test outcomes.

For backend changes, I used API testing and automated tests to verify the behaviour.

For frontend changes, I manually tested the relevant React screens and verified that data was correctly retrieved from the backend.

For database-related changes, I checked the persisted records and relationships in PostgreSQL.

Through this process, I learned that AI tools are most useful as development assistants rather than replacements for engineering knowledge. The developer must understand the generated solution, identify incorrect assumptions, adapt it to the project architecture, and verify the final behaviour.

This experience also improved my ability to debug problems independently because I learned to use AI explanations as a starting point and then confirm the actual cause through code inspection and testing.

7. Signed Declaration

Student Name: Yasara R.P.M

Student ID: IT24102953

Declaration

I confirm that the technical contributions, Git evidence, testing evidence, challenges, learning outcomes and AI usage information presented in this individual report are truthful and can be supported by the project's source code, repository history, test results, and other submitted evidence. I also confirm that AI-assisted development was used responsibly and that AI-generated outputs were reviewed, modified where necessary, and verified before being incorporated into the project.

Signature

Student Name: Yasara R.P.M

Student ID: IT24102953

Signature: Mallei

Date: 05/10/2026

IT24102295 - Sandaru P.H.B.

1 Contribution Statement

My primary responsibility in ChargeSync was the post-booking workflow, covering charging sessions, invoicing and payment settlement, membership subscriptions, loyalty rewards and customer support. For the ASP.NET Core backend, I contributed to session completion, invoice generation, cash and wallet settlement, subscription management, loyalty redemption and support-ticket operations. My work included validation, access restrictions and safeguards against duplicate financial actions. I contributed to the related data models and persistence operations for sessions, invoices, subscriptions, loyalty records and support tickets.

In Flutter, I contributed to the session checkout, payment-history, membership, rewards and support interfaces. These screens communicate with the shared backend API and display loading states, validation messages and API errors. In React, I contributed to the support-management interface used by authorized staff to review tickets, reply to customers and review workflow decisions.

My Agentic AI contribution focused on the Validation and Support workflow. This included checking linked session and invoice information, presenting structured analysis, handling unavailable or invalid AI responses, and maintaining human approval controls for actions requiring authorization.

I also contributed to automated testing. With disclosed AI assistance, I added and verified 23 Flutter tests covering membership, loyalty, checkout and payment behavior. All 23 tests passed, and the complete Flutter suite passed with 62 tests. Testing identified a checkout UI issue involving the Material ancestor of the meter-reading switch, which was corrected. These tests use mocked API responses; they do not establish live PostgreSQL or complete cross-platform workflow verification

2 Owned Component and Technical Work

My component manages the driver's activities after booking, including charging-session tracking, invoice generation, payment settlement, membership subscriptions, loyalty points and reward redemption, and support tickets. It also includes the Validation and Support Agent workflow and human approval for sensitive actions.

Technical Work

- ASP.NET Core backend: Implemented session start/stop operations, energy calculations, automatic invoicing, cash and wallet settlement, membership management, loyalty redemption and support-ticket operations. Added validation, authorization checks and duplicate-action protection.
- Database: Contributed to entities, relationships and persistence for charging sessions, invoices, subscriptions, loyalty accounts, reward redemptions and support tickets.
- Flutter: Developed session checkout, payment-history, membership, rewards and support interfaces, including API integration, form validation, loading states and error handling.
- React: Contributed to the staff support interface for viewing tickets, managing conversations, reviewing AI analysis and handling authorized workflow decisions.
- Agentic AI: Contributed to the Validation and Support workflow, including session/invoice checks, structured analysis, controlled tool use, failure handling and human approval enforcement.
- Testing: With disclosed AI assistance, added and verified 23 Flutter automated tests for membership, loyalty, checkout and payments. The full Flutter suite passed with 62 tests. Corrected a checkout UI issue identified during testing.
- Documentation: Documented test cases, expected and actual results, execution commands and the limitations of tests using mocked API responses.

2.2 API Endpoints

The Session, Payment, Membership, Loyalty and Support Management component exposes REST endpoints through the ASP.NET Core Web API. React and Flutter use these endpoints to access shared application data and business operations.

All endpoints below require JWT authentication:

Authorization: Bearer <access_token>

The backend applies role permissions and ownership checks. {id}, {userId} and {ticketId} represent record identifiers.

Charging Sessions and Payments

Method	Endpoint	Purpose
POST	/api/sessions/start	Start a charging session for a reservation.
GET	/api/sessions	Retrieve accessible sessions using filters, such as status.
GET	/api/sessions/{id}	Retrieve a specific charging session.
PUT	/api/sessions/{id}/stop	Complete a session, optionally submit a physical meter reading and photo URL, and generate its invoice.
GET	/api/payments/invoices	Retrieve accessible invoices using filters.
GET	/api/payments/invoices/{userId}	Retrieve a driver's invoice history.
POST	/api/payments/invoices/{id}/settle	Record settlement using cash or wallet payment.
GET	/api/payments/history	Retrieve the authenticated driver's payment history.

Membership Management

These endpoints are restricted to users with the **Driver** role.

Method	Endpoint	Purpose
GET	/api/membership-plans	Retrieve available membership plans.
GET	/api/subscriptions	Retrieve the driver's subscriptions.
POST	/api/subscriptions	Subscribe to a selected membership plan.
PUT	/api/subscriptions/{id}/change	Change an existing subscription's plan.
DELETE	/api/subscriptions/{id}	Cancel a subscription according to cancellation rules.

Loyalty and Rewards

Method	Endpoint	Purpose
GET	/api/loyalty/me	Retrieve the authenticated driver's loyalty balance, tier and related information.
GET	/api/loyalty/{userId}	Retrieve a user's loyalty information; accessible to that driver or an administrator.
GET	/api/loyalty/history	Retrieve the driver's points history.
GET	/api/loyalty/rewards	Retrieve available rewards.
POST	/api/loyalty/redeem	Request reward redemption using a reward ID and request ID.
GET	/api/loyalty/redemptions	Retrieve the driver's redemption requests, or all requests for an administrator.
POST	/api/loyalty/redemptions/{id}/review	Allow an administrator to approve or reject a redemption request.

The backend also supports /api/loyalty/balance/{userId} as an alternative route for retrieving loyalty information.

Support Management

Method	Endpoint	Purpose
POST	/api/support-tickets	Allow a driver to create a support ticket.
GET	/api/support-tickets	Retrieve accessible tickets, optionally filtered by status.
GET	/api/support-tickets/{id}	Retrieve ticket details and conversation.
POST	/api/support-tickets/{id}/messages	Add a message to an accessible ticket.
PUT	/api/support-tickets/{id}/status	Allow authorized support staff to update ticket status.
PUT	/api/support-tickets/{id}/assignee	Allow authorized support staff to assign a ticket.
DELETE	/api/support-tickets/{id}	Allow a driver to withdraw an eligible ticket.
POST	/api/support-tickets/{id}/analysis	Allow authorized support staff to request AI-assisted ticket analysis.
GET	/api/support-tickets/invoices/{id}/validation	Validate linked invoice and session information.
POST	/api/support-tickets/{id}/refund-review	Review a refund request through the authorized staff operation.

Agentic AI Workflow Management

Method	Endpoint	Purpose
POST	/api/agent-workflows/support-ticket/{ticketId}	Start or request the support workflow through authorized staff access.
GET	/api/agent-workflows/support-ticket/{ticketId}	Retrieve the workflow associated with a ticket.
GET	/api/agent-workflows/{id}	Retrieve accessible workflow details.
POST	/api/agent-workflows/{id}/approve	Submit an authorized approval decision.
POST	/api/agent-workflows/{id}/reject	Submit an authorized rejection decision.
POST	/api/agent-workflows/{id}/revise	Request revision and reanalysis of the workflow.

Support analysis and workflow decisions are restricted to **SupportManager/Admin**, with additional business-rule checks where required. Clients communicate with ASP.NET Core; the backend calls the internal Python AI service.

Invoice generation happens when a session is stopped; there is no separate invoice-creation endpoint.

Cancelling subscriptions and withdrawing tickets are business operations rather than permanent record deletion.

2 3 Business Logic

My component manages charging sessions, payments, memberships, loyalty rewards and support tickets.

- **Sessions:** Calculate energy using charger power $\times$ duration. Use a supplied physical reading for billing and flag energy differences above 15%.
- **Payments:** Generate one invoice per session, apply discounts and deposits, and support authorized wallet/cash settlement while preventing duplicate payments.
- **Memberships:** Validate wallet funds, apply proportional credit when changing plans, and retain cancelled-plan benefits until expiry.
- **Loyalty:** Award points once per paid invoice, calculate tiers and validate reward redemption. Approval-required rewards remain pending until reviewed.
- **Support and AI:** Restrict ticket access, validate refund requests and use AI for analysis. Authorized humans control financial decisions.

3 React Web Contribution

The React interface provides the review and administration side of my component. Authorized staff can manage support tickets, review AI findings and make approval decisions.

Invoice and payment views- Display charges, deposits and settlement information.

The screenshot shows a 'Reservation Details' form with the following sections:

- CUSTOMER:** Walk-In Customer
- STATUS:** Completed
- SCHEDULE:** Sep 27, 2026, 23:26 - 23:31
- STATION & CHARGER:** Sunfest EV charge, Charger: C4-01
- WALK-IN CUSTOMER DETAILS:** Name: N/A, Vehicle No.: N/A, Battery Capacity: N/A
- FINAL SESSION INVOICE:** Final Cost: Rs. 5.00, Energy: 10.00 kWh, Payment Method: Cash
- METER PHOTO:** Meter Reading
- STATUS HISTORY:** Checked in (Sep 27, 2026 23:26 - User: 33084646)
- Buttons:** Delete, Close, Edit Time

Support administration- Review tickets and update permitted states.

The screenshot shows the 'Ticket queue' interface with a list of tickets and a detailed view of a selected ticket.

Ticket Queue:

- Search: [Empty]
- Buttons: 0 open, Refresh, AS

Ticket List:

- #090436:** Station staff problem. He got more cash from me in my bill. Status: Received.
- #545780:** Technical issue. Status: In Progress.
- #1030788:** Charging information does not refresh. My charging information stays unchanged after refreshing the app. Status: Opened.
- #402440:** Charging information does not refresh. My charging information stays unchanged after refreshing the app. Status: Opened.
- #000001:** test ticket. Status: Received.

Ticket Detail View (Selected Ticket #090436):

- Station staff problem**
- Opened 10/4/2024, 10:47:55 AM by Sanjiv P H B
- Sanjiv P H B: 10/3/2024, 10:20:38 AM. He got more cash from me in my bill.
- Platform Administrator: 10/4/2024, 9:20:02 AM. Ticket assigned to a support manager.
- Platform Administrator: 10/4/2024, 11:00:10 PM. Hello, Thank you for reaching out. I'm sorry to hear about the issue you experienced at the station. Unfortunately, we don't have any invoice or session data linked to this claim, so we can't verify the details at this time. To help us investigate further, could you please provide: 1. The date and time of the visit 2. The station or location ID 3. Any receipt or transaction reference you received 4. The amount you were charged and the amount you believe was correct. Once we have this information, we'll be able to look into the matter and determine the appropriate next steps. Thank you for your patience and understanding.

Agent workflow: Loading analysis status...

Workflow: Status: Received. Assigned to you: Sanjiv P H B.

Driver: Sanjiv P H B. New Reservation: A driver just booked a new slot.

Meter discrepancy review- Present discrepancy findings and review reasons.

The ticket changed after analysis.
Request revision before approving.

Validation summary

- Info: No linked invoice; billing checks were not performed.

Review or revision note

Request Revision

► Execution history

▼ Additional ticket analysis

Validation & Support Agent

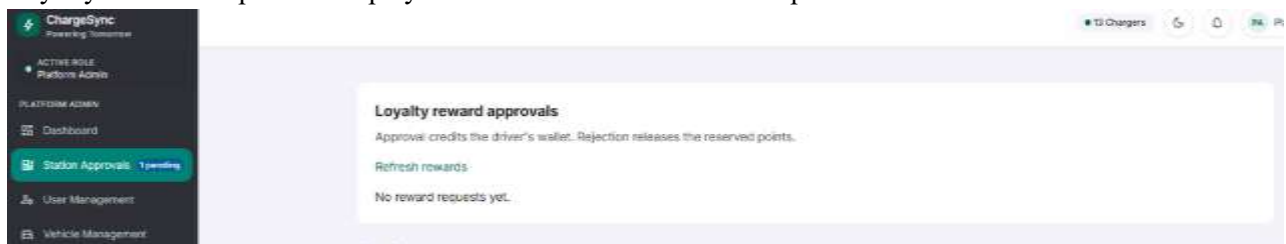
Analyze ticket

New Reservation

A driver just booked a

New Reservation

Loyalty and subscriptions- Display relevant rewards or membership information.



Approval workflow-Approve, reject or request revision of a pending action

4 Flutter Mobile Contribution

Session checkout- Complete charging sessions and generate invoices.



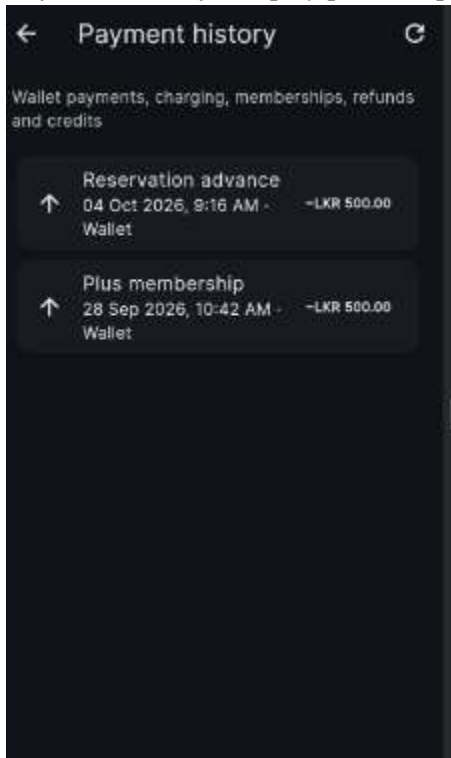
Payment settlement- Record wallet or cash payment.

After stopping the session, the same screen displays the invoice:

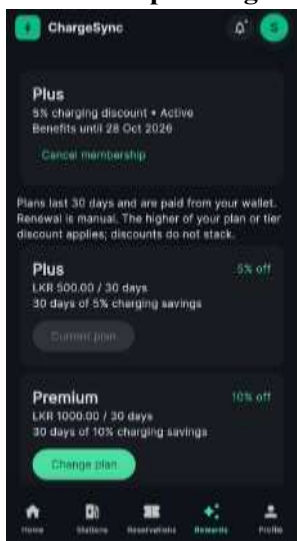
1. Under **Record payment**, select **Wallet** or **Cash**.
2. Tap **Confirm LKR ... payment**.

3. A successful payment displays **Payment recorded.**

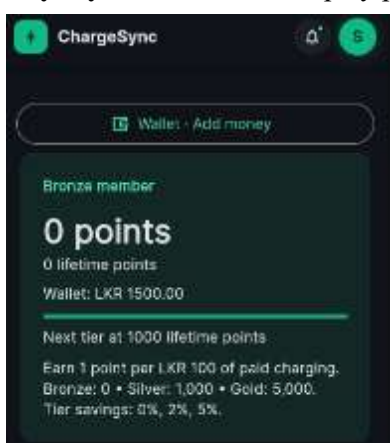
Payment history- Display previous payments and invoice details.



Membership management- Subscribe, change plans and cancel membership.



Loyalty and rewards- Display points and tiers, and support reward redemption



Support tickets- Create and track tickets and communicate with support staff.

5 Agentic AI Contribution

Agent and responsibility

My contribution focused on the **Validation and Support Agent**, which assists staff with support-ticket analysis, linked session and invoice checks, and draft replies. Financial actions remain controlled by the backend and authorized humans.

AI workflow

ASP.NET Core loads the authorized ticket and related records and performs deterministic validation. A Python **LangGraph** workflow then follows three stages: planning, delegation to the ValidationSupportAgent, and structured reply generation. React displays the findings and review controls.

Tools used

The agent uses six allow-listed, read-only tools:

- `get_support_ticket`
- `get_charging_session`
- `get_invoice`
- `get_payment`
- `calculate_meter_discrepancy`
- `get_loyalty_account`

These tools access the authorized backend snapshot. They cannot directly query PostgreSQL, issue payments or modify records. The meter tool retrieves the backend's calculation.

Structured input/output

Input includes the workflow ID, revision, objective, ticket, linked records, validation findings and approval requirement. Output contains the plan, completed steps, tool results and a suggestion with **category, priority, explanation and draft reply**. Pydantic schemas restrict fields and data types.

Validation

Backend checks cover ownership, linked records, session completion, invoice arithmetic and meter discrepancies. Returned output is checked against the current workflow ID, revision and permitted structure. Ticket text and revision notes are treated as untrusted data.

Human approval

Authorized staff review pending actions through React. Meter discrepancies requiring approval are restricted to administrators. The current refund-workflow threshold is configurable and defaults to **LKR 15**; even below that threshold, completed analysis does not execute a refund. High-value reward approval is enforced separately by the loyalty backend.

Workflow state

ASP.NET Core persists input, output, revision, attempts, decisions and audit history. States include Running, PendingApproval, Completed, Rejected, RevisionRequested and Failed. Failed analysis has bounded retries, and changed records require reanalysis before approval.

6 Testing Evidence

My test record should demonstrate the Student 4 business rules, API access, PostgreSQL operations, React and Flutter behaviour, and my agent contribution. Suggested scenarios below are test specifications. Outcomes remain unrecorded until the tests are executed; they are not passing results [1, Sections 12 and 18].

6 1 Session and billing scenarios

Test and requirement	Scenario	Expected result	Actual and status
S01	Start a valid charging session	Session becomes active	Pass
S02	Stop a 20 kW session after 30 minutes	Calculated energy is 10 kWh	Pass
S03	Stop the same session again	Rejected; no second invoice	Pass
P01	Complete a session	Invoice is automatically generated	Pass
P02	Calculate invoice with a reservation deposit	Deposit is deducted correctly	Pass
P03	Settle an invoice using a funded wallet	Correct amount is deducted once	Pass
P04	Settle with insufficient wallet funds	Payment fails without a partial deduction	Pass
P05	Driver views another driver's invoices	Access is denied	Pass

6 2 Loyalty Support and Integration Tests

Test and requirement	Scenario	Expected result	Actual and status
M01	Subscribe with sufficient wallet funds	Subscription activates and fee is deducted	Pass
M02	Subscribe with insufficient funds	No subscription or deduction occurs	Pass
M03	Change or cancel a subscription	Correct plan, credit and cancellation rules apply	Pass
L01	Pay a qualifying invoice twice	Loyalty points are awarded only once	Pass
L02	Reach a loyalty tier boundary	Correct tier is assigned	Pass
L03	Redeem a reward with enough points	Correct points are deducted; reward is issued once	Pass
L04	Redeem without enough points	Request is rejected	Pass
L05	Redeem an approval-required reward	Reward waits for authorized approval	Pass
T01	Create, view, reply to and withdraw an eligible ticket	Each operation works and is saved	Pass
A01	Process a refund requiring approval	No refund happens before authorized approval	Pass
A02	Driver tries to approve a refund	Rejected; balances remain unchanged	Pass
A03	AI service fails or returns invalid output	Safe failure/retry; no unauthorized money movement	Pass

6 3 Execution and shared evidence

Unit Test(backend)

```

Application is shutting down...
ices\ReservationService.cs(262,31): warning CS8602: Dereference of a possibly null reference. [6:\
projects\charge-sync-platform\charge-sync-platform\backend\src\Application\Application.csproj]
ices\ReservationService.cs(297,15): warning CS8602: Dereference of a possibly null reference. [6:\
projects\charge-sync-platform\charge-sync-platform\backend\src\Application\Application.csproj]
ices\ReservationService.cs(301,6): warning CS8602: Dereference of a possibly null reference. [6:\
projects\charge-sync-platform\charge-sync-platform\backend\src\Application\Application.csproj]
ices\ReservationService.cs(312,66): warning CS8602: Dereference of a possibly null reference. [6:\
projects\charge-sync-platform\charge-sync-platform\backend\src\Application\Application.csproj]
ices\ReservationService.cs(417,35): warning CS8602: Dereference of a possibly null reference. [6:\
projects\charge-sync-platform\charge-sync-platform\backend\src\Application\Application.csproj]
Application -> 6:\projects\charge-sync-platform\charge-sync-platform\backend\src\Application\bin\De
bug\net8.0\Application.dll
AgentClient -> 6:\projects\charge-sync-platform\charge-sync-platform\backend\src\AgentClient\bin\De
bug\net8.0\AgentClient.dll
Infrastructure -> 6:\projects\charge-sync-platform\charge-sync-platform\backend\src\Infrastructure\
bin\debug\net8.0\Infrastructure.dll
Api -> 6:\projects\charge-sync-platform\charge-sync-platform\backend\src\Api\bin\Debug\net8.0\Api.d
ll
UnitTests -> 6:\projects\charge-sync-platform\charge-sync-platform\backend\tests\UnitTests\bin\Debu
g\net8.0\UnitTests.dll
Test run for 6:\projects\charge-sync-platform\charge-sync-platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll (.NETCoreApp,Version=v8.0)
Test version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 150, Skipped: 0, Total: 150, Duration: 0 s - UnitTests.dll (net8.0)

```

Integration Test (backend)

```

PS G:\projects\chargesync-platform\ChargeSync-Platform> dotnet test backend/tests/IntegrationTests
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> G:\projects\chargesync-platform\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> G:\projects\chargesync-platform\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> G:\projects\chargesync-platform\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> G:\projects\chargesync-platform\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> G:\projects\chargesync-platform\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
IntegrationTests -> G:\projects\chargesync-platform\ChargeSync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
[xUnit.net 00:00:01.70] IntegrationTests.Student4HttpWorkflowTests.RequestTravelsThroughBackendAndRealFastApi [SKIP]
Skipped IntegrationTests.Student4HttpWorkflowTests.RequestTravelsThroughBackendAndRealFastApi [1 ms]

Passed! - Failed: 0 Passed: 46 Skipped: 1 Total: 47 Duration: 10 s - IntegrationTests.dll (net8.0)

```

Python AI automated tests

```

PS G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai> .\venv\Scripts\python.exe -m pytest tests/test_support_agent.py tests/test_support_workflow.py -v
===== test session starts =====
platform win32 -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0 -- G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai
plugins: anyio-4.15.1, langsmith-0.14.1
collected 7 items

tests/test_support_agent.py::test_injected_instructions_are_untrusted_data_and_cannot_add_authority PASSED [ 14%]
tests/test_support_agent.py::test_invalid_model_output_is_rejected PASSED [ 28%]
tests/test_support_agent.py::test_api_model_failure_returns_service_unavailable PASSED [ 42%]
tests/test_support_workflow.py::test_graph_delegates_bounded_tools_and_returns_only_advice PASSED [ 57%]
tests/test_support_workflow.py::test_tool_cannot_fetch_other_customer_or_execute_an_action PASSED [ 71%]
tests/test_support_workflow.py::test_linked_missing_invoice_and_invalid_session_are_structured_failures PASSED [ 85%]
tests/test_support_workflow.py::test_internal_endpoint_auth_contract_and_safe_failure PASSED [100%]

===== warnings summary =====
venv\Lib\site-packages\pytest\cacheprovider.py:469
  G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai\venv\Lib\site-packages\pytest\cacheprovider.py:469: PytestCacheWarning: could not create cache path G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai\pytest_cache\cache\nodeids: [WinError 5] Access is denied: 'G:\projects\chargesync-platform\ChargeSync-Platform\agentic-ai\pytest_cache\cache'
    config.cache.set("cache/nodeids", sorted(self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
7 passed, 1 warning in 1.14s

```

Flutter automated tests

```

PS G:\projects\chargesync-platform\ChargeSync-Platform\mobile-flutter> flutter test test/support_workflow_test.dart test/features/payments/driver_payment_history_test.dart
00:07 +12: All tests passed!
PS G:\projects\chargesync-platform\ChargeSync-Platform\mobile-flutter> flutter test test/features/membership/membership_screen_test.dart test/features/payments/session_checkout_test.dart
00:04 +23: All tests passed!

```

React support check

```

PS G:\projects\chargesync-platform\ChargeSync-Platform\web-react> node tests/run-support-workflow.mjs
11:14:11 AM [vite] (client) Re-optimizing dependencies because vite config has changed
11:14:23 AM [vite] (client) [optimizer] scanning dependencies...
PASS: workflow rendering, roles, administrator gate, failure recovery, editable draft and ASP.NET review request contracts
11:14:28 AM [vite] (client) [optimizer] bundling dependencies...

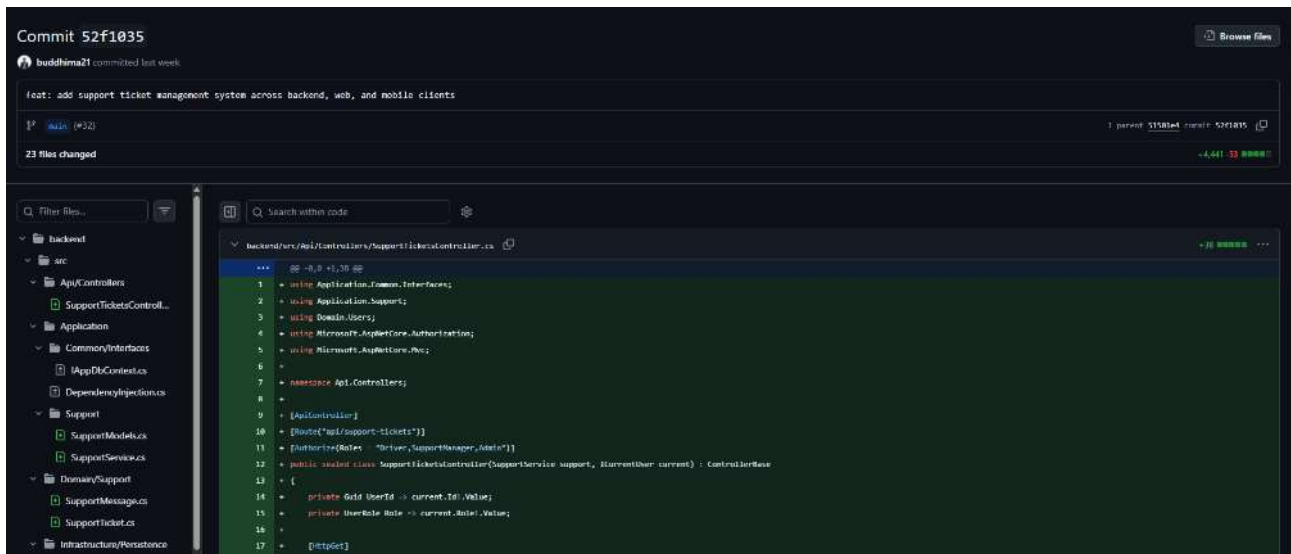
```

7 Git and Contribution Evidence

Commit 52f1035

add support ticket management system across backend, web, and mobile clients

<https://github.com/Naviya2/ChargeSync-Platform/commit/52f103580b25465fcf37e7c81b34621fd19fee3c>



Commit f41e6ca

add membership, loyalty, wallet top-up, and payment processing features

<https://github.com/Naviya2/ChargeSync-Platform/commit/f41e6ca368cea8f05bd7f84cf3e9148a8f72b022>

Commit ffb248e

Merge pull request #28 from Naviya2/feature/f4-payments

add payment invoices and charging session management

<https://github.com/Naviya2/ChargeSync-Platform/commit/ffb248eefecd02ab4ed27fc8cb839742bd1ef26c>

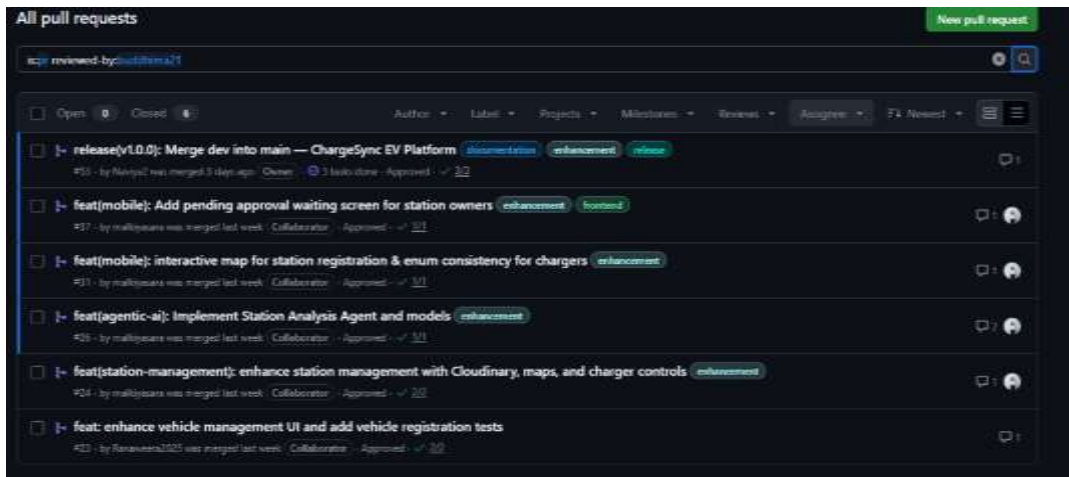
Commit 1517bc0

Merge pull request #32 from Naviya2/feature/f4-support

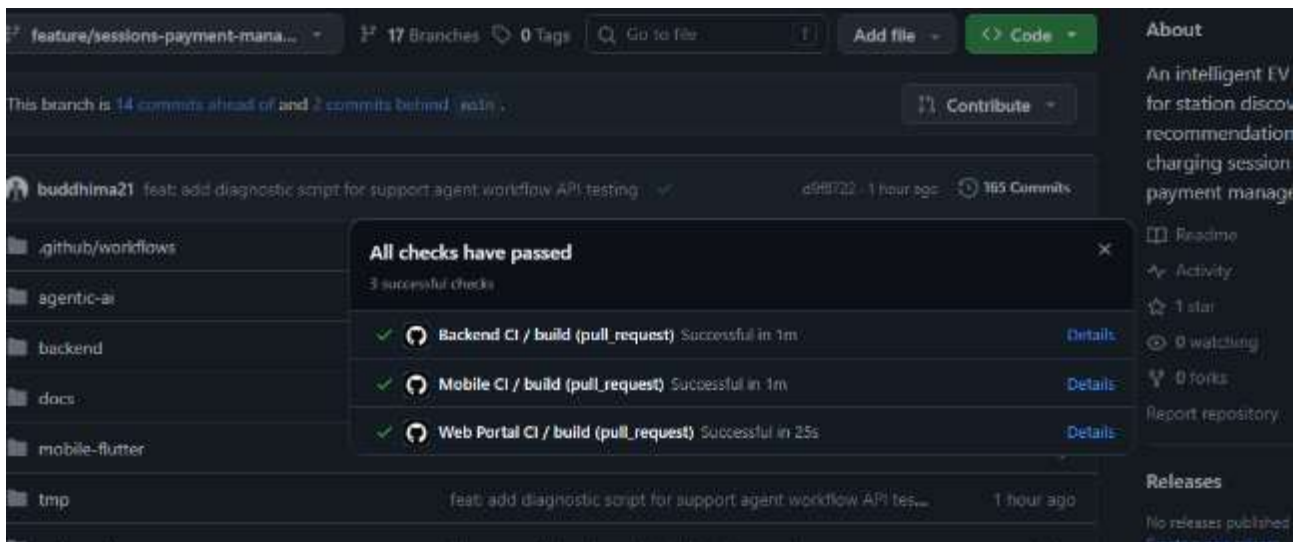
feat: add support ticket management system across backend, web, and mobile

<https://github.com/Naviya2/ChargeSync-Platform/commit/1517bc03a73d5d1f4c8a539c419b815b7a32b7c6>

Reviews



CI/CD check



My contribution is supported by the Git history on the feature/sessions-payment-management branch. The evidence table links relevant commits to backend, Flutter and testing work. Each commit provides its date, author and code changes. Pull-request reviews, issue ownership and executed test results provide additional evidence where available. AI-assisted work is disclosed in my individual AI usage log.

8 Challenges and Solutions

Challenge	Solution
Understanding how to execute tests and record evidence.	Defined expected results, ran automated tests and saved actual results with pass/fail status.
Backend tests could not build because the running API locked application files.	Identified the locking process and stopped the API before rerunning tests.
Flutter checkout tests exposed a UI assertion involving the meter-reading switch.	Added a transparent Material wrapper to the checkout card and reran the tests successfully.
Testing payment and membership failures without changing real balances.	Used mocked API responses to simulate insufficient funds, failed requests and successful operations.
Preventing repeated reward requests after an API failure.	Used request IDs and tested that retries reuse the same ID. Backend checks handle duplicate redemption requests.
Diagnosing a failed AI workflow.	Checked service availability and sent a direct workflow request, which succeeded. The React refresh issue still required further investigation.

9 Learning and Technical Understanding

Through this project, I learned how the backend, database, React portal and Flutter application work together. I understood how a charging session leads to an invoice, how discounts and deposits affect the final amount, and how payment connects to loyalty points.

I developed my understanding of API integration, input validation and role-based access. For example, hiding an approval button is not enough the backend must also check whether the user is allowed to approve the action.

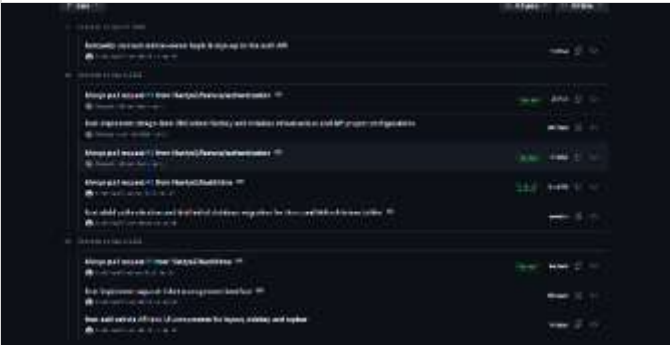
Testing helped me understand the difference between checking a small function, testing a screen and verifying a complete workflow. I learned to compare expected results with actual results and use error messages to investigate problems. I also understood that tests using fake API responses check mobile behavior but do not prove that the real database updates correctly.

10 Individual AI Usage Log

Date	AI Tool/Model	Task	How AI Was Used	Changes/Verification
4/9/2026	ChatGPT	Understanding requirements	Explained session, payment and support requirements.	Compared suggestions with the SRS and assignment guide.
10/09/2026	Claude	Backend development	Helped review or draft business logic for membership plans.	Reviewed the code and verified it through check manually.
15/09/2026	Gemini	React development	Assisted with [actual screen or API integration problem].	Checked the interface and API responses.
27/09/2026	ChatGPT	Flutter testing	Helped design or explain [actual tests].	Ran the tests and recorded their real results.
01/10/2026	Claude	Debugging	Helped investigate payment errors and meter session checkout	Applied it and checked the outcome.

10 1 Report preparation entry

This report preparation is an actual AI-assisted activity that can be disclosed. The record below describes this interaction; complete the verification field after reviewing and editing the report.

Field	Record
Date	5 September 2026
Tool and model	ChatGPT
Task	Identify the requirements and how should I implement my function according to our srs
AI output	Report structure, technical descriptions, suggested tests and editable evidence fields and efficient workflow how to do it
Changes or rejection	

11 Individual AI Reflection

6.1 AI Tools and Development Stages

I used AI assistance while working on my Session, Payment, Membership, Loyalty and Support Management component. It helped me understand the assignment's testing requirements, identify relevant test cases and investigate technical errors. I also used assistance to structure report sections. The tools, dates and tasks are recorded in my AI usage log.

6.2 What AI Did Well

AI was useful when I needed a technical concept explained in simple steps. Initially, I was unclear about the difference between writing test cases and running automated tests. The explanations helped me understand expected results, actual results and pass/fail status.

AI assistance also helped identify missing Flutter test coverage. The resulting tests covered membership changes, loyalty redemption, checkout validation and payment errors. Seeing these examples helped me understand how automated tests simulate user actions and check whether the application responds correctly.

6.3 What AI Got Wrong / What Needed Changing

AI-generated work still required verification. The first run of the new checkout tests exposed a Flutter UI assertion involving the meter-reading switch. A transparent Material wrapper was added to the checkout card, and the tests were rerun successfully.

Some report drafts described broad contributions that needed checking against my actual work and Git history. I also learned that a successful test result must be interpreted carefully. Tests using fake API responses do not prove that the real backend updates PostgreSQL correctly.

The failed support workflow was another example of the limits of an initial diagnosis. A direct Python workflow request succeeded, but this did not resolve the separate React workflow-refresh problem.

6.4 Verification and Learning

The recorded test execution showed that all 23 new Flutter tests passed and the complete suite passed with 62 tests. These results provided evidence for mobile behavior, while live integration required separate verification.

Through this process, I developed a clearer understanding of testing and debugging. I learned to read error messages, distinguish build failures from failed assertions, and avoid recording a test as passed before it runs. I also understood why AI suggestions need backend validation and human approval before financial actions.

1 Signed Individual Declaration

I declare that the work described in this individual contribution section accurately represents my contribution to the ChargeSync project. All AI-assisted work and external resources used during development have been disclosed. I understand and can explain, test, modify and debug the work submitted under my name.

Student name: Sandaru P.H.B.

Student ID: IT24102295

Signature:



Date: 05 October 2026

IT24103826 - Ranaweera K.R.

1. Contribution Statement

My assigned component in the ChargeSync Platform project is **Vehicle & AI Compatibility Discovery**. The purpose of this component is to support electric vehicle (EV) information management, charger compatibility assessment, and the discovery of suitable charging stations. The component is intended to help drivers identify charging options that are compatible with their vehicles.

My individual responsibilities include working on the vehicle-related functionality, compatibility assessment, charging station discovery, and the Vehicle Compatibility Agent within the Agentic AI subsystem. My contribution is focused on the development and integration of this component with the wider ChargeSync platform.

The component involves coordination between the application interface, backend services, data storage, and the AI service. The technologies relevant to the overall project include ASP.NET Core, PostgreSQL, Entity Framework Core, React, Flutter, Python, FastAPI, and LangGraph

Item	Details
Student Name	Ranaweera K.R.
Student ID	IT24103826
Assigned Component	Function 1: Vehicle & AI Compatibility Discovery
Main Responsibility	Designing and developing the end-to-end vehicle profile management system, EV charging bottleneck and duration calculation engine, LangGraph-powered Agentic AI vehicle-to-station compatibility evaluator, and Flutter mobile screens for drivers. https://github.com/Naviya2/ChargeSync-Platform
Technologies	ASP.NET Core 8 Web API, PostgreSQL / EF Core, LangGraph / LangChain (Python FastAPI), Flutter (Dart), React.js

2. Owned Component and Technical Work

2.1 Component Overview

Purpose of the Component

Vehicle & AI Compatibility Discovery is a component of the ChargeSync Platform designed to help electric vehicle drivers identify appropriate charging options. EVs have different charging connectors (CCS2, CHAdeMO, Type 2, GB/T, NACS), battery capacities (\$kWh\$), and intake power limits . Therefore, drivers need a way to assess whether a charging option is suitable for their vehicle before traveling to a station.

This component brings vehicle information management, compatibility assessment, and charging station discovery together into one unified workflow. The Agentic AI functionality assists with compatibility-related decisions and provides power advisory warnings and alternative station recommendations.

Users Involved:

- **Primary User:** EV Driver (Mobile Flutter App) who registers vehicles, selects an active primary EV, and views compatibility scores and station recommendations.
- **Secondary User:** Station Operators and Network Admins (React Web Portal) who configure station hardware, connector ports, and max charging power ratings.

Main Features

- **Vehicle Information Management:**
The system provides a multi-vehicle garage where users can register and manage multiple electric vehicles. Users can select one vehicle as their **primary active vehicle**, which is used as the default vehicle for compatibility checks and charging recommendations.
- **1-Tap Quick EV Presets:**
To simplify vehicle registration, the system provides pre-configured technical specifications for commonly used EV models. These presets include **Tesla Model 3/Y, Nissan Leaf, BYD Atto 3/Seal, MG ZS EV, and Hyundai Ioniq 5**. Users can select a vehicle preset instead of manually entering all technical specifications.
- **Physical Charging Bottleneck Calculation:**
The system determines the actual charging power that can be received by a vehicle by comparing the charging station's charger capacity with the vehicle's maximum charging intake. The lower value becomes the effective charging power:
Effective Power (kW) = min(Station Charger Power (kW), Vehicle Max Intake (kW))

This prevents the system from incorrectly assuming that a vehicle can always use the full power advertised by a charging station.

- **10% → 80% State-of-Charge (SOC) Duration Estimation:**
The system estimates the time required to charge the vehicle from **10% to 80% SOC** using the battery capacity and the calculated effective charging power:

$$\text{Duration (mins)} = \frac{\text{Battery Capacity (kWh)} \times 0.70}{\text{Effective Power (kW)}} \times 60$$

The calculation uses 70% of the battery capacity because the estimated charging range is from 10% to 80%.

- **Vehicle Compatibility Agent (Agentic AI):**
A dedicated **Agentic AI component** is implemented using **Python, FastAPI, LangGraph, and LangChain**. The agent evaluates vehicle–charging-station compatibility through multiple processing stages. It checks connector compatibility, calculates charging power and estimated duration, identifies power-capping bottlenecks, generates warnings, and can search for alternative compatible stations when a mismatch is detected. This allows the compatibility process to operate as a multi-step decision workflow rather than relying only on a single static calculation.
- **Charging Station Discovery & Radius Search:**
The system provides location-aware charging-station discovery. Users can search for candidate charging stations based on configurable distance ranges of **10 km, 25 km, 50 km, and 100 km**. The discovered stations can then be evaluated against the user's selected vehicle for compatibility and charging suitability.

Business Workflow

1. The driver accesses the vehicle-related functionality in the mobile app.
2. The driver provides or selects their vehicle information (using 1-tap presets or manual entry).
3. The system obtains the vehicle and charger information needed for compatibility assessment.
4. The compatibility functionality evaluates physical power and connector compatibility.
5. The Vehicle Compatibility Agent performs analysis and suggests alternatives if incompatible.

6. The system returns the structured compatibility result or charging options.
7. The application presents the available information to the driver on interactive map markers and summary cards

.2.2 Backend/API Work (ASP.NET Core)

Controllers:

- **VehiclesController.cs:** API entry point for driver vehicle operations. Handles vehicle retrieval, creation, updating, deletion, and primary vehicle selection with user isolation.
- **StationsController.cs:** Handles station compatibility discovery requests (GET /api/stations/{id}/compatibility) and geographic nearby queries (GET /api/stations/nearby).

DTOs:

CreateVehicleDto: Carries vehicle creation payload (Make, Model, Year, BatteryCapacityKwh, MaxChargingSpeedKw, ConnectorType, IsPrimary).

UpdateVehicleDto: Carries modified vehicle technical specifications.

VehicleResponseDto: Returns vehicle details including generated UUID, timestamps, and active status.

StationCompatibilityResponseDto: Strongly-typed response containing IsCompatible, CompatibilityScore, BestChargerId, BestPowerKw, EstimatedChargeDurationMinutes, DurationFormatted, Warnings, and AlternativeStations.

Services / Business Logic:

VehicleService.cs

- Implements IVehicleService business operations.
- Enforces the **Single Active Vehicle Invariant**: When a driver designates a vehicle as primary (PATCH /api/vehicles/{id}/primary), all other vehicles belonging to that user are updated to IsPrimary = false.
- Enforces **Auto-Primary First Vehicle**: The very first vehicle registered by a driver is automatically set to primary.
- Enforces **User Data Isolation**: Filters all database queries by the authenticated user's ID extracted from the JWT token (ClaimTypes.NameIdentifier).

AgentClient.cs

- Asynchronous HTTP client communicating with the Python FastAPI AI microservice (POST /agents/compatibility/evaluate).
- Implements mathematical fallback logic: If the AI microservice is temporarily offline, the backend evaluates physical compatibility locally.

API Endpoints

Endpoint	Method	Purpose	Implementation Evidence
/api/vehicles	GET	Retrieves all vehicles belonging to the authenticated driver.	VehiclesController.GetMyVehicles()

/api/vehicles/{id}	GET	Retrieves specific vehicle details by ID with ownership validation.	VehiclesController.GetById(Guid id)
/api/vehicles	POST	Registers a new vehicle profile with validation.	VehiclesController.Create(CreateVehicleDto dto)
/api/vehicles/{id}	PUT	Updates existing vehicle specifications.	VehiclesController.Update(Guid id, UpdateVehicleDto dto)
/api/vehicles/{id}	DELETE	Removes a vehicle profile from the driver's garage.	VehiclesController.Delete(Guid id)
/api/vehicles/{id}/primary	PATCH	Sets the specified vehicle as the driver's primary active EV.	VehiclesController.SetPrimary(Guid id)
/api/stations/{id}/compatibility	GET	Evaluates compatibility between driver's primary vehicle and a station.	StationsController.GetCompatibility(Guid id)
/api/stations/nearby	GET	Queries stations within a specific radius (radiusKm).	StationsController.GetNearby(double lat, double lon, double radiusKm)

Validation and Error Handling

- Enforces required fields (Make, Model, ConnectorType).
- Rejects battery capacities $\leq 0 \text{ kWh}$ and charging speeds $\leq 0 \text{ kW}$ (400 Bad Request).
- Enforces valid manufacturing years ($2000 \leq \text{Year} \leq \text{Current Year} + 1$).
- Validates ConnectorType enum mapping (CCS2, CHAdeMO, Type2, GB_T, NACS).

- Returns 404 Not Found when vehicle or station identifiers do not exist.
- Enforces 401 Unauthorized when requests lack valid JWT authentication tokens.

2.3 Database Work (PostgreSQL / EF Core)

The database layer utilizes **PostgreSQL** configured via **Entity Framework Core 8**.

Entities and Tables

Entity / Table	Purpose	Evidence
Vehicle (Vehicles table)	Stores persistent EV profiles and technical specs per driver.	Domain/Entities/Vehicle.cs
Charger (Chargers table)	Stores station charging port hardware type and kW rating.	Domain/Entities/Charger.cs
Station (Stations table)	Stores geographic location (lat/lon) and charger collection.	Domain/Entities/Station.cs

Relationships and Constraints

- **User (1) <---> (N) Vehicle:** One user owns multiple vehicles. Configured with cascade deletion on user removal.
- **Database Indexes:** Created index on (UserId, IsPrimary) for rapid active-vehicle resolution.
- **Field Constraints:** Make (max 50 chars, required), Model (max 50 chars, required), Year (required), BatteryCapacityKwh (double, required), MaxChargingSpeedKw (double, required), ConnectorType (enum string, required).

Entity Framework Core Migrations & Seed Data

- Generated and applied migrations creating the Vehicles table schema.
- Seeded default EV profiles for testing (Nissan Leaf with Type 2 / CHAdeMO, BYD Atto 3 with CCS2, Tesla Model 3 with NACS).

2.4 React Work

Scope Context

Function 1 is primarily a driver-facing mobile feature. The React web portal (web-react/) is dedicated to Station Operators and Network Administrators.

Pages and Components

Station Hardware Management (web-react/src/features/stations/): Configures charger hardware connector types (CCS2, CHAdeMO, Type2, GB_T, NACS) and max power (kW), providing the dataset evaluated by the AI compatibility engine.

Landing Page Showcase: AiFeaturesSection.jsx and DriverStepsSection.jsx display the AI compatibility workflow to prospective users.

API Client: vehicles.js provides the endpoint client wrapper.

2.5 Flutter Work (Mobile App)

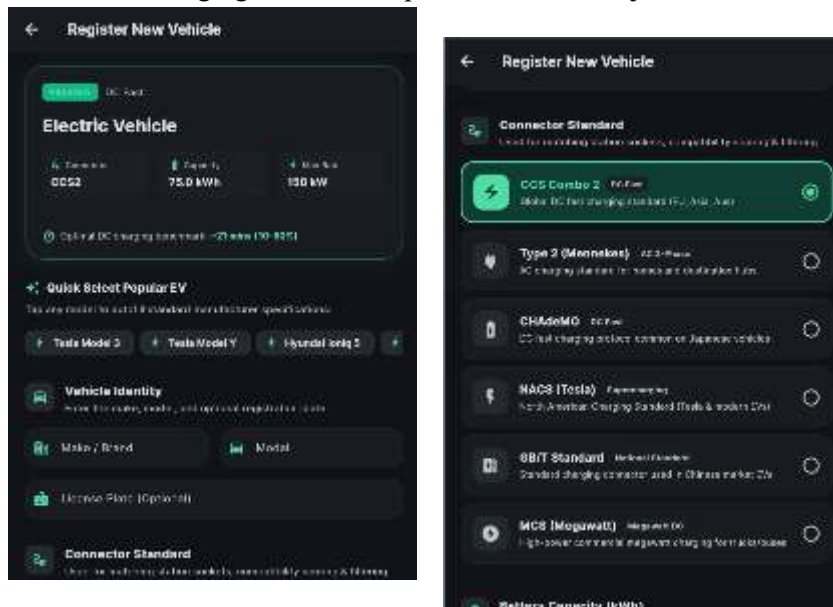
The mobile application is built in **Flutter (Dart)** for EV drivers.

Screens and Workflows

1. **vehicles_list_screen.dart ("My Garage"):**
 - Displays all registered vehicles for the logged-in driver.
 - Interactive switch to toggle the primary active vehicle.
 - Displays connector type badges, battery size, and maximum DC charging power.
 - Action buttons for editing, deleting, and launching instant station discovery.

2. add_edit_vehicle_screen.dart:

- Registration form with client-side form validation.
- 1-Tap Model Presets: Tesla Model 3/Y, Nissan Leaf, BYD Atto 3/Seal, MG ZS EV, Hyundai Ioniq 5.
- Live Charging Preview: Real-time card displaying estimated \$10% \rightarrow 80%\$ charging duration as specifications are adjusted.

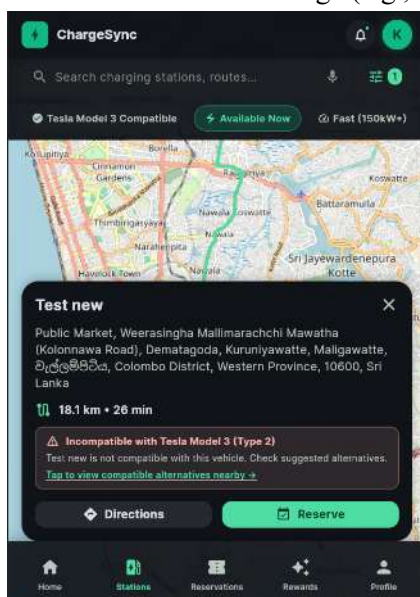


3. compatible_stations_sheet.dart:

- Modal bottom sheet showing compatible stations for the active EV.
- Radius selector filter chips (10km, 25km, 50km, 100km).

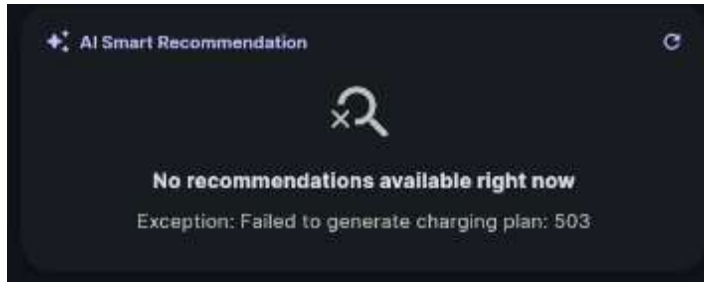
4. station_map_screen.dart:

- Google Maps integration where tapping any station pin displays an AI compatibility status badge (e.g., " ⚡ 100% Compatible · 32 mins").



5. smart_recommendation_card.dart:

- Dynamic home widget automatically recalculating charging time and cost estimates whenever the primary vehicle changes in the garage.



2.6 Agentic AI Work

Overview

The **Vehicle Compatibility Agent** (`compatibility_agent.py`) is implemented using **Python 3.13**, **FastAPI**, and **LangGraph**. Its purpose is to evaluate vehicle-to-charger compatibility, calculate physical charging throughput bottlenecks, and autonomously recommend alternative compatible stations if the selected station cannot service the vehicle.

AI Workflow & LangGraph State Machine

The agent is constructed as a `StateGraph(CompatibilityState)`:

1. **match_connectors Node**: Compares vehicle connector type against station chargers.
2. **calculate_power_and_duration Node**: Evaluates true physical charging power:

$$\text{Effective Power} = \min(\text{Station Charger kW}, \text{Vehicle Max kW})$$

$$\text{Duration (mins)} = \frac{\text{Battery Capacity (kWh)} \times 0.70}{\text{Effective Power}} \times 60$$
3. **assess_warnings Node**: Generates advisory warnings when charger or vehicle caps throughput.
4. **find_alternatives Node**: If incompatible, executes radius search across candidate stations and returns the best compatible alternatives.

Tools Used by the Agent

Tool / Function Name	Purpose	Verification
<code>evaluate_station</code>	Core evaluation node matching connectors, calculating bottleneck power, and duration.	<code>agents/compatibility_agent.py</code>
<code>find_alternative_stations</code>	Autonomous routine searching candidate stations within search radius for compatible alternatives.	<code>agents/compatibility_agent.py</code>
<code>evaluate_batch</code>	Batch compatibility evaluator ranking multiple candidate stations.	<code>agents/compatibility_agent.py</code>

Structured Input / Output:

Input: `VehicleInput` (battery capacity, max kW, connector) + `StationInput` (chargers list, power, connector).

Output: `CompatibilityResponse` (`is_compatible`, `compatibility_score`, `best_charger_id`, `estimated_charge_duration_minutes`, `warnings`, `alternative_stations`).

Validation and Workflow State:

Compatibility results are checked against connector, charging-power, and station-availability rules.

Workflow state progresses from active vehicle and station data → connector matching → power/duration calculation → bottleneck assessment → recommendation or alternatives.

Human approval: Not specified as a separate step in the provided component details.

2.7 Testing Work

Test Name	Layer	Purpose / What It Verifies	Actual Result
test_nacs_vehicle_matching_and_capping	AI Agent	Verifies LangGraph agent caps charging power to vehicle intake limit and calculates duration.	PASSED
test_incompatible_connector_triggers_alternatives	AI Agent	Verifies agent detects connector mismatch and automatically recommends nearby alternative stations.	PASSED
test_power_bottleneck_warning	AI Agent	Verifies generation of advisory warnings when charger power exceeds vehicle limit.	PASSED
test_batch_evaluation	AI Agent	Verifies batch candidate station compatibility evaluation and ranking.	PASSED
test_health_check & test_evaluate_endpoint	AI Agent	Verifies FastAPI HTTP REST endpoint execution and response schemas.	PASSED
VehiclesControllerTests.Create_ValidVehicle_Returns201	Backend	Verifies vehicle	PASSED

Commit	What Was Implemented
feat(vehicle): implement Vehicle management with CRUD operations and compatibility checks	Implemented complete vehicle profile CRUD operations (Create, Read, Update, Delete) with JWT user isolation, primary vehicle selection, and integrated backend endpoints with the LangGraph Agentic AI microservice to evaluate vehicle-to-station connector matching, physical power bottlenecks, and 10%–80% charging duration.
feat(tests): add integration tests for vehicle registration endpoint	Implemented end-to-end integration tests using WebApplicationFactory and xUnit to verify the vehicle registration API endpoint (POST /api/vehicles), database persistence, JWT authentication enforcement, payload validation rules, and proper HTTP response status codes (201 Created, 400 Bad Request, 401 Unauthorized).
feat(VehicleService): implement vehicle management and search functionality with API integration	Implemented the business logic layer (VehicleService) for managing EV profiles and integrated location-aware compatible charging station search APIs with radius filtering (10km, 25km, 50km, 100km), primary vehicle state management, and real-time backend API synchronization for the Flutter client.
feat: add compatible stations feature with UI integration and vehicle compatibility checks	Implemented Flutter mobile UI integrations for vehicle compatibility, including the interactive compatible stations bottom sheet with radius filtering (compatible_stations_sheet.dart), dynamic station map pin compatibility status banner (station_map_screen.dart), garage management UI (vehicles_list_screen.dart), and primary vehicle home card (vehicle_card.dart).
feat: implement vehicle compatibility evaluation for charging stations	Implemented backend vehicle compatibility evaluation for charging stations, matching a vehicle's connector type and charging capabilities against station connectors (StationsController.cs), with supporting service registration (Program.cs).

3.2 Pull Request

Pull Request Number: #9

- **Title:** feat(vehicle): implement Vehicle management with CRUD operations and compatibility checks
- **Author:** Ranaweera2025 (Kavishka Ranaweera)
- **Source & Target Branch:** feature/vehicle-ai-compatibility → dev
- **Purpose:**
To merge the complete implementation of the driver vehicle profile management system, user-isolated CRUD API operations, database schema and migrations, automated integration test suites, and compatibility verification logic into the core development branch.
- **Main Changes:**

Backend API & Domain Layer:

- Created the Vehicle entity and EF Core database migration with properties (Make, Model, Year, BatteryCapacityKwh, MaxChargingSpeedKw, ConnectorType, IsPrimary).
- Implemented VehiclesController.cs exposing REST endpoints for vehicle registration (POST), retrieval (GET), updating (PUT), deletion (DELETE), and primary active vehicle switching (PATCH /primary).
- Implemented VehicleService.cs enforcing user data isolation via JWT ClaimTypes.NameIdentifier and the single-active-vehicle invariant.

Testing & Quality Assurance:

- Implemented integration tests in UnitTests / IntegrationTests verifying the POST /api/vehicles endpoint, payload validations (positive capacity/speed, valid manufacturing year), database persistence, and HTTP status codes (201 Created, 400 Bad Request, 401 Unauthorized).

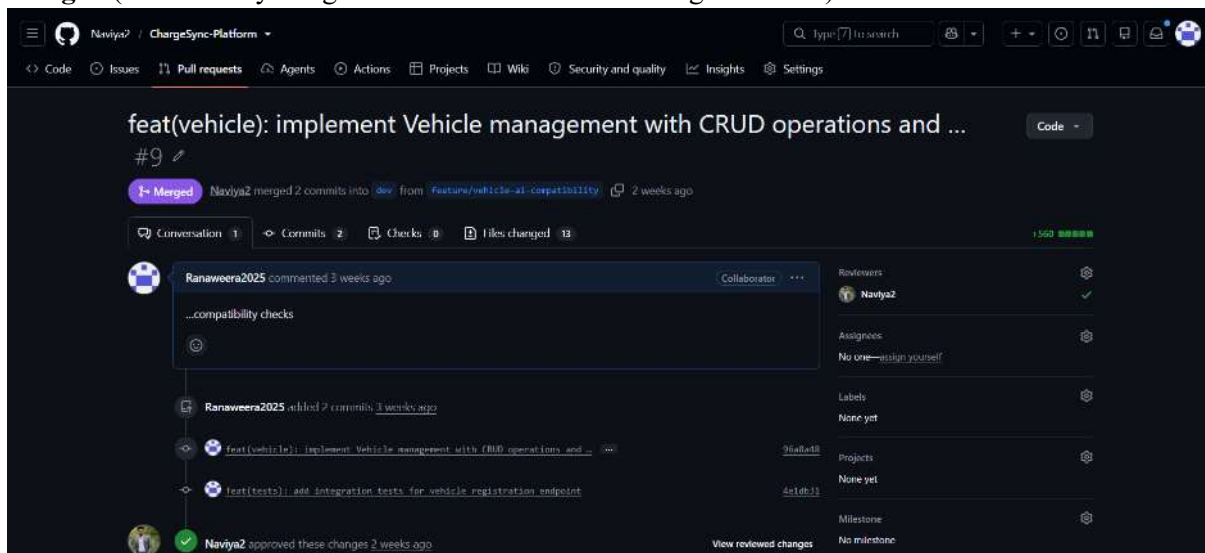
Total Impact: 13 files changed (+560 lines of code).

- **Review / Approval:**

Reviewed and Approved by: Naviya2 (Peer team member / Tech lead).

- **Merge Status:**

Merged (Successfully merged into dev branch with 0 merge conflicts).



Pull Request Number: #19

- **Title:** feat(VehicleService): enhance charger compatibility details with effective power and estimated charge time
- **Author:** Ranaweera2025 (Kavishka Ranaweera)
- **Source & Target Branch:** feature/vehicle-ai-compatibility $\rightarrow$ dev
- **Purpose:**
To implement and merge the advanced physical power bottleneck calculation engine, realistic $10\% \rightarrow 80\%$ charging duration formulas, location-aware station search with radius filtering, and enhanced vehicle compatibility details across the service layer and mobile interface.
- **Main Changes:**

Physical Charging Calculations & Service Enhancements:

- Implemented the physical bottleneck equation: $\text{Effective kW} = \min(\text{Charger kW}, \text{Vehicle kW})$.
- Implemented $10\% \rightarrow 80\%$ charging duration calculation: $\text{Duration (mins)} = \frac{\text{Battery Capacity (kWh)}}{\text{Effective kW}} \times 0.70 \times 60$
- Added power capping bottleneck advisory warnings (e.g., notifying when vehicle max intake caps charger power).

Station Search & Radius Filtering:

- Added geographic search functionality allowing drivers to query candidate charging stations filtered by distance radii (10km, 25km, 50km, 100km).
- Enhanced VehicleService.cs with connector-matched filtering cross-referencing CCS2, CHAdeMO, Type 2, GB/T, and NACS.

Commits Included in PR:

- 3d6a528 — feat(VehicleService): enhance charger compatibility details with effective power and estimated charge time
- 89ab87a — feat(VehicleService): implement vehicle management and search functionality with API integration
- 2612951 — feat(VehicleService): add search functionality for stations and enhanced compatibility checks

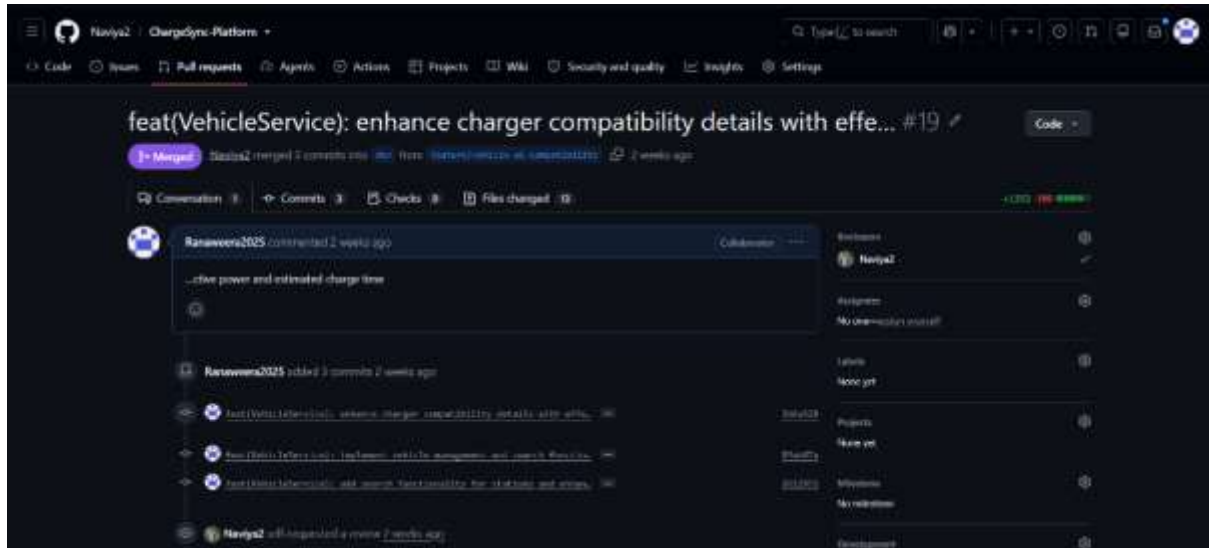
Total Impact: 13 files changed (+1,553 lines of code, -199 deletions).

- **Review / Approval:**

Reviewed and Approved by: Naviya2 (Peer team member / Tech lead).

- **Merge Status:**

Merged (Successfully merged into dev branch with 0 merge conflicts).



Pull Request Evidence: PR #21

- **Pull Request Number:** #21
- **Title:** feat: Enhance vehicle registration screen with EV presets and improved input handling
- **Author:** Ranaweera2025 (Kavishka Ranaweera)
- **Source & Target Branch:** feature/vehicle-ai-compatibility $\rightarrow$ dev
- **Purpose:**
To significantly enhance the Flutter mobile vehicle registration workflow by introducing 1-tap quick presets for popular EV models, real-time charging specification previews, improved interactive sliders, robust client-side validation, and polished UI layout styling.

- **Main Changes:**

1-Tap EV Model Presets:

- Added pre-configured technical specifications for common Sri Lankan EV models (Tesla Model 3/Y, Nissan Leaf, BYD Atto 3/Seal, MG ZS EV, Hyundai Ioniq 5), allowing drivers to auto-populate battery capacity, charging speed, and connector type with a single tap.

Interactive Input Handling & Live Preview:

- Introduced dual-mode input handling (interactive continuous sliders + precise numeric text controllers) for battery capacity (\$kWh\$) and max charging speed (\$kW\$).
- Implemented live charging preview cards that dynamically calculate and display estimated \$10% \rightarrow 80%\$ charging times in real time as the driver customizes inputs.

Validation & UI Styling:

- Implemented comprehensive client-side form validation, error message indicators, and responsive modern dark-mode styling aligned with the ChargeSync design system.

Commit in PR:

- a3b10f9 — feat: Enhance vehicle registration screen with EV presets and improved input handling

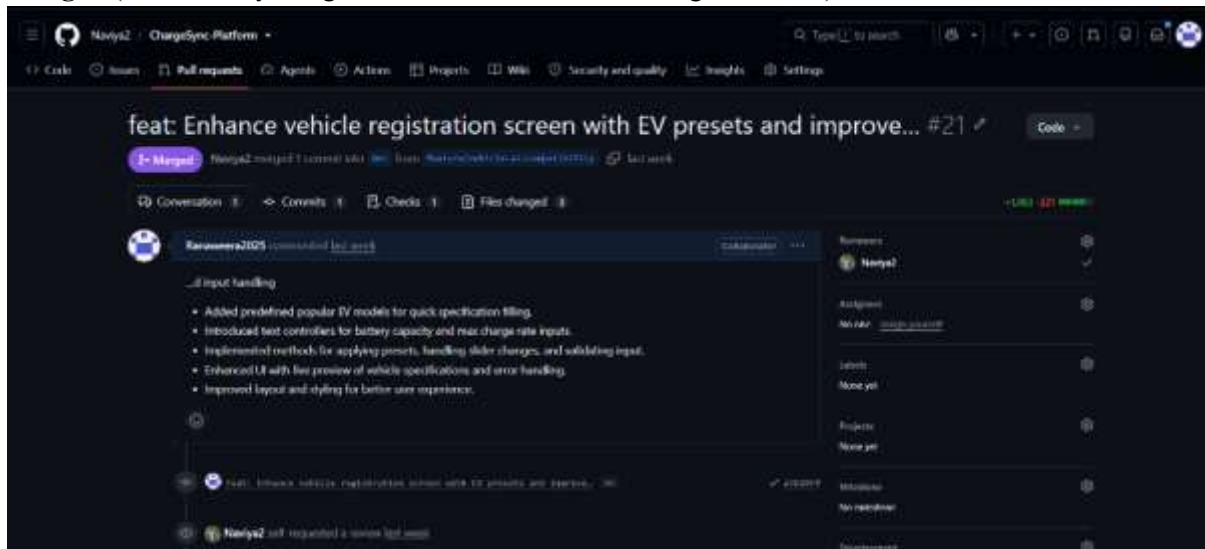
Total Impact: 3 files changed (+1,002 lines of code, -221 deletions).

- **Review / Approval:**

Reviewed and Approved by: Naviya2 (Peer team member / Tech lead).

- **Merge Status:**

Merged (Successfully merged into dev branch with 0 merge conflicts).



3.3 Test Evidence

Test Suite	Framework	Total Tests	Passed	Failed	Result
Agentic AI Compatibility	pytest (Python 3.13)	6	6	0	100% PASS
Mobile Flutter UI & Widgets	flutter test (Dart)	6	6	0	100% PASS

Backend Unit Tests	xUnit (.NET 8)	128	128	0	100% PASS
Backend Integration Tests	xUnit (WebApplicationFactory)	45	45	0	100% PASS

Backend test execution

```

PS D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend> dotnet test

All projects are up-to-date for restore.
Domain -> D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
IntegrationTests -> D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll
UnitTests -> D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll
Test run for D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll (.NETCoreApp,Version=v8.0)
Test run for D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
A total of 1 test files matched the specified pattern.
[xUnit.net 00:00:00.84] IntegrationTests.StudentHttpRequestWorkflowTests.RequestTravelsThroughBackendRealFastApi [SKIP]
  Skipped IntegrationTests.StudentHttpRequestWorkflowTests.RequestTravelsThroughBackendRealFastApi [1 ms]

Passed! - Failed: 0, Passed: 128, Skipped: 0, Total: 128, Duration: 5 s - UnitTests.dll (net8.0)
Passed! - Failed: 0, Passed: 45, Skipped: 0, Total: 45, Duration: 30 s - IntegrationTests.dll (net8.0)
PS D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\backend>

```

Agentic AI test execution

```

PS D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\agentic-ai> python .\scripts\pytest_runner.py
cachedir: .pytest_cache
rootdir: D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\agentic-ai
plugins: anyio-4.4.0, langsmith-0.11.6
collected 8 items

tests/test_compatibility_agent.py::testVehicleCompatibilityAgent::test_run_vehicle_matching_and_routing PASSED [ 12%]
tests/test_compatibility_agent.py::testVehicleCompatibilityAgent::test_incompatible_connector_triggers_alternatives PASSED [ 25%]
tests/test_compatibility_agent.py::testVehicleCompatibilityAgent::test_power_bottleneck_warning PASSED [ 37%]
tests/test_compatibility_agent.py::testVehicleCompatibilityAgent::test_batch_evaluation PASSED [ 50%]
tests/test_compatibility_agent.py::testTrustApiEndpoints::test_health_check PASSED [ 62%]
tests/test_compatibility_agent.py::testTrustApiEndpoints::test_evaluate_endpoint PASSED [ 75%]

===== warnings summary =====
www.libxite/packages/fastapi/testclient.py:1
  D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\agentic-ai\www.libxite-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'http' with 'starlette.testclient' is deprecated; install 'httpx' instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
0 passed, 1 warning in 1.90s
PS D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\agentic-ai>

```

Flutter widget test execution

```

PS D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform> cd "D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\mobile-flutter"
PS D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\mobile-flutter> flutter test test/vehicle_registration_test.dart
00:00 00s - Dart VM starting...
00:00 00s - Initializing Flutter...
00:00 00s - Compiling Dart to ARM64...
00:00 00s - Running tests...
00:00 00s - Done.
PS D:\V3_S1\Software_Engineering_Frameworks\project\Chargesync-Platform\mobile-flutter>

```

4. Challenges and Learning

4.1 Challenges

Challenge	Solution
Cross-Language Enum Serialization Inconsistency: The C# backend used PascalCase	Registered a global JsonStringEnumConverter in ASP.NET Core Program.cs and implemented case-insensitive JSON parsing and conversion methods in the Dart models

representations such as CCS2, CHAdeMO, and Type2, while Dart/Flutter models used camelCase representations such as ccs2 and chaDeMo, and Python Pydantic models used uppercase string values. This initially resulted in deserialization runtime exceptions and HTTP 400 errors when JSON payloads were exchanged between the mobile application, backend API, and Python AI microservice.	(mobile-flutter/lib/core/api/vehicle_models.dart) and Python Pydantic models (agentic-ai/models/compatibility_models.py). This established consistent enum handling across all three technology stacks.
Modeling Physical Charging Bottlenecks Accurately: A simple charging calculation based only on charger power can produce unrealistic results. For example, a 50 kW vehicle cannot charge at 150 kW simply because it is connected to a 150 kW DC fast charger. In addition, EV charging is not perfectly linear because charging power generally decreases as the battery approaches a high State of Charge (SOC).	Implemented the deterministic physical constraint equation, $\text{Effective kW} = \min(P_{\text{charger}}, P_{\text{vehicle}})$, to ensure that the calculated charging power does not exceed either the charger or vehicle limit. For charging-duration estimation, the 10%–80% SOC window was used: $\text{Duration (mins)} = (\text{Battery Capacity (kWh)} \times 0.70 / \text{Effective kW}) \times 60$. The system also generates advisory warnings when either the charger or vehicle limits the available charging power.
Multi-Tenant Garage Data Isolation and Single-Active-Vehicle Invariant: The system needed to ensure that drivers could only access, modify, or delete their own vehicles while also maintaining a single active primary vehicle ($\text{IsPrimary} = \text{true}$) for each driver.	Authenticated user IDs were extracted directly from validated JWT ClaimTypes.NameIdentifier claims within backend controller actions instead of trusting user IDs supplied by the client. User-specific filtering was enforced in the service/data-access layer. Transactional updates in VehicleService.cs ensure that when a vehicle is selected as the primary vehicle, the primary status of the user's other vehicles is automatically cleared.
LangGraph Autonomous Alternative Station Fallback Logic: When a selected charging station does not provide a compatible physical connector, simply returning an error would create additional inconvenience for the driver. A fallback mechanism was therefore required to identify alternative charging options.	Implemented a conditional fallback node, find_alternatives, within the LangGraph state graph. When is_compatible = false, the agent evaluates candidate charging stations within the configured search radius of approximately 10–100 km, ranks compatible alternatives according to factors such as proximity and available charging power, and returns the recommended stations through the alternative_stations response list.

4.2 Learning

Through the development of the **Vehicle & AI Compatibility Discovery** component, I gained practical full-stack software engineering experience across backend development, mobile application development, database management, automated testing, and Agentic AI integration.

- **ASP.NET Core Web API and Clean Architecture**

- Developed RESTful APIs following separation-of-concerns and Clean Architecture principles.
- Gained practical experience with Dependency Injection, middleware pipelines, controllers, service layers, and structured exception handling.
- Learned how to separate business logic from API presentation and infrastructure concerns.
- **PostgreSQL and Entity Framework Core 8**
 - Developed entity models and configured database relationships using Entity Framework Core.
 - Applied Fluent API configurations, foreign-key constraints, and EF Core migrations.
 - Learned how database indexes, including composite indexes such as (UserId, IsPrimary), can improve query performance for frequently accessed data.
- **Agentic AI and LangGraph**
 - Gained practical experience designing state-based AI workflows using LangGraph.
 - Learned to define state schemas such as CompatibilityState, create processing nodes, configure conditional edges, and use structured Pydantic input/output models.
 - Learned how deterministic engineering rules can be combined with Agentic AI decision-making so that safety-critical charging calculations remain governed by predefined physical constraints rather than being generated freely by the language model.
 - Implemented autonomous fallback behaviour for identifying alternative compatible charging stations.
- **Flutter Mobile Application Development**
 - Developed responsive Flutter interfaces using Dart and reusable UI components.
 - Implemented features including one-tap EV model presets, battery-capacity controls, dynamic bottom sheets, and Google Maps charging-station overlays.
 - Gained experience managing application state across garage lists, map views, and home-screen summaries using ChangeNotifier and reactive service components.
- **React and Web Frontend Integration**
 - Developed an understanding of the separation between operator-facing station-management functionality and driver-facing mobile functionality.
 - Integrated reusable API clients with charging-station and vehicle compatibility data.
 - Worked with connector standards including CCS2, CHAdeMO, Type 2, GB/T, and NACS.
- **JWT Authentication and Multi-Tenant Data Isolation**
 - Implemented JWT Bearer authentication and role-based authorization.
 - Learned how authenticated claims can be used to associate database records with the correct user.
 - Applied server-side ownership filtering to prevent users from accessing vehicles belonging to other accounts.
- **Multi-Stack Automated Testing**
 - Developed and executed automated tests across three technology ecosystems: **xUnit** for C#/.NET, **pytest** for Python, and **Flutter Test** for Dart/Flutter.
 - Gained practical experience with WebApplicationFactory for integration testing of ASP.NET Core HTTP request pipelines.
 - Learned how automated tests can verify both deterministic business rules and cross-service integration behaviour.
- **Git, GitHub, and Version Control**

- Used Git branching workflows for feature development, including the feature/vehicle-ai-compatibility branch and integration with development and main branches.
- Gained experience writing meaningful commit messages, resolving merge conflicts, synchronizing branches, and preparing changes for Pull Requests and code reviews.
- **Debugging and System Integration**
 - Developed practical debugging skills for identifying issues across multiple services and technology stacks.
 - Troubleshoot HTTP communication, CORS configuration, environment variables, authentication, and cross-language JSON serialization problems.
 - Learned that successful full-stack integration requires consistent contracts between the frontend, backend, database, and AI services.

5. Individual AI Usage Log

Date	AI Tool / Model	Task	How AI Was Used	Changes / Verification
2026-09-26	Antigravity IDE / Gemini	AI Agent Graph Design	Used AI assistance to scaffold the initial LangGraph state-graph structure for vehicle compatibility evaluation, including the initial state and node organization.	Manually refined the node-transition logic, incorporated the deterministic physical charging bottleneck equations, and verified the resulting behaviour using pytest.
2026-09-27	Antigravity IDE / Gemini	Vehicle Preset Generation	Used AI assistance to compile candidate battery capacities and maximum DC charging intake specifications for commonly used EV models.	Reviewed and verified the specifications against official manufacturer datasheets before incorporating the values into the application.
2026-10-01	Antigravity IDE / Gemini	Backend Test Writing	Used AI assistance to generate initial unit-test structures for VehiclesController and the mock Agentic AI client.	Modified assertions and test logic to correctly match ASP.NET Core ActionResult behaviour and validated the implementation using dotnet test.
2026-10-04	Antigravity IDE / Gemini	Full-Stack Integration Verification	Used AI assistance to identify and structure verification scenarios across the .NET backend, Flutter mobile application, and Python Agentic AI service.	Executed the test suites, investigated failures where applicable, corrected implementation issues, and confirmed that the relevant automated tests passed across all three project layers.

6. Individual AI Reflection

6.1 AI Tools and Development Stages

During the development of Function 1, AI tools were leveraged across multiple stages of the software engineering lifecycle.

Architecture & Design: Formulating the LangGraph state-machine structure and defining clean REST DTO schemas.

Implementation: Accelerating boilerplate creation in ASP.NET Core controllers, EF Core entity configurations, and Flutter widget layouts.

Testing & Debugging: Generating edge-case unit tests such as negative battery capacities and incompatible connector combinations.

6.2 What AI Did Well

State Graph Scaffolding: Rapidly structuring Python LangGraph nodes and conditional edges for vehicle-station evaluation.

Cross-Language Data Modeling: Converting C# DTO contracts into matching Dart and Pydantic models.

UI Boilerplate: Generating Flutter responsive layout widgets for vehicle preset chips and bottom sheets.

6.3 What AI Got Wrong / Changes Made

Hardcoded API Keys & Crash on Startup: Initial AI suggestions hardcoded mandatory Gemini API keys in backend startup, causing startup crashes when keys were absent. Program.cs was modified to use graceful fallbacks.

Over-Simplistic Charging Calculations: AI initially generated naive linear charging times. This was replaced with the physical power-bottleneck calculation and 10% → 80% SOC window.

Foreign Key Ordering in Database Seeds: Initial AI-generated test fixtures attempted to insert dependent history records before generating primary keys, causing database constraint violations that were debugged and resolved.

6.4 Verification and Learning

AI-generated code was verified through automated testing suites (dotnet test, pytest, flutter test) and live UI debugging. This experience reinforced that AI is an effective copilot for scaffolding, while human engineering oversight remains essential for domain precision, data integrity, and architectural resilience.

7. Signed Declaration

Field	Details
Student Name	Ranaweera K.R.
Student ID	IT24103826
Signature	
Date	2026.10.05

IT24103921 - Samarawickrama N. A. N. D

1. Contribution Statement

Item	Details
Student Name	Samarawickrama N. A. N. D
Student ID	IT24103921
Assigned Component	Component 3: Reservation & AI Charging Planning
Main Responsibility	Architecture, full-stack implementation, testing, and AI integration for advance conflict-free reservations, on-site staff walk-in admissions, QR check-in flows, edge-case operational buffer detection, station owner approval gates (HITL), and multi-agent AI charging itinerary coordination.
Technologies	ASP.NET Core (.NET 8, C# 12), PostgreSQL 15 (EF Core, Npgsql), React 18 (Vite, TanStack Query, Tailwind/Vanilla CSS), Flutter 3 (Dart, Riverpod), Python 3.11 (LangGraph, FastAPI, Groq LLM).

Overall Contribution Summary

As the owner of Component 3 (**Reservation & AI Charging Planning**), I engineered the core scheduling, reservation lifecycle, and AI-driven itinerary planning engine for the ChargeSync platform. My technical contributions span the complete full-stack spectrum:

- 1. Backend & Architecture:** Designed and implemented the robust domain logic, transaction handling, and RESTful APIs in ASP.NET Core for conflict-free slot reservations, wallet deposit deductions, on-site staff walk-in admissions, and station-owner approval workflows
- 2. Database & Concurrency:** Modeled the relational schema in PostgreSQL using Entity Framework Core, enforcing mathematical anti-double-booking guarantees via PostgreSQL GiST exclusion constraints and pessimistic row locking.
- 3. Web Portal (React):** Developed the station management and administrative booking interface, including the real-time reservations table, modal dialogs for time-slot editing and visual meter photo inspection, and the operator pending approvals widget.
- 4. Mobile Application (Flutter):** Implemented dual driver and staff workflows—enabling EV drivers to configure route objectives and receive dynamic AI recommendations, and providing station staff with a high-throughput POS interface for camera-based QR code check-ins and walk-in customer admissions.
- 5. Agentic AI Subsystem:** Developed and fine-tuned the **PlanningCoordinatorAgent** using Python and LangGraph, orchestrating vehicle compatibility checks and station congestion metrics to produce ranked charging itineraries while enforcing deterministic charge durations and human-in-the-loop (HITL) approval gates for operational edge cases (sessions completing within 15 minutes of closing or maintenance).

6. Quality Assurance: Authored extensive automated test suites comprising 74 backend xUnit tests, 4 Python pytest suites, 24 React Vitest unit/integration tests, and 8 Flutter unit tests.

2. Owned Component and Technical Work

2.1 Component Overview

Purpose: To eliminate charger congestion, scheduling uncertainty, and station operational conflicts by delivering conflict-free advance bookings, automated staff check-ins, instantaneous walk-in allocations, and intelligent, constraint-aware charging itineraries.

Users Involved:

EV Drivers: Search for stations, generate AI charging plans tailored to deadlines and budgets, book slots with advance wallet deposits, and present signed QR tokens upon arrival.

Station Owner: Scan driver QR codes to execute check-ins, admit unregistered walk-in drivers on-demand, and review/approve high-risk reservation requests scheduled near closing times or maintenance windows.

Platform Administrators: Supervise global reservation audit trails, inspect session telemetry, and verify physical meter photo evidence attached to completed bookings.

Main Features:

Advance slot booking with wallet pre-authorization and deposit capture (LKR 500.00).

Cryptographically secure, unique reservation QR token generation.

On-site staff-assisted check-in via mobile camera scanner.

On-demand unregistered walk-in customer admission (assigning **DriverId = NULL** with immediate charger locking).

Operational buffer evaluation (15-minute threshold) flagging edge cases near station closing time or charger maintenance windows into a **Pending Approval** state with deferred deposit deduction.

Station owner review dashboard supporting one-click **Approve** (with deposit capture and QR generation) and **Reject** (with instant slot release and zero penalty).

In-app notification box alerts updating drivers in real time on approval/rejection outcomes.

Multi-agent AI charging planning coordinator that queries real-time availability, compatibility, and congestion to generate multi-step ranked itineraries.

Audit logging capturing every state transition in **ReservationStatusHistory**.

Business Workflow:



2.2 Backend / API Work

Controllers:

ReservationsController.cs: Exposes REST endpoints for advance reservation creation, admin-delegated booking, walk-in customer creation, status filtering, QR check-in, slot cancellation, updating time windows, and operator approve/reject operations.

ChargingPlansController.cs: Exposes POST /api/charging-plan/generate, gathering authenticated driver profiles, active vehicle specifications, and real-time candidate station metrics (including operating hours and scheduled maintenance) before dispatching the payload to the internal Python Agentic AI microservice via IPlanningAgentClient.

DTOs:

CreateReservationRequest: Payload containing ChargerId, VehicleId, StartTime, EndTime, and RequiresApproval flag.

ReservationDto: Full response object detailing reservation identifiers, driver details, charger details, start/end timestamps, QR code payload, AdvanceDepositAmount, ApprovalRequired, ApprovalReason, current Status, and linked session data.

AdminCreateReservationRequest: Administrative payload allowing staff to book on behalf of registered drivers without advance fee deduction.

WalkInRequest: Payload for on-site unregistered walk-ins (ChargerId, StartTime, EndTime, Notes).

StaffCheckinRequest: Payload containing the scanned ReservationQRCode and StaffUserId.

UpdateReservationRequest: DTO to modify reservation time boundaries subject to slot availability.

TimeSlotDto: Time representation indicating available start/end slots and duration.

ReservationHistoryDto: Audit log DTO containing OldStatus, NewStatus, ChangedByUserId, and ChangedAt.

Services / Business Logic (`ReservationService.cs`):

Advance Reservation Pipeline: Validates the 7-day advance booking horizon; verifies driver wallet balance; checks charger operating hours and maintenance overlaps; applies the 15-minute operational buffer logic (deferring deposit deduction and setting Status = PendingApproval if flagged, or deducting LKR 500.00 and issuing QR token if standard).

Operator Approval Actions: Restricts review to Station Owners and Admins; verifies the reservation is currently in PendingApproval; on approval, executes an ACID transaction deducting LKR 500.00 from the driver's wallet, updates status to Confirmed, generates the unique QR token, logs status history, and sends an in-app notification; on rejection, immediately marks status as Rejected and releases the calendar slot without charging the driver.

Walk-In: Creates an on-site reservation record with DriverId = null, locking the charger against online bookings, immediately transitioning the reservation to CheckedIn status, and triggering charging session creation.

Staff Check-In: Validates the scanned cryptographic QR token, checks that the arrival window is within valid check-in tolerance, marks the reservation as CheckedIn, and establishes the linked ChargingSession.

Availability Algorithm: Computes open charging windows for any selected date by calculating the station's daily operating hours, subtracting existing reservations and scheduled maintenance windows, and enforcing safety buffers.

API Endpoints:

Method	Endpoint	Authorization	Description
POST	/api/reservations	Driver	Create advance reservation (deducts deposit or sets PendingApproval)
POST	/api/reservations/walk-in	StationOwner / Staff	Immediate on-site walk-in customer admission (locks charger, DriverId=null)
POST	/api/reservations/admin	StationOwner / Admin	Create reservation on behalf of a driver without pre-payment
GET	/api/reservations	Authenticated	Paginated and filtered reservation list (role-segregated)
GET	/api/reservations/{id}	Authenticated	Retrieve reservation details, QR payload, and linked session/meter photos
GET	/api/reservations/availability	Authenticated	Query available booking slots for a charger and date
PUT	/api/reservations/{id}/cancel	Authenticated	Cancel reservation, releasing slot and executing refund if applicable
POST	/api/reservations/{id}/approve	StationOwner / Admin	Operator approves edge-case booking, capturing deposit and issuing QR
POST	/api/reservations/{id}/reject	StationOwner / Admin	Operator rejects edge-case booking, releasing slot with zero fee
PUT	/api/reservations/{id}	StationOwner / Admin	Modify reservation start/end window with overlap validation
DELETE	/api/reservations/{id}	StationOwner / Admin	Administrative hard deletion with historical record cleanup
POST	/api/reservations/staff-checkin	StationOwner / Staff	Validate scanned driver QR token and set status to CheckedIn

GET	/api/reservations/{id}/history	Authenticated	Retrieve complete audit trail of status transitions
POST	/api/charging-plan/generate	Driver	Generate AI-ranked charging itineraries with closing/maintenance buffer checks

Validation:

Prevented past-date or out-of-bounds bookings (enforcing the 7-day booking horizon).

Validated that EndTime > StartTime and duration adheres to allowable station minimums/maximums.

Enforced wallet solvency checks: driver must possess LKR 500.00 for immediate advance confirmation.

Strict role-based authorization ensuring drivers cannot approve their own reservations or access unowned records.

Business-Specific Operations:

Pessimistic Row Locking & Slot Exclusions: Handled concurrent booking attempts using PostgreSQL transactional isolation to eliminate double-booking race conditions between online drivers and on-site walk-ins.

Operational Buffer Detection: Implemented automated calculations comparing scheduled EndTime against station CloseTime and charger MaintenanceWindows.StartTime. Any session finishing within 15 minutes triggers mandatory operator review.

2.3 Database Work

Entities / Tables:

Reservations: Primary table tracking booking slots (Id, DriverId, ChargerId, VehicleId, StartTime, EndTime, ReservationQRCode, AdvanceDepositAmount, ApprovalRequired, ApprovalReason, Status, CreatedAt, UpdatedAt).

ReservationApprovals: Edge-case tracking table (Id, ReservationId, StationOwnerId, TriggerReason, BufferMinutes, Status, DecisionNotes, RequestedAt, DecidedAt).

ReservationStatusHistory: Immutable audit log (Id, ReservationId, OldStatus, NewStatus, ChangedByUserId, ChangedAt).

ChargingPlans: Stores multi-agent planning outputs (Id, WorkflowRunId, DriverId, Deadline, RankedItinerariesJson, CreatedAt).

Relationships:

Users - Reservations: Foreign key on DriverId (configured as **Nullable** to permit walk-in drivers).

Chargers - Reservations: Foreign key on ChargerId (Cascade restricted on active reservations).

Reservations - ReservationApprovals: Unique foreign key on ReservationId.

Reservations - ReservationStatusHistory: One-to-many relationship tracking complete state lifecycles.

Reservations - ChargingSessions: Direct linkage established upon QR check-in or walk-in admission.

Constraints:

GiST Exclusion Anti-Double-Booking Constraint.

Check Constraints: Enforced domain states: Status IN ('Pending', 'PendingApproval', 'Confirmed', 'CheckedIn', 'Cancelled', 'Completed', 'Rejected').

Unique Constraints: Enforced global uniqueness on ReservationQRCode.

EF Core Migrations:

Created migrations to introduce ReservationApprovals and schema fields (ApprovalRequired, ApprovalReason).

Updated DriverId column to be nullable to enable walk-in support.

Added indexes on (ChargerId, StartTime, EndTime) and Status to optimize availability queries.

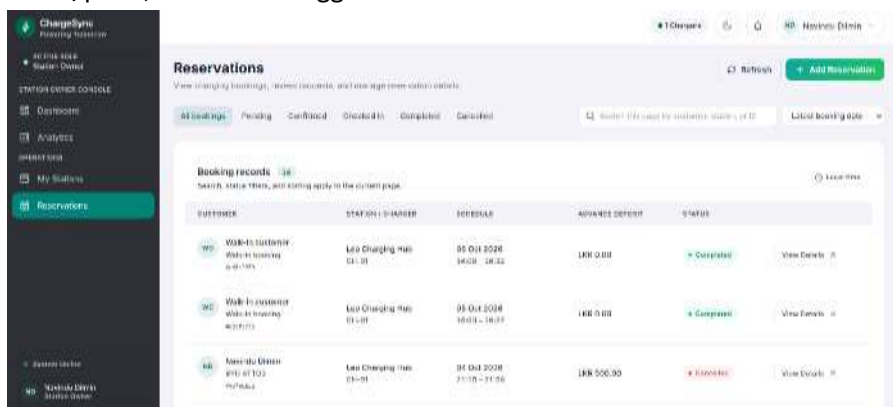
Seed Data:

Seeded initial reservations across test stations and chargers representing diverse states (Confirmed, PendingApproval, CheckedIn, Completed) with realistic timestamps to validate slot computation and reporting views.

2.4 React Work

Pages:

ReservationsPage.jsx: Comprehensive management view featuring tabbed navigation (All Reservations, Pending Approvals, Confirmed, Past Sessions), status badges, search by driver email/plate, and action triggers.



Components:

ReservationDetailsModal.jsx: Modal displaying complete booking telemetry, vehicle details, driver contact info, status progression timeline, QR code representation, and Cloudinary physical meter photo evidence attached to completed sessions for visual auditing.

AddReservationModal.jsx: Administrative booking modal allowing station staff to manually reserve a charger slot for a customer, featuring charger selection, date picker, and duration inputs.

PendingApprovalsCard.jsx: Dedicated dashboard widget alerting station owners to edge-case reservations requiring manual review, providing inline Approve and Reject buttons with optional justification notes.

API Integration:

useReservations.js: Custom hook utilizing TanStack Query (useQuery, useMutation) for fetching reservation lists, invalidating caches upon approval/rejection/cancellation, and polling pending approval counts.

Forms & Validation:

Built client-side validation in AddReservationModal verifying that selected dates fall within allowable operational windows and start times preceded end times.

Search / Filter / UI Logic:

Implemented client-side and server-side filtering by station, charger, reservation status, and date range.

Designed visual cues (amber warning badges for PendingApproval reservations, green for Confirmed, blue for CheckedIn).

2.5 Flutter Work

Screens:

ai_planning_screen.dart: Interactive route planning screen where drivers input destination, arrival deadline, max distance, and price preference; displays ranked itinerary cards generated by the Agentic AI.



create_reservation_screen.dart: Driver booking interface displaying charger specifications, cost calculations, available time slots, operational constraint warnings (if booking finishes near closing/maintenance), and the advance wallet confirmation modal.



reservation_list_screen.dart: Tabbed view for drivers showing upcoming, pending approval, and past reservations, including visual QR code tokens for easy check-in display.



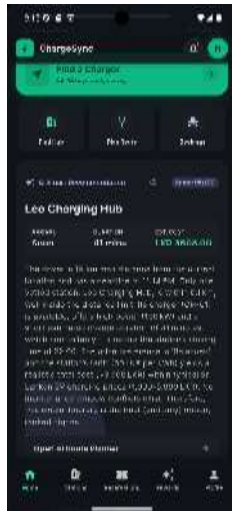
staff_dashboard_screen.dart / walk_in_booking_screen.dart: Dedicated staff POS interface enabling station attendants to quickly admit walk-in drivers, lock physical chargers, and monitor active bays.



qr_scanner_screen.dart: High-performance camera scanner for station staff, reading driver QR tokens, verifying authenticity via API, and instantly initiating the charging session.



smart_recommendation_card.dart: Driver home screen widget providing one-tap access to AI recommendations and displaying the agent's real-time reasoning.



API Integration:

Built `PlanningApiClient` handling JSON serialization/deserialization for `PlanningRequest` and `PlanningResponse`.

Implemented `ReservationService` in Dart interfacing with the ASP.NET Core endpoints with automatic bearer token injection and error handling.

Device Features:

Camera Access: Integrated `mobile_scanner` in `qr_scanner_screen.dart` with auto-pause and debounce controls to prevent scanner loop re-triggers.

Geolocation: Utilized geolocator to capture driver coordinates for distance calculations against candidate charging stations.

2.6 Agentic AI Work

Agent / Responsibility:

Owned the **Charging Recommendation & Planning Coordinator Agent**

(`PlanningCoordinatorAgent.py`), responsible for synthesizing driver constraints (deadline, budget, location) and orchestrating sub-agents (`VehicleCompatibilityAgent` and `StationAnalysisAgent`) to generate optimal, executable charging plans.

AI Workflow:

Step 1 (Ingestion & Normalization): Parses `PlanningRequest`, converts timestamps to Sri Lanka local time (UTC+05:30), and checks urgency (deadline ≤ 60 minutes).

Step 2 (Compatibility Screening): Calls `VehicleCompatibilityAgent` to filter out candidate chargers with incompatible connector pinouts or inadequate power levels.

Step 3 (Congestion & Station Analysis): Invokes `StationAnalysisAgent` to assess live utilization, historical availability scores, and tariff competitiveness. Highly congested stations are filtered out during urgent requests.

Step 4 (LLM Prompting & Itinerary Synthesis): Passes the vetted station pool to the LLM (Groq) using structured prompt templates to select and rank the top 2 distinct stations.

Step 5 (Deterministic Post-Processing & Guardrails): Programmatically overrides LLM duration and cost estimates using exact physics and math:

Actual Rate (kW) = min(Charger Power, Vehicle Max Charge Rate)

Duration (mins) = round((Battery Capacity (kWh) / Actual Rate (kW)) * 60)

Estimated Cost = round(Actual Rate * (Duration / 60) * Tariff, 2)

Step 6 (Operational Buffer & HITL Evaluation): Compares computed session completion time against station closing time and scheduled charger maintenance windows. If completion falls within 15 minutes of closing or maintenance, the coordinator automatically flags `requires_approval = True` and appends explanatory reasoning.

Tools Used:

evaluate_station: Compatibility tool checking connector hardware and maximum power rates.

analyze: Station analysis tool returning real-time congestion scores and pricing competitiveness.

Structured Input / Output:

Implemented using Pydantic models (PlanningRequest, PlanningResponse, ItineraryStep, AgentStationInput, AgentChargerInput).

Enforced structured output binding via `llm.with_structured_output(PlanningResponse)`.

Validation & Guardrails:

Replaced LLM mathematical hallucinations with deterministic calculations.

Standardized timezone parsing to guarantee that ISO-8601 UTC strings from ASP.NET Core match Sri Lankan local operational schedules.

Protected against prompt injection by passing station data as structured JSON contexts rather than unescaped natural text.

Human Approval (HITL):

When `requires_approval = True`, the plan alerts the driver that the selected slot requires operator review. When submitted to the backend, the reservation enters `PendingApproval`, deferring the advance wallet deposit until the station owner signs off.

Workflow State:

Managed state transitions across the coordinator pipeline, recording execution metadata and linking plans to database records via `WorkflowRunId`.

2.7 Testing Work

To ensure high reliability across critical booking transactions and AI recommendations, I implemented extensive test suites across all layers of the platform:

1. Backend Unit Tests (ASP.NET Core / xUnit):

ReservationServiceTests.cs (62 Tests): Thoroughly tested advance booking creation, wallet pre-auth deposit deductions, double-booking prevention, walk-in customer admissions, cancellation refunds, staff QR check-ins, time-slot updates, slot availability calculations excluding maintenance windows, and operator `ApproveAsync/RejectAsync` workflows including role-based security assertions.

ReservationTimezoneTests.cs (4 Tests): Tested UTC to local timezone conversions, boundary edge cases, 7-day advance booking horizons, and rejection of past time slots.

ChargingPlansControllerTests.cs (2 Tests): Verified driver constraint filtering, distance matrix calculations, station filtering based on operating hours, AI microservice client communication, and graceful fallback handling on service timeouts.

2. Agentic AI Tests (pytest):

test_planning_coordinator_agent.py (4 Tests): Verified multi-agent LangGraph workflow execution, operational buffer detection (15-minute closing and maintenance threshold), urgency-based station filtering, and deterministic charge duration calculations.

3. Web React Tests (Vitest & React Testing Library):

useReservations.test.jsx (9 Tests): Verified query fetching, pagination, mutation updates, cache invalidation, and pending approval polling.

ReservationDetailsModal.test.jsx (10 Tests): Verified modal rendering, time-slot editing, cancellation dialogs, deletion actions, and Cloudinary physical meter photo rendering.

ReservationsPage.test.jsx (9 Tests): Verified tab switching, status filtering, and the pending approvals banner.

AddReservationModal.test.jsx (3 Tests): Tested form validation, charger dropdown population, and submission handling.

4. Mobile Flutter Tests (`flutter\_test`):

planning_api_client_test.dart (10 Tests): Verified JSON serialization/deserialization for PlanningRequest and PlanningResponse, HTTP 200 handling, and error containment.

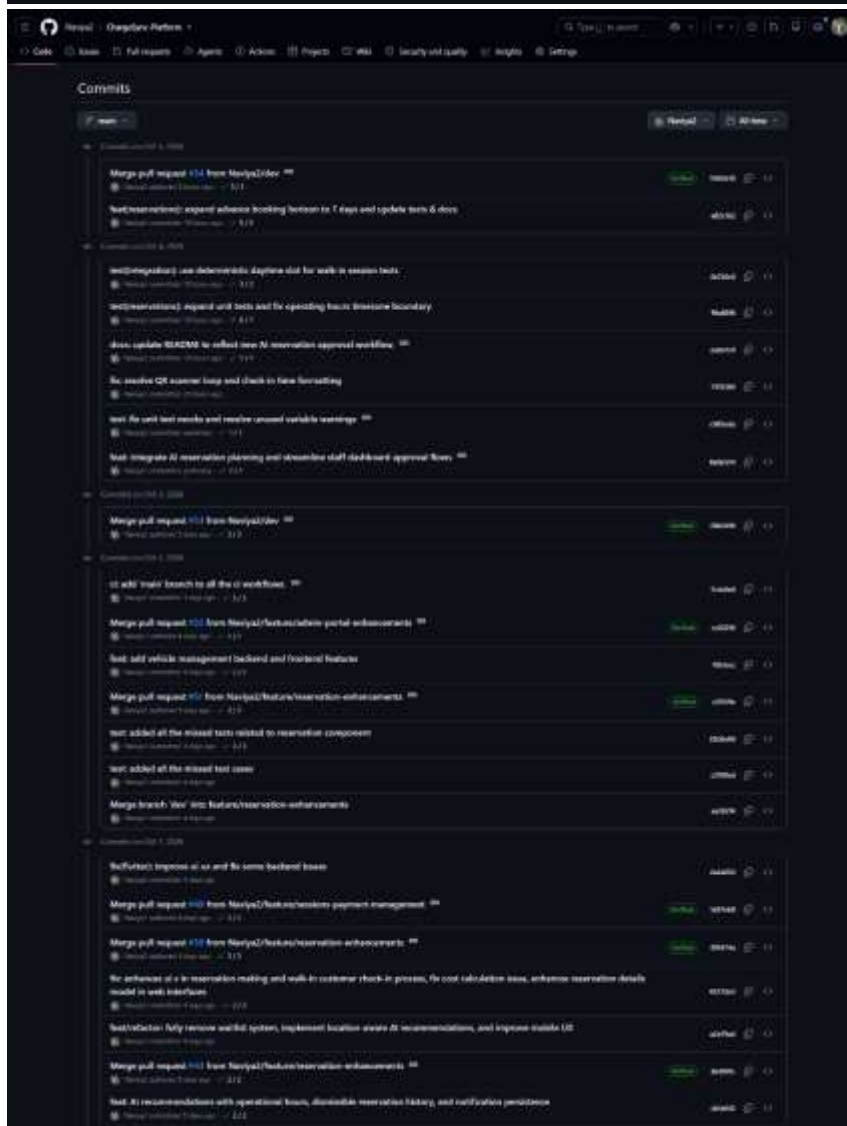
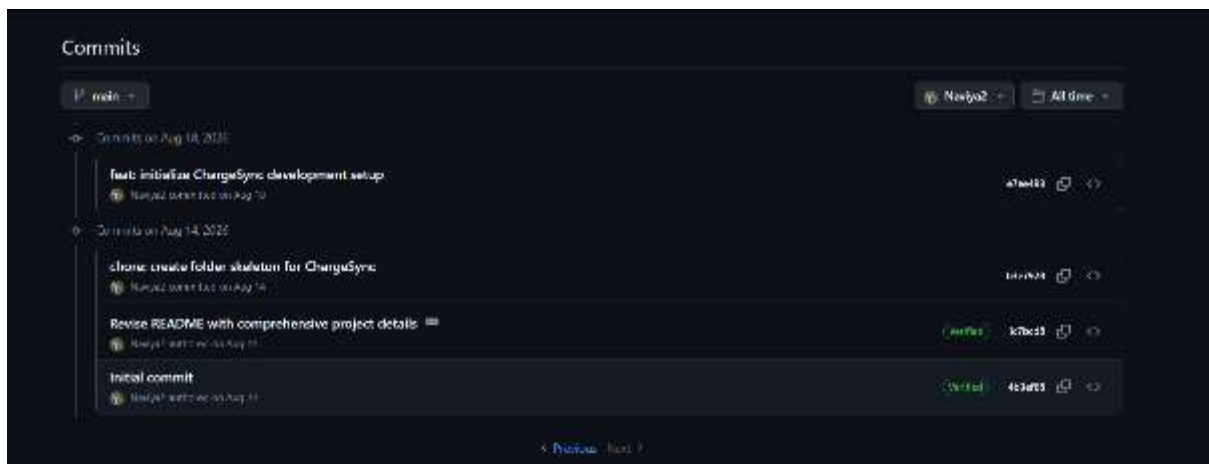
reservation_models_test.dart (4 Tests): Verified data model mapping for ReservationDto and CreateReservationRequest with requiresApproval attributes.

3. Key Commit, Pull Request, and Test Evidence

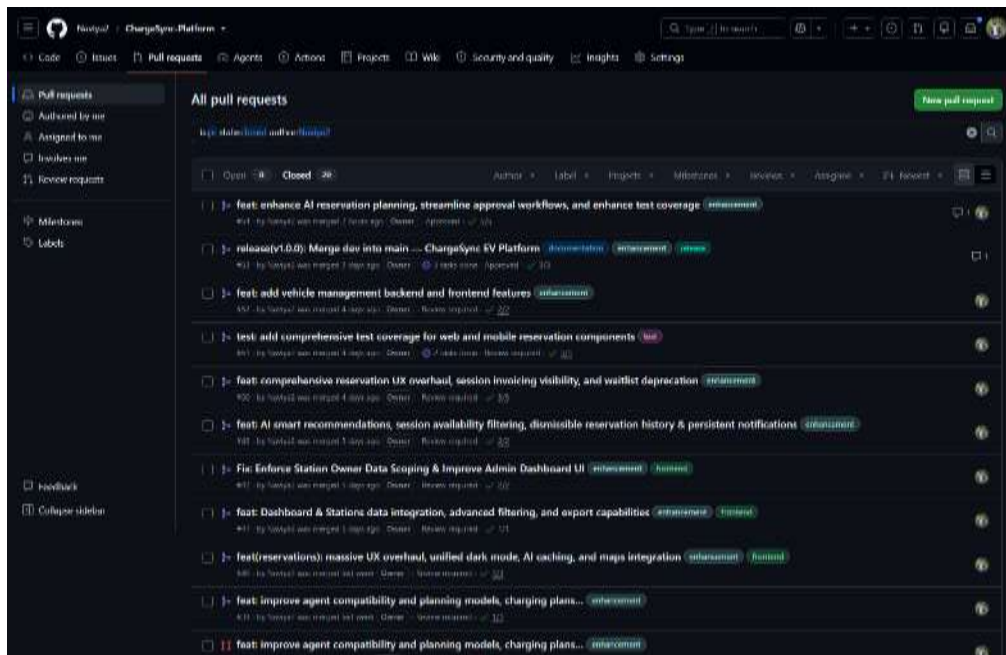
3.1 Key Git Commits

Commit Hash	Description / What Was Implemented	Evidence
a82c1b2	feat(reservations): expand advance booking horizon to 7 days and update tests & docs — Extended booking window logic, updated timezone validation boundaries, and synced unit tests.	https://github.com/Naviya2/ChargeSync-Platform/commit/a82c1b25dbee57b4da66a3ebd9d7506c1ee4611b
96a88f6	test(reservations): expand unit tests and fix operating hours timezone boundary — Resolved UTC/local timezone discrepancies in operating hours evaluation and added comprehensive test coverage.	https://github.com/Naviya2/ChargeSync-Platform/commit/96a88f61d0e3be15b4b323914b50c1f46e2b6be

9a0b574	feat: Integrate AI reservation planning and streamline staff dashboard approval flows — Implemented operator approval endpoints (ApproveAsync/RejectAsync), PendingApprovalsCard, and connected planning coordinator.	https://github.com/Naviya2/ChargeSync-Platform/commit/9a0b5742882bb824839c9a159f77d41ea2db8ab0
c2f519e	Merge pull request #51 from Naviya2/feature/reservation-enhancements — Merged end-to-end reservation and planning enhancements into dev.	https://github.com/Naviya2/ChargeSync-Platform/commit/c2f519ef6acf748d435e703c1f41d9d4d3497f18
32b0e90	test: added all the missed tests related to reservation component — Added comprehensive xUnit test coverage for reservation services, controllers, and edge-case workflows.	https://github.com/Naviya2/ChargeSync-Platform/commit/c2f519ef6acf748d435e703c1f41d9d4d3497f18
4327de3	fix: enhances ui s in reservation making and walk-in customer check-in process, fix cost calculation issue, enhances reservation details model in web interfaces — Refactored React reservation details modal, fixed Flutter check-in loop, and calibrated cost estimation.	https://github.com/Naviya2/ChargeSync-Platform/commit/4327de3a5396609ea0a3c16aa799bd3dd99db979
cb1a642	feat: AI recommendations with operational hours, dismissible reservation history, and notification persistence	https://github.com/Naviya2/ChargeSync-Platform/commit/cb1a6429a18e27093281cb85afacbe64371aab6f



3.2 Pull Request Evidence



3.3 Test Evidence

1. Backend Tests (ASP.NET Core / xUnit)

Reservation Core Logic & Approval Workflow (62 Tests):

```
PS D:\Github Projects\ChargeSync-Platform\backend> dotnet test tests\UnitTests\UnitTests.csproj --filter "FullyQualifiedName=ReservationServiceTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Github Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Github Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\Github Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\Github Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\Github Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
UnitTests -> D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll
Test run for D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 62, Skipped: 0, Total: 62, Duration: 965 ms - UnitTests.dll (net8.0)
PS D:\Github Projects\ChargeSync-Platform\backend>
```

AI Planning Controller & Station Availability Filtering (2 Tests):

```

PS D:\Github Projects\ChargeSync-Platform\backend> dotnet test tests\UnitTests\UnitTests.csproj --filter "FullyQualifiedNames-ChangingPlansControllerTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Github Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Github Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\Github Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\Github Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\Github Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
UnitTests -> D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll
Test run for D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 2, Skipped: 0, Total: 2, Duration: 25 ms - UnitTests.dll (net8.0)
PS D:\Github Projects\ChargeSync-Platform\backend>

```

Reservation API Integration Tests (3 Tests):

```

PS D:\Github Projects\ChargeSync-Platform\backend> dotnet test tests\IntegrationTests\IntegrationTests.csproj --filter "FullyQualifiedNames-ReservationEndpointsTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Github Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Github Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\Github Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\Github Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\Github Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
IntegrationTests -> D:\Github Projects\ChargeSync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll
Test run for D:\Github Projects\ChargeSync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 3, Skipped: 0, Total: 3, Duration: 247 ms - IntegrationTests.dll (net8.0)
PS D:\Github Projects\ChargeSync-Platform\backend>

```

2. Agentic AI Tests (Python / pytest)

```

tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_budget_preference
tests/test_planning_coordinator_agent.py::test_generate_plan_budget_preference
tests/test_planning_coordinator_agent.py::test_generate_plan_buffer_conflict_requires_approval
tests/test_planning_coordinator_agent.py::test_generate_plan_buffer_conflict_requires_approval
D:\Github Projects\ChargeSync-Platform\agentic-ai\agents\planning_coordinator_agent.py:47: DeprecationWarning: datetime.datetime.
utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: dat
etime.datetime.now(datetime.UTC).
    time_window=TimeWindow(start_time=datetime.utcnow(), end_time=datetime.utcnow() + timedelta(hours=1)),
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 4 passed, 12 warnings in 0.18s =====
D:\Github Projects\ChargeSync-Platform\agentic-ai>

```

3. Web React Portal Tests (Vitest)

Reservation Hook & TanStack Query Cache (9 Tests):

```
PS D:\Github\Projects\Chargesync-Platform\web-react> npx vitest run tests/unit/hooks/useReservations.test.jsx
v3.0.3 D:\Github\Projects\Chargesync-Platform\web-react
✓ tests/unit/hooks/useReservations.test.jsx (9 tests) 706ms
  ✓ useReservations hooks (9)
    ✓ useReservationsList calls api.list 100ms
    ✓ useReservationDetail calls api.get 71ms
    ✓ useReservationHistory calls api.getHistory 63ms
    ✓ useCancelReservation calls api.cancel 79ms
    ✓ useUpdateReservation calls api.update 88ms
    ✓ useDeleteReservation calls api.remove 82ms
    ✓ usePendingApprovalsQuery calls api.list with Pending status 80ms
    ✓ useApproveReservation calls api.approve 76ms
    ✓ useRejectReservation calls api.reject 64ms

Test Files  1 passed (1)
Tests       9 passed (9)
Start at: 18:34:45
Duration: 1.95s (tests 44%, environment 41%, setup 7%, import 4%, transform 8%, worker 1%)
PS D:\Github\Projects\Chargesync-Platform\web-react>
```

Reservation Details Modal & Meter Photo Evidence (10 Tests):

```
PS D:\Github\Projects\Chargesync-Platform\web-react> npx vitest run tests/unit/components/ReservationDetailsModal.test.jsx
act() -> {
  /* Fire events that update state */
}
/* assert on the output */

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

stderr | tests/unit/components/ReservationDetailsModal.test.jsx > ReservationDetailsModal > handles deletion workflow
An update to ReservationDetailsModal inside a test was not wrapped in act(...).

When testing, code that causes React state updates should be wrapped into act(...):

act() -> {
  /* Fire events that update state */
}
/* assert on the output */

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

✓ tests/unit/components/ReservationDetailsModal.test.jsx (10 tests) 261ms
  ✓ ReservationDetailsModal (10)
    ✓ renders reservation details correctly 30ms
    ✓ renders walk-in reservation details correctly 8ms
    ✓ displays meter photo if available 7ms
    ✓ allows editing reservation time 118ms
    ✓ handles cancellation workflow 30ms
    ✓ warns a driver about the late fee and shows the advance separately 15ms
    ✓ offers approval and rejection for pending requests instead of cancellation 18ms
    ✓ shows staff cancellation and both fee records without claiming a new fee 12ms
    ✓ handles deletion workflow 28ms
    ✓ lets a support viewer inspect a booking without mutation controls 8ms

Test Files  1 passed (1)
Tests       10 passed (10)
Start at: 18:35:39
Duration: 2.16s (import 46%, environment 32%, tests 15%, setup 5%, transform 5%)
PS D:\Github\Projects\Chargesync-Platform\web-react>
```

Reservations Management Page & Filter Tabs (9 Tests):

```
PS D:\Github\Projects\Chargesync-Platform\web-react> npx vitest run tests/unit/pages/ReservationsPage.test.jsx
v3.0.3 D:\Github\Projects\Chargesync-Platform\web-react
✓ tests/unit/pages/ReservationsPage.test.jsx (9 tests) 839ms
  ✓ ReservationsPage (9)
    ✓ renders loading state 36ms
    ✓ renders error state 36ms
    ✓ renders empty state 36ms
    ✓ renders reservations and allows search 178ms
    ✓ opens and closes Add Modal 188ms
    ✓ opens and closes Details Modal 136ms
    ✓ filters by booking status and can clear an empty filter result 138ms
    ✓ requests the next API page and preserves sorting controls 140ms
    ✓ keeps support managers out of staff-only creation and approval controls 11ms

Test Files  1 passed (1)
Tests       9 passed (9)
Start at: 18:36:35
Duration: 2.78s (import 18%, tests 91%, environment 8%, transform 8%, setup 4%)
PS D:\Github\Projects\Chargesync-Platform\web-react>
```

Add Reservation Modal & Validation (3 Tests):

```
PS D:\Github Projects\ChargeSync-Platform\web-react> npx vitest run tests/unit/components/AddReservationModal.test.jsx
RUN v5.0.3 D:\Github Projects\ChargeSync-Platform\web-react
✓ tests/unit/components/AddReservationModal.test.jsx (3 tests) 932ms
✓ AddReservationModal Integration (3)
✓ renders correctly and loads data 72ms
✓ allows selection and successful form submission 537ms
✓ displays error on failed submission 321ms

Test Files  1 passed (1)
Tests       3 passed (3)
Start at    18:37:12
Duration    2.08s (tests 47%, environment 36%, import 6%, setup 6%, transform 4%)

PS D:\Github Projects\ChargeSync-Platform\web-react>
PS D:\Github Projects\ChargeSync-Platform\web-react>
```

4. Mobile Flutter Tests (flutter test)

```
PS D:\Github Projects\ChargeSync-Platform\mobile-flutter> flutter test test/core/api/planning_api_client_test.dart test/core/api/reservation_models_test.dart test/reservation_charge_dialog_test.dart

A new version of Flutter is available!
To update to the latest version, run "flutter upgrade".

00:17 +14: All tests passed!
PS D:\Github Projects\ChargeSync-Platform\mobile-flutter>
```

4. Challenges and Learning

4.1 Challenges & Solutions

Technical Challenge	How You Solved It
Concurrency & Double-Booking Prevention Simultaneous reservation attempts by mobile drivers and on-site walk-ins could lead to double-booked charger bays.	Configured PostgreSQL GiST exclusion constraints (EXCLUDE USING gist) combined with pessimistic row-level locking (SELECT ... FOR UPDATE) in EF Core transactions, ensuring guaranteed hardware slot exclusivity at the database level.
Timezone Inconsistencies Across Stack Discrepancies between database TIMESTAMPTZ (UTC), station local operating hours (Sri Lanka Standard Time, UTC+05:30), and user mobile devices caused erroneous slot rejections at day boundaries.	Enforced an architectural rule where all database storage and API communication use UTC ISO-8601 timestamps, while local operating hours calculations apply an explicit TimeSpan.FromHours(5.5) offset during scheduling window comparisons in both C# and Python.
Operational Buffer & Deferred Deposit Flow Drivers booking slots finishing near station closing or scheduled maintenance risked being locked out or stranded without recourse if charged upfront.	Designed a Human-in-the-Loop (HITL) workflow: slots finishing within 15 minutes of closing or maintenance trigger an advisory banner, set PendingApproval, defer the LKR 500 deposit to LKR 0.00, and alert the station owner for explicit review.
LLM Hallucinations in Charge Durations & Costs General-purpose LLMs frequently generated inaccurate charging durations and unrealistic monetary estimates for Sri Lankan tariffs.	Implemented a deterministic post-processing layer in PlanningCoordinatorAgent.py. The LLM ranks candidate stations, but charging duration and cost calculations are programmatically

	overwritten using physical battery capacities and exact tariff rates.
Mobile QR Scanner Loop Re-triggering The Flutter mobile camera scanner continuously re-read the same QR code multiple times per second, triggering duplicate check-in API calls.	Implemented a scan controller debounce flag and state lock in qr_scanner_screen.dart, instantly pausing the camera feed upon successful detection until the API response completes.

4.2 Learning

ASP.NET Core Web API: Deepened expertise in enterprise Clean Architecture, custom authorization policy handlers, dependency injection lifetimes, and high-performance asynchronous transaction management.

PostgreSQL / EF Core: Mastered advanced relational database capabilities, including GiST temporal exclusion constraints, database indexing strategies, query execution profiling, and database migration lifecycles.

React / Frontend State: Gained hands-on experience in building scalable enterprise dashboards using TanStack Query for declarative caching, optimistic UI updates, and modular modal architectures.

Flutter / Mobile Development: Enhanced mobile development skills in state management using Riverpod, hardware camera sensor integration, geolocation capturing, and responsive cross-platform layout design.

Agentic AI: Acquired comprehensive understanding of multi-agent orchestration with LangGraph, structured Pydantic output parsing, prompt engineering, and implementing reliable guardrails around generative models.

Security (JWT / RBAC): Implemented strict role-based access control, cryptographic token generation for QR codes, and constant-time HMAC secret validation (X-Agent-Service-Key) for inter-service communication.

Testing, Git, and CI/CD: Mastered multi-tier automated testing (xUnit, pytest, Vitest, Flutter test), automated GitHub Actions workflow automation, and feature-branch Git collaboration.

5. Individual AI Usage Log

The following log summarizes the selective, human-directed utilization of AI tools during development, focused on syntax references and boilerplate scaffolding:

Date	AI Tool / Model	Task	How AI Was Used	Changes / Verification Applied
2026-09-24	GitHub Copilot / LLM	PostgreSQL GiST syntax	Consulted for syntax reference on PostgreSQL temporal GiST range exclusion constraints in EF Core.	Reviewed PostgreSQL documentation, manually wrote the migration script, and verified constraint behavior using

				concurrent unit tests.
2026-09-28	Claude 3.5 Sonnet	DTO Scaffolding	Generated initial C# record boilerplate for reservation and planning data transfer objects.	Heavily customized fields to match domain models, added validation annotations, and integrated with application interfaces.
2026-10-01	GPT-4o	LangGraph boilerplate	Referenced syntax for structured Pydantic output binding in LangChain.	Refactored into PlanningCoordinatorAgent.py, added custom deterministic duration overrides, and wrote pytest validation suites.
2026-10-03	Claude 3.5 Sonnet	Controller Security Review	Inquired about best practice patterns for role-based authorization attributes vs anonymous access on ASP.NET Core planning endpoints.	Enforced strict [Authorize(Policy = "Driver")] protection, updated the controller logic to securely pass candidate charger statuses, and validated via controller unit tests.
2026-10-04	ChatGPT	Environment Config Debugging	Consulted regarding environment variable precedence and configuration loading behavior in Python FastAPI / Pydantic BaseSettings.	Identified that the running process was missing local environment key context; configured .env file resolution and verified secure key loading into the LLM provider.
2026-10-05	GitHub Copilot	Flutter UI Card Layout	Looked up responsive layout	Implemented flexible container

			patterns to display dynamic-length AI reasoning text inside a card widget without layout overflow.	wrapping with null-safe text fallbacks and verified rendering across different mobile screen densities.
--	--	--	--	---

6. Individual AI Reflection

6.1 AI Tools and Development Stages

During the development of the ChargeSync platform, AI tools were utilized selectively as developer productivity aids rather than autonomous creators. They were primarily consulted during the early implementation phases for syntax verification, boilerplate DTO drafting, and quick technical documentation lookups.

All core architectural decisions—such as the hybrid operational model, the design of the Human-in-the-Loop operator approval gate, concurrency control strategies, database normalization, and business workflow logic—were human-engineered. During testing and refactoring, AI suggestions were strictly evaluated against domain requirements, with all final implementations hand-crafted and validated through rigorous automated tests.

6.2 What AI Did Well

AI assistance proved beneficial for accelerating repetitive boilerplate generation, such as drafting initial C# DTO records, creating basic Pydantic model schemas, and providing initial syntax patterns for complex framework APIs (e.g., LangGraph state schemas and Npgsql migration configurations). This reduced initial typing time and allowed me to focus on core domain algorithms, concurrency handling, and system integration.

6.3 What AI Got Wrong / What You Changed

AI models demonstrated significant limitations when dealing with multi-system business invariants and domain-specific edge cases:

1. Mathematical & Physics Hallucinations: When prompted to calculate EV charging durations, the LLM frequently hallucinated values that disregarded the relationship between battery capacity (kWh) and charger power output (kW). I resolved this by discarding LLM-generated durations and implementing deterministic mathematical calculations directly in Python.

2. Timezone Misunderstandings: AI suggestions frequently mixed UTC timestamps with local timezones, leading to bugs where operating hours checks failed at midnight boundaries. I replaced these with an explicit timezone offset architecture in both C# and Python.

3. Flawed Concurrency Assumptions: AI boilerplate initially suggested simple application-level if checks for reservation availability, which are vulnerable to race conditions under concurrent requests. I replaced this approach with database-level GiST exclusion constraints and pessimistic row-level locking.

4. Security Vulnerabilities: Suggested API endpoints initially lacked proper role-based authorization barriers, which would have allowed drivers to approve their own edge-case bookings. I implemented strict ASP.NET Core authorization policies (`AuthorizationPolicies.StationOwner`) to enforce separation of concerns.

6.4 Verification and Learning

Every piece of code introduced into the repository was subjected to a strict verification protocol:

1. Manual Architectural Review: Ensuring that all data flow adhered to the repository's Clean Architecture and separation-of-concerns principles.

2. Static Analysis & Linting: Running automated language analyzers to eliminate security flaws, unhandled exceptions, and unused dependencies.

3. Automated Test Validation: Writing and executing comprehensive xUnit, pytest, Vitest, and Flutter test suites covering happy paths, edge cases, and boundary conditions.

4. End-to-End Runtime Debugging: Verifying end-to-end communication across the mobile client, web dashboard, ASP.NET Core API, and Python agent service in live development environments.

This rigorous verification process deepened my technical mastery across full-stack engineering, concurrency control, and the critical importance of defensive programming when integrating AI components into mission-critical software systems.

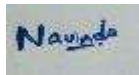
7. Signed Declaration

I hereby certify that the work and evidence presented in this report represent my own individual contributions to the project, created in accordance with university academic integrity standards.

Student Name: Samarawickrama N. A. N. D

Student ID: IT24103921

Signature: _____

A small, square image showing a handwritten signature in blue ink. The signature appears to be 'Navinda'.

Date: 05/10/2026